IT 494 · PHASE 1 · RESEARCH AND PROPOSAL

Persistent Memory for Stateless Models

A research package: proposal, literature, and candidate directions

Justin Hoffman
Illinois State University, MS Computer Science
Supervisor: Dr. Xing Fang

Assembled August 25, 2026. Literature summaries were produced with LLM assistance from the source documents and verified citations; the proposal is the author's own, drafted before the literature review. See the provenance note inside each part.

CONTENTS

Index

PART 1

IT 494 project proposal, pre-research draft

Drafted from my own notes and records for my correction. Not yet reviewed.

What I have running and what it has done

Every conversation I have with an AI is saved as a raw transcript and never edited. At the end of a working session a summary gets written and filed into a tree, organized by area of my life and then by topic. Those summaries roll up, so a higher level can answer a question without reading everything underneath it. A maintenance pass runs on a schedule: it folds in new material, audits a rotating subset of summaries against the original transcripts, and re-summarizes anything whose source changed after it was written. Nothing is deleted. Children fold up into parents and redundant siblings merge, so what has to be read stays bounded while the detail stays recoverable at the bottom. It is wired into my tools, so a new chat starts knowing the map exists and can search it.

This was built inartfully. I had no knowledge of the field when I started, so nothing about it follows a known design. It accreted as I hit problems, and it is a pile of scripts and text files rather than a system anyone would draw on a whiteboard. It is also in daily use and doing real work.

The clearest proof was annual training. I ran separate chats at the same time for the flight schedule, the storyboard, the weather update, and the situation report, all against one shared knowledge base. For the storyboards I pulled images and video out of email and matched them against the mission tracker, AMRS, and the flight schedule by their metadata, then built the boards from a cached template. Each of those threads would have exceeded a single conversation on its own, and they had to agree with each other. The D&D campaign I run is the same shape: long-running, many sessions, dependent on decisions made months ago staying findable and staying correct. When the system gets something wrong, I tell it and it corrects itself; I am entirely hands off, and I never touch the files myself.

The end state I am aiming at

The model is essentially a function. It takes a prompt and predicts the best answer: a decision engine. But an engine does not make an airplane. The memory system is the airframe, the part that carries state from one call to the next, and the airframe is what I am building. The end state in one sentence: a user level knowledge back end that can be scaled to an organization.

Today the backend serves one user, me. The organizational version I can picture concretely is a chat help desk that learns from its own conversations. An example from an AWS Summit talk: a utility answering billing questions with access to account information. Store the raw conversations, map how conversations actually progress, track what was concluded, digest it into long-term memory. Many users, one organization. My GAO work is why I think this is tractable rather than speculative. Pulling central ideas out of free human text worked. What failed was asking one tool to discover topics and enforce a rigid classification at the same time; separate those jobs, discovery on one side and the organization’s

own fixed categories on the other, and it works. The utility already has its categories in its dispute codes, the same way the agency had its survey indexes. I also came out of that work with methods for judging whether this kind of output is any good, and that is what I would bring.

The help desk has something my system does not: an outcome signal. Ticket closed, reopened, escalated, repeat contact. My system cannot learn that it gave a wrong answer. It records filing something in the wrong place, but a wrong answer leaves no trace, so there is nothing to measure improvement against. What I am really after is an efficient framework for ingesting data, organizing it, retrieving it, then injecting it, and maintaining it over time. I think in those five steps, and all five need to be built in phases. The idea is a living tree that ingests data over time, not an index built once.

What I do not know

I have no formal grounding in knowledge graphs or in persistent memory for AI, and I am unlikely to encounter either in a classroom. A model is stateless, so anything that appears to remember is machinery built around it: vector stores, retrieval-augmented generation, caching. I know roughly what these do. I do not know how they compare, where each breaks down, or what else exists. I do not even know that knowledge graphs are the way I want to go.

I do not have real metrics. I set up a fixed set of questions with known answers, and the system answers them, but the set is imperfect and I would not claim it measures much. So I cannot state the system's limitations with any confidence. Problems get patched as they surface, which is fine for my own use and is not evidence of anything.

I do not know whether my structural instincts hold up. I chose written summaries over similarity search on raw text, and the reason is merging. "Jenny is employed by GAO" and "Jennifer works at the Government Accountability Office" are the same fact, so some degree of merging base facts, or inferring until you reach the things that matter, has to happen; a hundred fact tokens might merge into a single attribute in a summary. How to approach that merging, and why, is one of the questions I want this project to answer. The merge cannot be a black box either: I would keep a ledger of which tokens, in sequence, merged into an attribute understanding, and a list of the keywords that all map to the same individual. It would have to understand pronouns too. That is where an LLM comes in, of course: distilling all of that from raw text. The bookkeeping around it does not need one. I chose a record where facts are never overwritten: new facts supersede old ones, and the superseding itself is part of what the system knows. I think the real trick is having the AI organize information on its own, a learned database. The open questions are easiest to see on a concrete corpus. Say the corpus is Harry Potter: which entities get their own pages? What attributes of a character exist on a page, and how do those change over time? How are characters grouped, by house, by school, by allegiance, and which groupings are most relevant for organization? How is all of it structured and searched through, and what data structures are we using? My first approach was chats at the leaves. Then it became clear the system needed to maintain knowledge about entities that exist outside any topic, me, my wife, my internship, my school, so the organization cannot be one tree; entities have to cut across it. How the rest is organized probably depends on how the text is ingested, and the use case is LLM chat logs and uploaded documents as the

corpus. I do not have settled answers past that point, and settling them defensibly is the design work. A drive can be as simple as observe, learn, adapt, improve, but the model does nothing unless directed: my schedulers fire on time, a drive would fire on condition, and my maintenance log already computes conditions it is not allowed to act on. Whether any of this matches how the field thinks about these problems, I cannot say. And one claim under the whole multi-chat use case gets checked before anything else: that concurrent sessions cannot share a context window regardless of its size.

How I would spend the semester

Research first, then the proposal: the near work is gathering resources and reading, and I will not commit to a design before that is done. Where I expect the time to go after that: taking the tree from something that works for me to something validated and tested. That would be a good use of the time, and I currently do not know how to do it. Working out how systems like this are measured, and by what, is what I need the reading for before I can claim anything about mine. One constraint stands regardless of direction, and it is the difference between the mock and the real thing. The mock was built without me examining the execution at all: I told the AI to build it, used the result, and told it to adjust when it was not working the way I wanted. There are about 2,300 lines of code in the current solution and I have not looked at a single line. That was the right way to prove the idea, and it is the wrong way to build an actual solution. For the real system I implement the execution and build the intermediate steps myself.

PART 2

Reading list

The Phase 1 working list, assistant-compiled from its survey with citations verified against source pages, for Justin's review. Each entry except the nine marked *library* names its reference copy in `papers/` and its one-pager in `summaries/one-pagers/` by slug; *library* entries have neither yet (the digest covers them from abstracts) and carry DOIs in `papers/MANIFEST.md` for an ISU library pull.

The advisor-meeting record of 2026-08-19 keeps the original 44 items with fuller annotations; this working index supersedes it and adds the ten knowledge-graph items requested on 2026-08-25.

Start here

1. Kumaran, Hassabis, McClelland 2016. What Learning Systems do Intelligent Agents Need? Trends in Cognitive Sciences 20(7). *library* · kumaran2016-cls-updated
2. Park et al. 2023. Generative Agents. UIST 2023. arXiv:2304.03442 · park2023-generative-agents
3. Sarthi et al. 2024. RAPTOR. ICLR 2024. arXiv:2401.18059 · sarthi2024-raptor
4. Rasmussen et al. 2025. Zep: A Temporal Knowledge Graph Architecture for Agent Memory. arXiv:2501.13956 · rasmussen2025-zep
5. Edge et al. 2024. From Local to Global: A Graph RAG Approach. arXiv:2404.16130 · edge2024-graphrag

Consolidation and agent memory

1. McClelland, McNaughton, O'Reilly 1995. Psychological Review 102(3). *library* · mcclelland1995-cls
2. Teyler, Rudy 2007. The hippocampal indexing theory. Hippocampus 17(12). *library* · teyler2007-hippocampal-index
3. Tononi, Cirelli 2014. Sleep and the Price of Plasticity. Neuron 81(1). *library* · tononi2014-shy
4. Zacks et al. 2007. Event perception. Psychological Bulletin 133(2). *library* · zacks2007-event-perception
5. Sumers et al. 2024. Cognitive Architectures for Language Agents. TMLR. arXiv:2309.02427 · sumers2024-coala
6. Rezazadeh et al. 2025. MemTree. ICLR 2025. arXiv:2410.14052 · rezazadeh2025-memtree
7. Talebirad et al. 2026. Toward a Theory of Hierarchical Memory. arXiv:2603.21564 · talebirad2026-hierarchical-theory
8. Wu et al. 2021. Recursively Summarizing Books with Human Feedback. arXiv:2109.10862 · wu2021-recursive-books

Retrieval, vector search, caching

1. Wang et al. 2024. Searching for Best Practices in RAG. EMNLP 2024. arXiv:2407.01219 · wang2024-rag-best-practices
2. Douze et al. 2024. The Faiss library. arXiv:2401.08281 · douze2024-faiss
3. Kuffo, Krippner, Boncz 2025. PDX. SIGMOD 2025. arXiv:2503.04422 · kuffo2025-pdx
4. Malkov, Yashunin 2016. HNSW graphs. TPAMI. arXiv:1603.09320 · malkov2016-hnsw
5. Chan et al. 2025. Don't Do RAG: Cache-Augmented Generation. WWW 2025. arXiv:2412.15605 · chan2025-cag
6. Modarressi et al. 2025. NoLiMa. ICML 2025. arXiv:2502.05167 · nolima2025-long-context
7. Jeong et al. 2024. Adaptive-RAG. NAACL 2024. arXiv:2403.14403 · jeong2024-adaptive-rag

Benchmarks and faithfulness

1. Maharana et al. 2024. LoCoMo. ACL 2024. arXiv:2402.17753 · maharana2024-locomo
2. Hu, Wang, McAuley 2025. MemoryAgentBench. arXiv:2507.05257 · wu2025-memoryagentbench
3. Chang et al. 2024. BooookScore. ICLR 2024. arXiv:2310.00785 · chang2024-booookscore
4. Kim et al. 2024. FABLES. COLM 2024. arXiv:2404.01261 · kim2024-fables

Knowledge graphs: foundations

1. Chen 1976. The Entity-Relationship Model. ACM TODS 1(1). *library* · chen1976-er-model
2. Angles et al. 2017. Foundations of Modern Query Languages for Graph Databases. ACM CSUR 50(5). arXiv:1610.06264 · angles2017-graph-query-foundations
3. Hogan et al. 2021. Knowledge Graphs. ACM CSUR 54(4). arXiv:2003.02320 · hogan2021-knowledge-graphs
4. Snodgrass 1999. Developing Time-Oriented Database Applications in SQL. Morgan Kaufmann, author-released PDF · snodgrass1999-time-oriented-db
5. Weikum et al. 2021. Machine Knowledge. FnT Databases. arXiv:2009.11564 · weikum2021-machine-knowledge
6. Pan et al. 2024. Unifying Large Language Models and Knowledge Graphs. IEEE TKDE. arXiv:2306.08302 · pan2024-unifying-llm-kg
7. Rossi et al. 2021. KG Embedding for Link Prediction. ACM TKDD. arXiv:2002.00819 · rossi2021-kg-embedding

Knowledge graphs: added 2026-08-25

Curated to ten from a verified 28-item survey, weighted toward representation and current practice; the remainder are listed at the bottom. Knowledge graphs are one candidate representation, not a commitment.

1. Angles 2018. The Property Graph Database Model. AMW 2018, CEUR Vol-2100. What a property graph formally is; the model closest to a tree of pages with typed links · angles2018-property-graph-model
2. Hernandez, Hogan, Krotzsch 2015. Reifying RDF: What Works Well With Wikidata? SSWS at ISWC 2015. The four ways to represent qualified, non-binary facts, compared on real data · hernandez2015-reifying-rdf-wikidata
3. Cai et al. 2024. A Survey on Temporal Knowledge Graph. arXiv:2403.04782. Where facts-valid-for-an-interval lives; representation chapters, not the embedding chapters · cai2024-temporal-kg-survey
4. Rost et al. 2021. Bitemporal Property Graphs to Organize Evolving Systems. arXiv:2111.13499. Valid time and transaction time on every node and edge, so corrections never erase history · rost2021-bitemporal-property-graphs
5. Fowler 2021. Bitemporal History. martinfowler.com. The practitioner bridge to the same idea. HTML · fowler2021-bitemporal-history
6. Vrandecic, Krotzsch 2014. Wikidata. CACM 57(10). The largest deployed store of time-scoped, source-attributed, supersedable facts · vrandecic2014-wikidata
7. Zhong et al. 2024. A Comprehensive Survey on Automatic Knowledge Graph Construction. ACM CSUR 56(4). arXiv:2302.05019. The construction pipeline end to end · zhong2023-kg-construction-survey
8. Noy et al. 2019. Industry-scale Knowledge Graphs: Lessons and Challenges. CACM 62(8). How Google, Microsoft, Facebook, eBay and IBM actually run them. Free HTML at queue.acm.org/detail.cfm?id=3332266 · noy2019-industry-scale-kgs
9. Xu et al. 2024. RAG with Knowledge Graphs for Customer Service Question Answering. SIGIR 2024 industry track. arXiv:2404.17723. LinkedIn's deployed graph, directly on the help-desk use case · xu2024-kg-rag-customer-service
10. Balog, Kenter 2019. Personal Knowledge Graphs: A Research Agenda. ICTIR 2019. The academic niche closest to the existing system · balog2019-personal-kg-agenda

Conversation mining and outcomes

1. Kecht et al. 2021. Event Log Construction from Customer Service Conversations. ICPM 2021 · kecht2021-event-log-nli
2. Gung et al. 2023. Intent Induction from Conversations, DSTC 11. SIGDIAL 2023. arXiv:2304.12982 · gung2023-dstc11-intent

3. De Raedt et al. 2023. IDAS: Intent Discovery with Abstractive Summarization. NLP4ConvAI at ACL 2023 · deraedt2023-idas
4. Wegmann et al. 2022. Same Author or Just Same Topic? RepL4NLP 2022. arXiv:2204.04907 · wegmann2022-style-content
5. Brynjolfsson, Li, Raymond 2025. Generative AI at Work. QJE 140(2). *library* · brynjolfsson2025-genai-at-work
6. Ouyang et al. 2026. ReasoningBank. ICLR 2026. arXiv:2509.25140 · ouyang2026-reasoningbank
7. Liu, Zhang, Choi 2025. User Feedback in Human-LLM Dialogues. EMNLP 2025. arXiv:2507.23158 · liu2025-user-feedback
8. Rezazadeh et al. 2025. Collaborative Memory. arXiv:2505.18279 · rezazadeh2025-collaborative-memory

Shared state across sessions

1. Erman, Hayes-Roth, Lesser, Reddy 1980. The Hearsay-II Speech-Understanding System. ACM CSUR 12(2). *library* · erman1980-hearsay-ii
2. Hayes-Roth 1985. A Blackboard Architecture for Control. Artificial Intelligence 26(3). *library* · hayesroth1985-blackboard-control
3. Salemi et al. 2026. LLM-Based Multi-Agent Blackboard System. arXiv:2510.01285 · salemi2026-blackboard-llm
4. Pollertlam, Kornsuwannawit 2026. Beyond the Context Window. arXiv:2603.04814 · pollertlam2026-beyond-context-window

Build references: added 2026-08-25 for the harness

Twenty-two sources surveyed and verified for the six gaps the harness build opened. These are stage references, consulted when building the stage they govern, not read-through material. Grouping matches the pipeline.

Contracts and cheap models: 54. Willard, Louf 2023. Efficient Guided Generation. arXiv:2307.09702 · willard2023-guided-generation / 55. Geng et al. 2023. Grammar-Constrained Decoding. EMNLP 2023 · geng2023-grammar-constrained / 56. Tam et al. 2024. Let Me Speak Freely? EMNLP 2024 Industry. arXiv:2408.02442 · tam2024-format-restrictions / 57. Chen, Zaharia, Zou 2023. FrugalGPT. arXiv:2305.05176 · chen2023-frugalgpt / 58. Ong et al. 2025. RouteLLM. ICLR 2025. arXiv:2406.18665 · ong2024-routellm / 59. Zhou et al. 2024. UniversalNER. ICLR 2024. arXiv:2308.03279 · zhou2024-universalner

Segmentation and extraction: 60. Hearst 1997. TextTiling. CL 23(1) · hearst1997-texttiling / 61. Koshorek et al. 2018. Text Segmentation as Supervised Learning. NAACL 2018 · koshorek2018-supervised-segmentation / 62. Xing, Carenini 2021. Dialogue Topic Segmentation. SIGDIAL 2021 · xing2021-dialogue-segmentation / 63. Niklaus et al. 2018. OpenIE Survey. COLING 2018 · niklaus2018-openie-

survey / 64. Zhang, Soh 2024. Extract, Define, Canonicalize. EMNLP 2024 · zhang2024-edc-kg-construction / 65. Caufield et al. 2024. SPIRES. Bioinformatics 40(3) · caufield2024-spires

Judging and agreement: 66. Zheng et al. 2023. Judging LLM-as-a-Judge. NeurIPS 2023 · zheng2023-llm-as-judge / 67. Wang et al. 2023. LLMs are not Fair Evaluators. ACL 2024. arXiv:2305.17926 · wang2023-not-fair-evaluators / 68. Panickssery, Bowman, Feng 2024. Self-Preference in LLM Evaluators. NeurIPS 2024 · panickssery2024-self-preference / 69. Kamalloo et al. 2023. Evaluating Open-Domain QA. ACL 2023 · kamalloo2023-open-qa-eval / 70. Artstein, Poesio 2008. Inter-Coder Agreement. CL 34(4) · artstein2008-intercoder-agreement

Narrative ground truth and injection: 71. Kocisky et al. 2018. NarrativeQA. TACL · kocisky2017-narrativeqa / 72. Pang et al. 2022. QuALITY. NAACL 2022 · pang2021-quality / 73. Wang et al. 2024. NovelQA. arXiv:2403.12766 · wang2024-novelqa / 74. Liu et al. 2023. Lost in the Middle. TACL 2024. arXiv:2307.03172 · liu2023-lost-in-the-middle / 75. Jiang et al. 2023. LongLLMLingua. ACL 2024. arXiv:2310.06839 · jiang2023-longllmlingua

Cut from the knowledge-graph survey, available on request

Named Graphs (Carroll et al., WWW 2005); RDF-star foundations (Hartig 2017); OWL 2 Profiles tutorial (Krotzsch 2012); the OneGraph vision paper (Lassila et al. 2023); LLM information-extraction survey; LLM-empowered KG construction survey (Bian 2025); entity-resolution overview (Christophides et al.); Open IE survey; Knowledge Vault (Dong et al. 2014); AutoKnow (Amazon, KDD 2020); GQL and SQL/PGQ pattern matching (SIGMOD 2023); a practitioner piece on why KG projects fail; the PKG ecosystem survey and PKG API tool paper; text-lifelog knowledge-base construction; LifeGraph; PIMO.

PART 3

The literature, work by work

Prepared by the assistant. One paragraph per work, for looking a paper up rather than reading straight through; the narrative document serves the read-through. Paragraphs compress the one-page summaries filed under summaries/one-pagers, which the assistant wrote from the papers themselves. Nine works whose full text sits behind publishers are marked as summarized from their abstracts; their identifiers are in the reading list for library access.

Start here

Kumaran, Hassabis, and McClelland 2016. Complementary learning systems updated for AI. A Trends in Cognitive Sciences review that revisits complementary learning systems theory with artificial agents in view. The architecture it defends has two stores: a fast episodic store that records individual experiences as they happen, and a slow structured store that holds generalized knowledge, with replay carrying material from the first into the second. Consolidation is that replay process; the fast store buffers new experience so the slow store can integrate it gradually rather than being disrupted by it. For a personal knowledge backend, this is the direct biological warrant for keeping raw conversation logs under a distilled summary layer and running a scheduled process that moves content between them. (Summary from published abstract; full text via library.)

Park et al. 2023. Generative Agents. A UIST paper that populates a Sims-like town, Smallville, with twenty-five LLM-driven agents to test whether an architecture around gpt-3.5-turbo can sustain believable long-horizon behavior. Each agent keeps a memory stream of timestamped natural language records; retrieval sums recency (exponential decay at 0.995 per game hour), LLM-rated importance (1 to 10), and embedding relevance. Reflection fires when recent importance passes 150, writing cited higher-level inferences back into the stream; recursive planning re-enters it too. In a 100-rater evaluation the full architecture scored TrueSkill mu 29.89 against 21.21 with no memory, and each ablated component cost believability; over two simulated days, party knowledge spread from one agent to thirteen with no hallucinated claims of knowing. For a personal backend, the ablations show a flat log with scored retrieval is not enough; the synthesized reflection layer did real work, and the authors' memory-hacking worry becomes an access-control question.

Sarthi et al. 2024. RAPTOR. An ICLR paper that replaces the flat chunk store of standard retrieval augmentation with a bottom-up summary tree: 100-token chunks are embedded, soft-clustered with Gaussian mixtures over UMAP-reduced embeddings, summarized by gpt-3.5-turbo, and the cycle repeats until clustering fails. Because clustering runs on embeddings rather than position, distant but related passages can share a parent. The preferred query strategy collapses the tree, letting every node at every layer compete in one cosine search under a 2,000-token budget. Adding the tree improved SBERT, BM25, and DPR on all three benchmarks tested; with GPT-4 it reached 82.6 percent on QuALITY against a prior best of 62.3, and about 4 percent of generated summaries contained minor hallucinations. For a memory backend, the lesson is that recall should span abstraction levels so each query picks its own

granularity; the index is batch-built, though, and an incremental variant remains the open engineering question.

Rasmussen et al. 2025. Zep. A preprint describing a commercial memory service and its engine Graphiti, a temporally aware knowledge graph built continuously from conversation. Messages persist as non-lossy episode nodes; an LLM pipeline extracts entities and facts, resolves duplicates against the existing graph, and maintains community summaries incrementally by label propagation. The distinctive mechanism is bi-temporal bookkeeping: every fact edge carries four timestamps covering when the system learned it and when it held true in the world, and contradicting edges invalidate rather than overwrite old ones. On LongMemEval, with conversations averaging 115k tokens, Zep beats full-context prompting by 18.5 percent with gpt-4o (71.2 versus 60.2) while cutting latency about 90 percent; on the small DMR benchmark, full-context stuffing nearly ties it, so the graph earns its cost mainly at scale. For a personal backend this is close to a reference design: episodic raw storage under a derived semantic layer, with explicit fact validity intervals answering knowledge that changes.

Edge et al. 2024. GraphRAG. A preprint targeting questions that need global understanding of a whole corpus, which are query-focused summarization tasks that fetching similar chunks cannot answer. An LLM extracts entities, relationships, and claims from 600-token chunks into a weighted knowledge graph; hierarchical Leiden community detection partitions it, every community at every level gets a pre-written summary, and queries map the summaries to partial answers in parallel before reducing them into one. On two roughly million-token corpora with GPT-4 as judge, every global condition beat vector RAG on comprehensiveness (72 to 83 percent win rates) and diversity, and root-level answers used 97 percent fewer context tokens than direct map-reduce summarization (26,657 versus about a million). Indexing cost 281 minutes of gpt-4-turbo per million tokens, with incremental update left open. For a personal backend this mechanizes summarize once, query summaries forever, and its persona-driven evaluation machinery transfers to any memory system lacking gold answers.

Consolidation and agent memory

McClelland, McNaughton, and O'Reilly 1995. Why there are complementary learning systems. The Psychological Review paper that founded complementary learning systems theory, the argument behind nearly every two-store memory design that followed. It lays out why a single learning system cannot both acquire new material quickly and hold structured knowledge stably, and answers with a division of labor: one system captures experience fast, the other integrates it slowly into organized knowledge. Kumaran, Hassabis, and McClelland's 2016 update carries the same architecture forward into the AI setting. For a personal knowledge backend, this is the original justification for splitting raw capture from consolidated knowledge rather than maintaining one store that must serve both jobs. (Summary from published abstract; full text via library.)

Teyler and Rudy 2007. The hippocampal index revisited. A Hippocampus paper updating hippocampal indexing theory. Its claim is that the hippocampus stores an index pointing back to the cortical activity patterns of an experience, not a copy of the experience itself; recall works by reactivating the original pattern through that pointer, so the index stays compact while the content remains where it first formed. The economy of the scheme is the point: one small structure can address a lifetime of

distributed detail. For a memory backend, the design translation is to build summaries and graph nodes as pointers carrying references back to source transcripts rather than duplicating content upward, so every derived record can be traced and re-expanded. (Summary from published abstract; full text via library.)

Tononi and Cirelli 2014. Sleep and synaptic homeostasis. A Neuron perspective arguing that a core function of sleep is synaptic downscaling: connections strengthened during waking are broadly weakened during sleep, and what survives the subtraction is what mattered. Consolidation, on this account, works by removal rather than addition; the signal improves because the noise is pruned away, and forgetting is part of the mechanism rather than a failure of it. For a personal knowledge backend, the offered idea is the maintenance pass: a store that only appends will degrade under its own accumulation, and a periodic offline pass that prunes, downweights, and compacts is what keeps the distilled layer trustworthy as an archive grows. (Summary from published abstract; full text via library.)

Zacks et al. 2007. Event segmentation theory. A Psychological Bulletin paper on event perception. People segment continuous experience into discrete events as it unfolds, and the boundaries where one event ends and the next begins are where memory gets organized; the segments themselves become the units that are later remembered. For a memory backend built over conversational history, this nominates the event, not the message and not the fixed-size chunk, as the natural unit of storage: segment sessions at topic boundaries and let those segments become the leaves the summary layers stand on. (Summary from published abstract; full text via library.)

Sumers et al. 2024. Cognitive Architectures for Language Agents. A TMLR conceptual paper, not an experimental one, arguing that LLM agents recapitulate the production-system lineage running through Soar, with the LLM as a probabilistic production system. CoALA describes any language agent along three dimensions: memory (working memory plus episodic, semantic, and procedural long-term stores), an action space split between external grounding and internal actions (retrieval, reasoning, learning), and a decision cycle of propose, evaluate, select. Its Table 2 casts SayCan, ReAct, Voyager, Generative Agents, and Tree of Thoughts into the frame and exposes what each lacks; from those gaps come research directions, with learned retrieval and deliberate unlearning named as nearly unstudied. For a personal backend the useful export is the memory typology plus the reflection step that turns accumulated episodes into reusable semantic facts, the move a conversation archive needs in order to scale past retrieval alone.

Rezazadeh et al. 2025. MemTree. An ICLR paper storing an agent's accumulating history as a dynamically grown tree, with abstraction increasing toward the root. New content descends from the root by cosine similarity against a depth-adaptive threshold, attaches as a new leaf where no child matches, and every ancestor summary is LLM-rewritten to fold it in; insertion is O(log N), and retrieval collapses the tree into one top-k search over all nodes. On the authors' 70-session MSC-E extension, MemTree (82.5) beats full-history prompting (78.0), and it roughly matches the offline RAPTOR on QuALITY (59.8 versus 59.0) despite building its index incrementally, at about 1.4x cumulative cost. Ancestor summaries drift with insertion order, and the tested corpora top out at hundreds of documents. For a conversational

backend the central lesson is that online versus offline matters more than the specific structure: a modest ongoing tax keeps the index current, where full-rebuild methods cannot.

Talebirad et al. 2026. A theory of hierarchical memory. An ICLR theory paper formalizing what RAPTOR, GraphRAG, and a dozen conversational and agent memory systems share: extraction of raw data into atomic units, iterated coarsening into a multi-level hierarchy, and budgeted traversal at query time. From information theory it proves that each lossy coarsening level strictly decreases entropy, so the atom layer caps what any query can recover from the levels above it. The central contribution is self-sufficiency, SS = I(G; rho(G)) / H(G), and its coupling to retrieval: self-sufficient summaries support collapsed search across all levels at once, referential ones demand top-down refinement, and mismatching the two wastes budget, though the coupling awaits a controlled test. Appendices bound branching factor and required depth from corpus size and context window. For a personal backend it supplies design vocabulary rather than results: choose summary style and search strategy jointly, and the depth bound predicts when one summary layer stops sufficing as a corpus grows.

Wu et al. 2021. Recursively summarizing books with human feedback. An OpenAI preprint training GPT-3 (6B and 175B) to summarize whole novels by fixed recursive decomposition: summarize chunks, then summaries of summaries, to depth 3, with one model handling every level, trained by behavioral cloning and then RL against human comparisons. Framed as scalable oversight, each subtask is judged locally against its input, making labels over 50x cheaper than end-to-end collection. On 40 unseen 2020 books the best summaries averaged well below human-written ones, but the 175B models set state of the art on BookSum, and depth-1 summaries let a 3B reader score competitively on NarrativeQA. The failures instruct: local context turned a dance request into a marriage proposal, and themes spread thinly across chunks vanished. For a memory tree, this is the canonical evidence that hierarchical summarization compresses arbitrarily long corpora, and its errors make cross-leaf context and audits at composition points design requirements, not refinements.

Retrieval, vector search, caching

Wang et al. 2024. Searching for best practices in RAG. An EMNLP paper treating the RAG pipeline as swappable modules and greedily optimizing one module at a time within a full pipeline, evaluated with Llama2-7B over 10 million Wikipedia passages plus 4 million medical texts. The cheapest win is a BERT query classifier deciding whether to retrieve at all: the average score rises from 0.428 to 0.443 while latency falls from 16.41 to 11.58 seconds per query. Best retrieval combines HyDE with hybrid sparse-dense search (BM25 weighted at alpha 0.3); monoT5 balances reranking quality and cost while TILDEv2 runs 225 times faster with some loss; reverse repacking, placing the best evidence nearest the query, beats other orderings. Two recipes result, best-performance and a balanced one at roughly 1.5 seconds per query. For a memory backend the exports are the when-to-retrieve gate, hybrid retrieval under a light reranker, and evidence ordering, offered as starting priors since the numbers come from public QA corpora and a 7B generator.

Douze et al. 2024. The Faiss library. The design document for Meta's open-source approximate nearest neighbor library, which organizes a dozen index types around two tools, vector compression and non-exhaustive search, under an embedding contract: the index's only job is to approximate exact search

under the metric the embedding makes meaningful. Benchmarks yield selection rules. Graph indexes (HNSW, NSG) win below roughly 1M vectors when memory allows; inverted-file indexes take over once compression must fit RAM; search time grows as N to the 0.29 at 50 percent recall and N to the 0.45 at 99 percent. A production deployment indexes 1.5 trillion vectors in an 83 TiB memory map at about a second per query. Faiss stores no metadata and handles deletion poorly, so a database layer belongs on top. For a memory backend the field guide is the sizing logic: a personal corpus sits below the threshold where indexing beats brute force, an organizational one crosses into graph-index territory, and the same code covers both.

Kuffo, Krippner, and Boncz 2025. PDX. A SIGMOD paper proposing a storage layout that groups vectors into blocks and stores each dimension's values contiguously within a block, the way Parquet holds columns, so that pruning algorithms which stop reading a vector once its partial distance disqualifies it can finally beat plain SIMD scans. Auto-vectorized PDX kernels averaged 2.0x over hand-written SIMD on the standard layout, and 5.5x when eight or fewer dimensions are read, the case pruning creates; ADSampling on PDX ran 4.6x its own horizontal version and up to 7.2x over FAISS. The companion PDX-BOND is exact and needs no preprocessing or transformation of the vectors, beating FAISS, USearch, and Milvus on exact search by 2.5x to 4.0x on average. For a memory backend the fit is direct: a store that appends embeddings daily cannot afford retraining on ingest, exact search stays feasible at personal scale, the IVF results carry to larger corpora, and retrieval cost turns out to be partly a systems problem.

Malkov and Yashunin 2016. HNSW. The TPAMI paper behind the graph index inside most current vector stores. Elements insert one at a time into layered proximity graphs, each element's top layer drawn from an exponentially decaying distribution; search greedily descends from the sparse top layer like a probabilistic skip list. A neighbor-selection heuristic keeps only candidates closer to the new element than to any already-selected neighbor, which preserves connectivity across cluster boundaries. The authors argue O(log N) search, and on the ann-benchmarks datasets HNSW beats Annoy, FLANN, FALCONN, and VP-tree by wide margins; against Faiss product quantization on 200M SIFT vectors it reaches higher recall at faster query times and builds in 42 minutes versus 11 hours, at more than double the RAM. For a conversational backend, insertion is truly incremental with no reshuffling, which fits an append-only history; the deferred problems, element update and deletion plus awkward distributed search, are exactly what surfaces as a store grows.

Chan et al. 2025. Cache-augmented generation. A WWW Companion paper proposing to skip retrieval entirely when the knowledge base fits a long-context window: preprocess the whole collection once into the model's key-value cache, persist that cache, and append only the query at inference time. With Llama 3.1 8B on SQuAD and HotPotQA corpora of 21k to 85k tokens, CAG beats the best RAG configuration in most settings (0.7951 versus 0.7676 on HotPotQA-small) and answers in 0.85 to 2.26 seconds against up to 92 seconds for re-encoding the reference text per query. The margin inverts at the largest corpus, where sparse RAG top-5 wins, tracking known long-context degradation. For a memory backend this marks one end of a design spectrum: below some corpus size, retrieval infrastructure is pure overhead, and the durable idea is a tiered design with hot, stable knowledge preloaded in cache and the long tail behind a retriever.

Modarressi et al. 2025. NoLiMa. An ICML benchmark of needle-in-a-haystack questions whose needles share almost no words with the question (ROUGE-1 overlap of 0.069 versus 0.905 for vanilla NIAH), so finding the fact requires a latent association, like knowing the Semper Opera House is in Dresden. Across 13 models claiming at least 128K context, short-context scores mostly exceed 84 percent, yet at 32K tokens 11 of 13 fall below half their base score; defining effective length as retaining 85 percent of base performance, most models sit at 2K or below, and GPT-4o reaches 8K of a claimed 128K. A single lexically similar distractor cuts GPT-4o's effective length to 1K. For a conversational backend the message is blunt: dumping transcripts into context fails well before the advertised window whenever the query is phrased differently than the original conversation, which is the normal case, so retrieval has to do the semantic bridging before the model sees anything.

Jeong et al. 2024. Adaptive-RAG. A NAACL paper that routes each question, before any answering begins, to one of three strategies: no retrieval, a single retrieve-then-read step, or an iterative loop interleaving reasoning with repeated retrieval. The router is a T5-Large classifier trained on labels generated automatically, by recording the cheapest strategy that answered each sampled query correctly and backfilling from dataset provenance. Averaged over six QA datasets with GPT-3.5, it reaches 50.91 F1 at 1.03 retrieval steps per query, matching the always-iterate baseline (50.87) at about a third of its 2.81 steps; per-query cost spans 0.35 seconds without retrieval to 27.18 with iteration. The classifier is only 54.52 percent accurate, and an oracle router reaches 62.80 F1, so routing quality is the current ceiling. For a memory backend the export is retrieval depth as a cheap per-query decision, and the self-labeling trick lets usage logs generate their own routing data at any scale.

Benchmarks and faithfulness

Maharana et al. 2024. LoCoMo. An ACL paper contributing 50 very long synthetic two-person conversations, averaging about 300 turns and 9,200 tokens over as many as 35 sessions, generated by persona-seeded gpt-3.5-turbo agents over temporal event graphs and then human-edited. Tasks cover QA across five reasoning types, event summarization, and multimodal dialogue generation. Every tested configuration sits far below the human QA benchmark of 87.9 F1; GPT-4-turbo manages 32.1. Longer context helps ordinary recall but collapses on adversarial questions (gpt-3.5-turbo-16k falls to 2.1 F1), hallucinating answers and misattributing speakers. The representational finding travels furthest: RAG over distilled per-turn observations beats RAG over both raw logs and session summaries (top-5 observations reach 41.4 F1), and accuracy degrades as more are retrieved. For a memory backend that ordering (distilled assertions over transcripts and summaries) and the precision-over-volume result are directly usable, and the adversarial collapse warns that context scale is not the same as memory.

Hu, Wang, and McAuley 2025. MemoryAgentBench. An ICLR benchmark that feeds context to memory agents incrementally, chunk by chunk inside user messages with memorize-this instructions, so storage mechanisms actually fire before any question arrives. It names four competencies: accurate retrieval, test-time learning, long-range understanding, and selective forgetting, tested through 2,071 questions over contexts of 103K to 1.44M tokens. No method masters all four. RAG wins retrieval (HippoRAG-v2 at 65.1 against the 49.2 backbone), long-context models win the holistic tasks (GraphRAG scores 0.4 on novel summarization), and forgetting breaks everyone: no system exceeds 28 percent on

multi-hop fact updates. Commercial memory products trail a plain long context badly (Mem0 at 21.1 and Zep at 24.0, versus 60.6 for GPT-5-mini). For a memory backend the finding is pointed: append-and-retrieve handles lookup but fails at superseding stale facts, so explicit update and consolidation machinery, newest-wins provenance with periodic re-summarization, is required equipment.

Chang et al. 2024. BooookScore. An ICLR study of book-length summarization comparing the two workflows any long-corpus compressor must choose between: hierarchical merging of chunk summaries versus incrementally updating a running summary. On a contamination-resistant corpus of 100 recent books averaging 190K tokens, 1,193 human span annotations yield an eight-type coherence error taxonomy, which a GPT-4 prompt then automates; the metric's 78.2 percent precision essentially matches the human annotators' own 79.7. Hierarchical merging is almost always more coherent, but Claude 2 with an 88K chunk lifts incremental updating from 78.6 to 90.9, so fewer compression steps largely close the gap; annotators still preferred incremental summaries for detail, 83 to 11 percent. Omission errors dominate the error distribution. For a memory backend this maps directly onto rolling-summary versus tree designs, argues for large update windows, and supplies a reference-free LLM-judge protocol for auditing summaries where no gold answers exist.

Kim et al. 2024. FABLES. A COLM paper reporting the first large-scale human evaluation of faithfulness in book-length summarization: 3,158 atomic claims decomposed from 130 summaries of 26 novels published in 2023 or 2024, labeled by annotators who had already read the books, collected for $5.2K. Claude-3-Opus leads at 90.9 percent faithful claims, yet only five of 130 summaries were entirely accurate. Unfaithful claims cluster on character and relationship states (38.6 percent) and events (31.5 percent), half requiring multi-hop reasoning to refute, and automatic verification largely fails: the best auto-rater, Claude-3-Opus reading the entire book, reaches 58.2 F1 at catching unfaithful claims, far ahead of BM25 retrieval at 24.9. Models also omit key events and over-weight book endings. For a memory backend the negative result matters most: retrieval-then-verify collapses on claims distributed across a long record, a summarizing archive inherits silent omission and recency bias, and the reader-annotator protocol is the workable audit template.

Knowledge graphs: foundations

Chen 1976. The entity-relationship model. (Summary from published abstract; full text via library.) Published in ACM Transactions on Database Systems 1(1), 1976, this paper introduces the entity-relationship model, the formalism of modeling data as entities and the relationships among them. It stands at the head of the lineage every graph data model in this digest descends from.

Angles et al. 2017. Foundations of graph query languages. A survey in ACM Computing Surveys 50(5) that organizes graph querying by feature rather than by language, bridging database theory and SPARQL, Cypher, and Gremlin. It fixes two data models, edge-labelled graphs and property graphs, and builds a feature hierarchy from basic graph patterns (equivalent to conjunctive queries) through complex patterns, regular path queries, and their navigational combinations, with worked queries in all three languages. The complexity catalog is the load-bearing part: basic patterns with projection are NP-complete in combined complexity but logarithmic space in data complexity; optional and difference lift complex patterns to PSPACE-complete; RPQs run in linear time O(|G| times |L|) under arbitrary-path

semantics yet turn NP-complete under simple-path semantics even in data complexity. For a personal backend, fixed small query templates over a growing graph sit in the tractable data-complexity regime, and multi-hop recall stays linear so long as queries return endpoints rather than enumerating paths.

Hogan et al. 2021. Knowledge graphs. A tutorial survey by seventeen authors in ACM Computing Surveys 54(4), the field's first unified introduction. It defines a knowledge graph inclusively as a graph of data accumulating real-world knowledge, then walks the full stack: data models (RDF-style edge-labelled graphs, property graphs, named-graph datasets), query languages, schema, identity, context, deductive and inductive reasoning, and the creation-to-publication lifecycle. It separates three schema kinds (semantic, validating, emergent), and its practice section fixes reference numbers: Freebase held over three billion edges at its 2015 shutdown, Cyc represents 900 person-years yielding 2.2 million facts, NELL extracted 120 million edges, and enterprise graphs share six traits including billion-edge scale and deliberately lightweight taxonomies. For a personal backend, graphs postpone schema commitment while scope evolves, the identity machinery decides when two mentions are one entity, and the lightweight-taxonomy finding cautions against over-engineering the ontology.

Snodgrass 1999. Developing time-oriented database applications in SQL. A Morgan Kaufmann book translating roughly 1600 temporal-database research papers into guidance for working SQL developers, organized around three orthogonal triples: temporal types (instant, interval, period), kinds of time (user-defined, valid, transaction), and kinds of statement (current, sequenced, nonsequenced). SQL-92 supports none of the temporal semantics, so the discipline falls entirely on the application, illustrated through case studies including Nykredit, a Danish mortgage bank tracking nine million loans with both valid time and transaction time on each row. The payoff is a repair procedure: when bad data surfaces, staff roll back to the transaction time of the error, read the valid-time history as it then stood, and fix it, with the fix itself logged because transaction time is append-only. That repair operation is exactly what a conversational memory backend needs when a stored belief turns out to be wrong.

Weikum et al. 2021. Machine knowledge. A 289-page survey in Foundations and Trends in Databases of how large knowledge bases are built and maintained automatically, organized as a four-phase pipeline: discovery, canonicalization, augmentation, and cleaning, with case studies of YAGO, DBpedia, NELL, Wikidata, and industrial graphs. Its engineering conclusions include a quality bar of error rates below 5 percent (ideally under 1), the tension between correctness and coverage resolved by seeding from premium sources before riskier open extraction, the primacy of canonicalized entities and types over everything built on them, and the finding that construction is a life-cycle with humans permanently in the loop. Public KBs hold millions of entities and up to billions of statements; proprietary ones run one to two orders larger. The wrap-up names the personal KB, an augmented memory built from a longitudinal digital footprint, as an open research problem, nearly a specification for a conversation-derived store.

Pan et al. 2024. Unifying LLMs and knowledge graphs. A roadmap survey in IEEE TKDE organizing roughly a hundred systems from 2018 to 2023 into three frameworks: KG-enhanced LLMs, LLM-augmented KGs, and synergized combinations. Its findings are documented trade-offs: pre-training

injection performs best but forbids knowledge updates without retraining, while inference-time retrieval handles changing facts at some performance cost; encoder-style completion methods cannot handle unseen entities while generative ones may hallucinate nonexistent ones. The probing evidence is sobering, since scaling fails to appreciably improve memorization of tail facts, and one causal analysis found models complete facts from positionally close words rather than knowledge-bearing ones. For a personal backend, the LLM-driven construction pipeline (entity discovery, coreference, relation extraction, iterative correction as in PiVE) is the machinery for distilling structured memory from transcripts, and the graph-traversal reasoning line (StructGPT, Think-on-Graph) shows path-citing answers, the auditability a memory backend would require.

Rossi et al. 2021. KG embedding comparative analysis. An ACM TKDD study that retrains 16 embedding-based link prediction models across three families from scratch, plus the rule-mining baseline AnyBURL, on five standard benchmarks, then correlates per-fact accuracy with structural features of the training graph. Three findings carry it. AnyBURL ranks at or near the top nearly everywhere while training in 100 to 1,000 seconds against 1 to 300 hours for embeddings; only ComplEx with N3 regularization is consistently comparable. Graph structure largely determines difficulty, and benchmark leakage is severe: a two-hop path-support score alone reaches 93.55 percent Hits@1 on WN18. Unreported score-tie policies produce incomparable numbers, with CapsE swinging from 73.01 to 1.93 Hits@1 on FB15k. A graph distilled from one person's conversations will be sparse and skewed, the regime where every tested model does worst, so cheap interpretable rules are the sensible start, and the tie-policy result is a standing caution for retrieval evaluation.

Knowledge graphs: representation, time, practice

Angles 2018. The property graph database model. A ten-page AMW paper giving the first formal definition of the model behind Neo4j, Neptune, and Cosmos DB, supplying Codd's three required components: a data structure, integrity constraints, and a query language. The structure is a tuple (N, E, rho, lambda, sigma) with partial labeling and property functions permitting multiple labels and multivalued properties; a schema tuple names node and edge types and a delta function whitelists which edge types may connect which node-type pairs, with four validity conditions for schema-instance consistency. Query semantics is given by translation to non-recursive safe Datalog with negation, shown by worked example, with recursion (hence path queries) out of scope. For a knowledge backend, the delta function is exactly the shape of a vocabulary contract for an extraction pipeline, the all-partial functions match a store growing before its ontology settles, and the four-predicate Datalog reduction is a clean mental model of what a graph database buys over a relational one.

Hernandez et al. 2015. Reifying RDF for Wikidata. A workshop paper at ISWC asking how to represent Wikidata's qualified statements in plain RDF, reduced to encoding quads (s, p, o, i) where the statement identifier i carries qualifiers. Four encodings (standard reification, n-ary relations, singleton properties, named graphs) are built from a February 2015 dump of 57.1 million quads and tested with 14 queries on five SPARQL engines. The results are unevenly instructive: singleton properties, the most concise scheme, broke four of the five engines because millions of unique predicates violated indexing assumptions; named graphs was fastest in 17 of 70 engine-query cases with reification and n-ary at 16

each, so no clear winner emerged; only Virtuoso handled all four models. For a memory backend, facts need provenance, temporal scope, and confidence attached, and this paper shows the reification choice determines which stores and query features remain usable, with quads plus a statement identifier as the clean conceptual core.

Cai et al. 2024. Temporal KG survey. An arXiv survey (2403.04782) of temporal knowledge graph representation learning, where facts are quadruples (head, relation, tail, timestamp), positioned as the first broad survey of the area. It catalogs roughly forty models from 2017 to 2024 in a ten-category taxonomy spanning transformation, tensor decomposition, graph neural networks, autoregression, temporal point processes, language-model methods, and few-shot learning, and documents the standing benchmark infrastructure: ICEWS14 with 7,128 entities, 230 relations, and 90,730 facts; GDELT with 2,278,405 facts over 2,751 timestamps. The task split running throughout, interpolation (completing missing past facts) versus extrapolation (forecasting future ones), demands different machinery. For a memory backend, interval-scoped facts and that split map directly onto recording when something was true and predicting what still is, and the few-shot chapter addresses the sparse new entities a personal graph accumulates; the survey is candid that scaling to real-world graph sizes remains open.

Rost et al. 2021. Bitemporal property graphs. A report (arXiv 2111.13499) on a one-year Oracle and University of Leipzig cooperation, presenting four connected artifacts: TPGM+, a bitemporal property graph model; T-PGQL, a temporal extension of PGQL; CGN, continuous event detection; and BiTeGra, a relational-backed prototype. The core is bitemporality carried down to the property level: every vertex, edge, and property value gets separate valid-time and transaction-time intervals, avoiding whole-element copies when one property changes. T-PGQL exposes the intervals through SQL:2011 period predicates and a FOR TX_TIME clause with AS OF, FROM TO, and BETWEEN variants for snapshot and range time travel. No benchmarks are reported. For a memory backend, the two-clock design distinguishes “he changed jobs in May” from “I learned this in July,” and FOR TX_TIME AS OF is ready-made vocabulary for auditing what the system believed at any past moment.

Fowler 2021. Bitemporal history. A pattern essay on martinfowler.com working one payroll example throughout: Sally is paid on her recorded $6000 salary, HR later reports a back-dated raise to $6500, then corrects it to $6400. Fowler treats time as two dimensions, actual history (what happened in the world) and record history (what the system believed when), his preferred terms for Snodgrass’s valid and transaction time. The machinery is small but exact: a query takes two time parameters, salaryAt(‘2021-02-25’, ‘2021-03-25’), and while actual history stops being append-only once retroactive change is allowed, record history stays append-only, yielding a reliable audit trail of the corrections themselves. He is explicit that bitemporality complicates systems and is avoidable when actions capture their inputs. For a conversation-derived backend, this is the pattern for correcting extracted facts without destroying the audit trail, and his generalization of record time into perspectives sketches multi-user ingestion.

Vrandecic and Krotzsch 2014. Wikidata. A CACM systems article by the project director and data model lead describing the collaboratively edited knowledgebase Wikimedia launched in October 2012 to end the duplication of facts across 287 language Wikipedias. The representational contribution is a

layered statement model rather than plain triples: statements carry qualifiers (“as of 2010”, “method: estimation”), references, and preferred or deprecated ranks so conflicting claims coexist, with “value unknown” distinguished from mere incompleteness, and properties themselves community-governed first-class pages. By August 2014 it held 15.8 million items and 43.2 million statements (23.2 million referenced), with about 90 percent of the half million daily edits coming from bots under human curation. A memory backend can adopt the qualifier-and-reference structure for provenance, the rank mechanism for coexisting contradictions, and stable never-reused IDs, while noting that Wikidata’s schema governance depends on a volunteer community an organization must replace with other curation.

Zhong et al. 2024. KG construction survey. An ACM Computing Surveys 56(4) survey of more than 300 methods for building knowledge graphs automatically, organized into acquisition (entity discovery, coreference resolution, relation extraction), refinement, and evolution. It defines a knowledge graph as G = {E, R, T, F_k}, arguing the background-knowledge component F_k, a rule set or schema, is what separates a knowledge graph from a mere data graph. The construction-from-text chapters fix the pipeline vocabulary, from named entity recognition and linking through open, schema-bound, and distantly supervised relation extraction, tracking each task from rules through CRFs to pre-trained language models, and catalog resources with sizes (Wikidata at 96M+ items, YAGO at 2B+ facts). Method coverage stops at BERT-era models. A memory system doing entity linking and coreference over chat logs runs exactly these tasks, and the flagged open problems, long intricate contexts, knowledge dynamics, and federated privacy, are the organizational-scale versions of the same work.

Noy et al. 2019. Industry-scale knowledge graphs. An experience paper in ACM Queue and CACM in which practitioners from Google, Microsoft, Facebook, eBay, and IBM describe their production graphs and pool recurring challenges. The scale numbers frame everything: Google holds roughly 1 billion entities and 70 billion assertions, Bing about 2 billion and 55 billion, and Bing’s single Will Smith entry is assembled from 108,000 facts across 41 websites against 200 Will Smiths in Wikipedia alone, which is why disambiguation ranks as the top industry challenge. The mechanisms are architectural: Facebook keeps provenance on every assertion and rebuilds the graph daily through an idempotent build, eBay funnels writes through a replicated log feeding specialized stores, IBM defers entity resolution to query time so context can settle names. Provenance-first assertions, late-binding disambiguation, and the log-plus-stores pattern all transfer to conversational facts, and the authors name personalized on-device graphs as an open direction.

Xu et al. 2024. KG-RAG for customer service. A five-page SIGIR industry-track paper from LinkedIn describing a deployed question-answering system over historical Jira tickets. Instead of chunking tickets as flat text, each ticket is parsed into a section tree (summary, steps to reproduce, root cause, comments) by rules plus a template-guided LLM, and trees are linked by explicit Jira references and thresholded title-embedding similarity, stored in Neo4j with embeddings in Qdrant. Query-time entity and intent parsing scopes retrieval to matching section nodes, then a generated Cypher query walks the winning ticket’s tree. With GPT-4 and E5 held fixed in both arms, MRR rises from 0.522 to 0.927 and Recall@3 from 0.640 to 1.000; in a randomized six-month team split, median resolution time falls 28.6 percent. The dual-level pattern, parse each unit into a tree then connect units, gives a backend

coherent subgraph retrieval instead of arbitrary chunks, with the hand-built template marking what must become automatic at scale.

Balog and Kenter 2019. Personal KG agenda. A four-page ICTIR position paper from two Google researchers defining the personal knowledge graph as a distinct research object: structured knowledge about entities of personal rather than general importance, with a mandatory spiderweb topology in which every node connects to one central user node. Three properties separate PKGs from general KGs: entities need not be prominent ("Mom's dentist"), entity information can be radically sparse, and relations are often short-lived. Four research questions cover sparse ephemeral representation, long-tail entity linking where linking, population, and NIL-detection intertwine, automatic population without curators, and continuous two-way integration with external sources; on evaluation the authors are blunt that no open PKG dataset exists. For a conversation-derived backend this is a checklist of the hard parts, and the spiderweb constraint is a concrete schema decision, though the paper says nothing about the multi-user case where the single central node no longer holds.

Conversation mining and outcomes

Kecht et al. 2021. Event logs from conversations via NLI. An ICPM paper that builds process-mining event logs from raw customer service conversations using zero-shot natural language inference: each message becomes the premise, a candidate topic or activity ("This example asks for iOS version") the hypothesis, and pre-trained BART returns an entailment probability, with per-label thresholds tuned by Matthews correlation on 100 to 300 hand-labeled messages. No model training occurs. On Twitter support corpora (288,828 AmazonHelp tweets, 231,683 AppleSupport, 88,774 SpotifyCares), 55 of 56 classifications exceed 0.72 MCC, and three discovery algorithms recover the same coherent process model from the exported XES log. Hypothesis wording is brittle: one rephrasing moved MCC from 0.53 to 0.91. The transferable finding is labeling economics, roughly 300 examples structuring 200,000-plus messages, a pattern that maps directly onto turning an organization's conversation archive into a typed event timeline, with the caveat that the event schema must be known in advance.

Gung et al. 2023. DSTC 11 intent induction. A SIGDIAL paper from AWS AI Labs describing a shared-task benchmark for inducing customer intents from raw transcripts, releasing three simulated call-center corpora (948, 1,000, and 2,000 conversations with 22, 29, and 39 intents, averaging 59 to 71 turns). Task 1 is intent clustering with the reference count withheld; Task 2 is open induction judged by training a downstream classifier on the induced schema and scoring it on an independent test set. The winner reached 69.8 and 76.3 accuracy against baselines of 55.8 and 63.6, with deep embedded clustering in nine of the top ten systems; reassigning one system's HDBSCAN noise points moved it from rank 33 to 9, exposing the metrics' own weakness. The methodological lesson for an archive backend is Task 2's framing: judge an induced topic structure by the system it trains, not cluster purity, and expect long-tailed request distributions to concentrate any induced taxonomy on a few head topics.

De Raedt et al. 2023. IDAS. An NLP4ConvAI paper on intent discovery that keeps the sentence encoder frozen and disambiguates in text: GPT-3 abstractively summarizes each utterance into a short label ("when is next payday"), bootstrapping demonstrations by labeling K-means prototypes first, then labeling remaining utterances with their 8 nearest already-labeled neighbors as in-context examples.

Utterance and label encodings are averaged, smoothed, and clustered at the true K. Against MTP-CLNN, the unsupervised state of the art, IDAS gains an average of 3.94 ARI across Banking, StackOverflow, and a private transport dataset; on CLINC it reaches 79.02 ARI with zero labels, beating two semi-supervised methods given labels for half the intents. Ablations show any use of generated labels beats raw encodings by 5 to 19 ARI points. For organizing an archive by meaning rather than surface form, summarize-then-embed is the transferable move, while the known-K assumption and per-utterance LLM cost are the constraints that bite as an archive grows.

Wegmann et al. 2022. Style versus content. A RepL4NLP paper attacking a confound in writing-style embeddings: authorship-verification models can score well by memorizing what an author writes about rather than how. The fix is a contrastive setup with controlled distractors, sampling the different-author candidate from the same conversation so topic cannot separate authors, trained on 210k tasks from 100 subreddits. The decisive probe, STEL-Or-Content, pits a same-style paraphrase against a same-content different-style sentence: base RoBERTa picks style 5 percent of the time, the conversation-controlled model reaches .42 on the formal/informal dimension, and clustering its embeddings surfaces concrete signatures like terminal punctuation and lowercase i. For a backend embedding utterances, the lesson is that same-source signals silently entangle who is speaking with what they discuss; conversation structure supplies the hard negatives for free at archive scale, and building probes where the shortcut and the target disagree is a discipline persistent-memory evaluation mostly lacks.

Liu, Zhang, Choi 2025. User feedback in human-LLM dialogues. An EMNLP paper asking whether implicit feedback in chat logs (a user saying “could you label the y-axis?”) can be detected and used for training. The authors densely annotate 109 conversations from LMSYS-chat-1M and WildChat with a six-way ontology (Cohen’s kappa 0.70 binary), finding feedback in more than half of later user turns, skewed heavily negative, with refusals explaining under 3 percent; prompts drawing praise are slightly more toxic, traced to users celebrating jailbreaks. Their detector roughly doubles prior binary accuracy (41.6 versus 31.5 percent). Training on feedback-conditioned regenerations is mixed: it wins on MTBench (6.86 versus 6.38) but loses on WildBench, and plain distillation does best. For a memory backend, later turns are dense with correction signal marking unmet intent, exactly what to weight when inferring what a user wants, but the signal cannot be consumed raw at any scale.

Brynjolfsson et al. 2025. Generative AI at work. (Summary from published abstract; full text via library.) A field deployment of AI assistance in customer support, published in the Quarterly Journal of Economics 140(2), 2025, measured on resolution outcomes. It stands in this digest as the outcomes-side evidence that conversational AI assistance in support work can be evaluated on what issues actually get resolved, the deployment-grade measurement standard an organizational memory backend would be held to.

Shared state across sessions

Erman et al. 1980. Hearsay-II. (Summary from published abstract; full text via library.) The Hearsay-II speech understanding system, described in ACM Computing Surveys 1980, introduced the blackboard architecture: independent knowledge sources coordinating through shared structured state rather than direct calls to one another. It is the origin of the shared-state pattern the modern multi-agent

work in this section revives, and its coordination-through-a-common-store design is the ancestral form of any memory layer multiple agents read and write.

Hayes-Roth 1985. A blackboard architecture for control. (Summary from published abstract; full text via library.) Published in Artificial Intelligence 1985, this work extends the blackboard architecture from the domain problem to control decisions themselves: the choice of what to do next is also mediated through shared structured state. For a shared memory backend, it marks the step from a common store of facts to a common store that also coordinates which contributor acts when.

Ouyang et al. 2026. ReasoningBank. An ICLR 2026 memory framework for LLM agents facing task streams without ground-truth feedback. Each finished episode is distilled into a structured item (title, one-sentence description, content holding the strategy or pitfall); an LLM judge labels trajectories success or failure, and both outcomes feed extraction, successes as validated strategies and failures as preventative lessons. Retrieved items are injected into the system instruction; memory-aware test-time scaling spends extra rollouts per task to curate better items. On WebArena with Gemini-2.5-flash, success rises from 40.5 to 48.8, and to 51.8 with scaling at k=5; SWE-Bench-Verified resolve rate rises 34.2 to 38.8, with steps per task dropping up to 26.9 percent on successful runs. The judge is only 72.7 percent accurate, yet results barely move across simulated accuracies of 70 to 90 percent. That distilled, self-contained lessons beat raw transcripts, with curation running on noisy self-judgment, is what unattended accumulation requires, though consolidation here has no pruning.

Rezazadeh et al. 2025. Collaborative Memory. An Accenture framework (arXiv 2505.18279) for sharing memory across users and agents without leaking beyond entitlements. Permissions are two time-varying bipartite graphs (user-to-agent, agent-to-resource); memory splits into private and shared tiers; every fragment carries immutable provenance (time, contributing user and agents, resources touched), and a fragment is readable only if its provenance falls within current permissions, so revoking an edge retroactively hides dependent fragments. On MultiHop-RAG with five users and six domain agents, fully shared memory holds accuracy above 0.90 while cutting external resource calls by up to 61 percent at 50 percent query overlap; a science-QA scenario shows accuracy tracking grants and revocations with no logged fragment crossing a boundary. The provenance-stamped fragment plus retrospective permission check is exactly the primitive a single-user memory tree needs to grow organizational tenants, and the call-reduction result quantifies memory's role as a cache displacing repeated retrieval.

Salemi et al. 2026. Blackboard LLM system. An arXiv paper (2510.01285) reviving the Hearsay-II blackboard for LLM multi-agent data discovery: answering data science questions when the relevant files sit unidentified in a large heterogeneous data lake. Instead of a master assigning subtasks, the main agent posts requests to a shared board; helper agents, each pre-assigned one cluster of the lake and having studied it offline, volunteer only when they judge themselves capable. Across KramaBench and discovery-augmented DSBench and DA-Code, the blackboard beats RAG and an otherwise-identical master-slave variant by 13 to 57 percent relative, with file-discovery F1 of 0.561 versus 0.247 for embedding retrieval, and the advantage growing with lake size (211 percent relative on a 1,556-file lake) at about 2.3 times RAG's cost. Routing by self-selection beats a central index of who knows what because

the coordinator's model goes stale; partition, pre-digest, and broadcast is a pattern that scales in the direction an organization's archive grows.

Pollertlam and Kornsuwannawit 2026. Beyond the context window. A cost-performance study (arXiv 2603.04814) comparing fact-based memory (the Mem0 pipeline: extracted atomic facts, pgvector retrieval, top-20 into a GPT-5-mini reader) against resending full history to long-context models, on LongMemEval, LoCoMo, and PersonaMem v2. Long-context wins factual recall decisively (92.85 versus 57.68 on LoCoMo, 82.40 versus 49.00 on LongMemEval), but the gap shrinks to 7.3 points on stable persona attributes. The economics invert with use: extraction compresses a 101,600-token history roughly 35:1 for a one-time $0.0430 write, then about $0.0013 per query, undercutting cached long-context after about ten turns at 100k tokens (nine at 500k). The error analysis names what compression loses: precise temporal markers, implicit coreferences, and updates that never propagate. For a memory backend this supplies the economic argument and its price, and the three failure modes are a concrete checklist for any fact schema.

PART 4

The field, by problem

This document is a narrative of the literature on persistent memory for language systems, organized not by method or year but by the problems the work attacks. Each of the eight sections takes one problem, from whether a context window can substitute for memory at all to what signal should teach a memory once it exists, and traces the approaches to it work by work and date by date, so the field's movement is visible inside each section rather than asserted over the whole. The corpus is the project's reading list, minus the cognitive-science foundations and Chen 1976, treated in the digest only. The assistant wrote it from the one-page summaries, each drawn from the paper itself except nine paywalled works characterized from abstracts (three appear here: Brynjolfsson, Li and Raymond; Erman and colleagues; Hayes-Roth); no claim goes beyond what those sources state. Read the sections in order for the argument, since each hands a question to the next; read any one alone for a briefing on its problem, since each closes with what its own papers admit is unsettled. A short coda gathers the observations that recur across the sections.

Can a Context Window Substitute for Memory?

The simplest way to make a language model persistent is to give it no memory at all: resend everything it needs to know, every time. The literature of the past two years is largely an argument over how far that trick stretches, and the argument began not with context windows but with retrieval. Wang and colleagues, in their 2024 EMNLP study of RAG pipelines, treated the whole retrieve-and-generate stack as a sequence of swappable modules and searched greedily for the best method at each position. Buried in their module-by-module results is the finding that matters most here: a small BERT classifier deciding whether to retrieve at all, with 0.95 accuracy, raised the pipeline's average score from 0.428 to 0.443 while cutting latency from 16.41 to 11.58 seconds per query. The cheapest win in the whole search was not a better retriever but a gate in front of it.

Jeong and colleagues pushed that gate further the same year. Their Adaptive-RAG, presented at NAACL 2024, routes each question to one of three strategies, no retrieval, one retrieval step, or an iterative retrieve-and-reason loop, using a 770M-parameter T5 classifier trained on labels the system generates for itself: run all three strategies and record the cheapest one that succeeded. On GPT-3.5-Turbo-Instruct the router matched the always-iterate baseline (50.91 versus 50.87 F1) at roughly a third of its retrieval steps, and the timing spread they report, 0.35 seconds for a no-retrieval query against 27.18 for multi-step, makes the case in seconds rather than points. Their own caveat is the honest one: the classifier is only 54.52 percent accurate, and an oracle router would reach 62.80 F1, so routing quality, not the strategies themselves, is the current ceiling.

If retrieval needs a gate, the opposite temptation is to skip retrieval entirely and preload everything. Chan, Chen, Cheng, and Huang formalized this in 2025 as cache-augmented generation: encode the whole corpus once into the model's key-value cache, persist that cache, and at inference time append only the query. On corpora of 21k to 85k tokens with Llama 3.1 8B, CAG beat both sparse and dense RAG

in most configurations and generated answers in 0.85 to 2.26 seconds where re-encoding the text per query took up to 92.1 seconds. But their own numbers mark the boundary: at their largest HotPotQA setting CAG fell below sparse retrieval, and the authors state plainly that the method is impractical beyond small collections. The KV cache turns inference state into a persistable artifact, which is genuinely memory-adjacent engineering, but it does not repeal the limits of the window it fills.

Those limits turned out to be far tighter than the specifications claim. Modarressi and colleagues showed at ICML 2025, with their NoLiMa benchmark, that needle-in-a-haystack tests had been measuring pattern matching, not memory: their questions share almost no words with the hidden fact (ROUGE precision of 0.069 against 0.905 for vanilla NIAH). Under that condition, 11 of 13 models claiming 128K contexts fell below half their base score by 32K tokens, and by their 85-percent criterion most models have an effective length of 2K or less; GPT-4o, the best, managed 8K. The same Llama 3.1 70B that reaches 32K effective length on RULER manages 2K here, which isolates the crutch precisely. This result sits directly against the CAG proposal from the same year: a preloaded 85k-token cache is comfortably inside the region where associative recall has already collapsed, so the two papers agree on latency and disagree, in effect, on what the resident context is good for.

Pollertlam and Kornsuwannawit closed the loop in 2026 by pricing the whole trade. Comparing full-history long-context inference against a Mem0-style fact-extraction store, they found long-context GPT-5-mini winning factual recall by 33 to 35 points on LoCoMo and LongMemEval, while extraction compressed a 101,600-token history roughly 35:1 and made the memory system cheaper after about ten turns even with a 90 percent caching discount granted to the long-context side. Their error analysis names what compression loses: temporal markers, coreferences, and updates that never propagate. Accuracy favors resending everything; economics favors extraction; neither side wins both.

What remains unsettled follows from the papers' own scope statements. NoLiMa's collapse is measured on synthetic needles in book text, not real dialogue, and Pollertlam and Kornsuwannawit caution that their accuracy gap is a fact about flat extraction, not about structured memory designs they did not test. Adaptive-RAG's gap between its 54.52 percent router and its oracle leaves open whether when-to-retrieve is learnable well enough to be trusted, and no work here evaluates any of this against a live, incrementally updated store rather than a frozen snapshot.

When Does a Memory Need an Index, and What Does One Cost?

Any memory system that stores more than it can reread must eventually answer a mechanical question: given a query embedding, how do you find the nearest stored vectors without scanning everything? The modern answer begins with Malkov and Yashunin, who introduced Hierarchical Navigable Small World graphs (HNSW) in work first circulated in 2016. Their construction inserts elements one at a time, assigns each a maximum layer drawn from an exponentially decaying distribution, and builds a proximity graph at every layer the element occupies; a search enters at the sparse top layer, greedily descends to a local minimum, drops down, and repeats, so the whole structure behaves like a probabilistic skip list whose linked lists have been replaced by proximity graphs. Their second contribution, a neighbor-selection heuristic that keeps only candidates closer to the new element than to

any already-chosen neighbor, is what preserves connectivity across cluster boundaries. The empirical results were decisive: wide margins over Annoy, FLANN, FALCONN, and VP-tree on the standard benchmark datasets, and, on one non-metric benchmark, nearly three orders of magnitude fewer distance computations than the flat NSW graph it descended from. Two properties matter for memory work in particular. Insertion is genuinely incremental, which suits an append-only conversational history, and the index handles arbitrary metrics. But the paper explicitly defers element update and deletion to future work, and its complexity claims rest on assumptions the authors could only verify at low dimension; at d=128 they concede the check would need infeasibly large datasets.

What practitioners actually deploy is best read from Douze and colleagues' 2024 account of Faiss, Meta's library that packages HNSW alongside a dozen other index types. Its value here is less any single algorithm than the selection rules its benchmarks establish. Graph indexes win below roughly one million vectors when memory is unconstrained; inverted-file indexes become the only option once compression is needed to fit in RAM; graph quality peaks at 64 edges per node. The comparison cuts both ways against the 2016 results: on a 200-million-vector SIFT subset, Malkov and Yashunin had already shown HNSW beating Faiss product quantization on recall and build time (42 minutes against 11 to 12 hours) while paying for it in RAM (64 Gb against under 30). Douze and colleagues are also blunt about what Faiss is not: it stores no metadata, filters only through id-bit tricks, and its graph indexes handle deletion poorly, which is why production systems such as Milvus and Pinecone wrap a database layer around it. Most usefully for this project, their sizing logic puts the threshold where indexing beats brute force near ten thousand vectors, far above any personal corpus.

Kuffo, Krippner, and Boncz sharpened that point at SIGMOD 2025 with PDX, a storage layout that partitions dimensions across blocks of 64 vectors so that pruning algorithms, which stop reading a vector once a partial distance rules it out, finally beat plain SIMD scans. The same pruning algorithm, ADSampling, swung from losing to a linear scan on the standard layout to running 4.6x faster on PDX, and their exact-search variant PDX-BOND beat Faiss, USearch, and Milvus by 2.5x to 4x with no preprocessing of the vectors at all. The lesson their paper draws is the one this project should carry: retrieval cost is partly a systems problem, and at personal scale exact search stays feasible, which sidesteps recall tuning entirely.

None of this machinery says when retrieval helps a language model. Wang and colleagues' module-by-module search, which opened this narrative with its retrieval gate, has more to say on that question. Hybrid sparse-plus-dense retrieval with HyDE gave the best retrieval quality, query rewriting did not help, and placing the most relevant document nearest the query beat the alternatives. They concede, however, that their chunk-size finding rests on sixty pages of a single financial filing, so the recipes are priors, not law.

Here too the limits are the authors' own admissions. HNSW still lacks a principled story for deletion and update, precisely the operations a maintained memory tree performs, and Malkov and Yashunin's logarithmic scaling was never verified at embedding-scale dimensionality. PDX's results stop at ten million vectors, single-threaded and in memory, with graph indexes untested. And Wang and colleagues'

pipeline evidence comes from public QA corpora and a 7B generator, leaving open how their recipes transfer to a personal store whose corpus, queries, and stakes look nothing like Wikipedia.

What Must an Agent Write Down, and When?

Retrieval machinery answers how to read; the systems gathered here share a conviction that reading is the easy half of the problem. The harder half is consolidation: deciding what an accumulating stream of experience should become, and building machinery that writes that structure rather than merely querying it.

The earliest demonstration that writing can be recursive comes from outside the agents literature. Wu and colleagues at OpenAI showed in 2021 that GPT-3 models could summarize entire novels by fixed task decomposition: chunk the book, summarize each chunk, then summarize concatenations of summaries, recursing to depth 3 for books averaging over 100,000 words. Their framing was scalable oversight (labelers judge each subtask locally, which they estimate made each data point over 50 times cheaper than end-to-end collection), but the durable finding for memory work is that a summary tree compresses an arbitrarily long corpus while keeping facts traceable to source, and that depth 1 summaries alone let a zero-shot question-answering model score competitively on NarrativeQA. The failure modes proved just as durable. Local context turned a request for a dance in Pride and Prejudice into a marriage proposal, book-level output read as event lists rather than narrative, and themes spread thinly across chunks vanished. Any tree that distills leaves independently inherits these risks, which makes cross-leaf context and auditing at composition points design requirements rather than refinements.

Park and colleagues brought consolidation into agents proper in 2023. Their generative agents record every experience as a timestamped natural language object in a memory stream, retrieve by an equally weighted sum of recency, importance, and embedding relevance, and, when the summed importance of recent events crosses a threshold of 150, run reflection: the model poses salient questions over recent records, retrieves against each, and writes cited higher-level inferences back into the stream. The controlled evaluation is the point. With 100 raters ranking interview responses, removing components cost believability stepwise, down from a TrueSkill mu of 29.89 for the full architecture to 21.21 with no memory, and the fully ablated condition trailed the full system by a standardized effect of 8.16. A flat log plus scored retrieval was not enough; the synthesized layer earned its keep empirically. The result holds at the tested scale of 25 agents over two simulated days, and the authors flag planted false memories as an open problem.

Sumers, Yao, Narasimhan, and Griffiths supplied the organizing frame in 2024. CoALA is conceptual rather than experimental: it reads language agents as heirs to production systems and cognitive architectures like Soar, and describes any agent by its memory stores (working, episodic, semantic, procedural), its action space, and its decision cycle. In that vocabulary, Park's reflection is a learning action that moves content from episodic to semantic memory, and Voyager writes code skills into procedural memory. The survey's diagnosis matters here: as of its 2022 to 2023 snapshot, almost no agent treated learning as a deliberately chosen action rather than a fixed schedule, and deletion and

modification of memory were neglected entirely. Writing to procedural memory is flagged as the riskiest form of learning.

Rezazadeh and colleagues answered part of that diagnosis in 2025 with MemTree, which makes the write path structural and continuous. New content is routed down a tree by embedding similarity against depth-adaptive thresholds, and every ancestor's summary is rewritten to fold it in, at O(log N) per insertion. Where Wu's decomposition was fixed and offline, MemTree is online, a distinction that arguably outweighs the choice of structure: RAPTOR and GraphRAG, both taken up in the next section, require full rebuilds to absorb new material, while MemTree pays about 1.4 times their cumulative cost to stay current, roughly matching RAPTOR on QuALITY and beating GraphRAG on MultiHop RAG. On their 70-session conversational extension it beats full-history prompting outright. The cost of perpetual rewriting is drift: ancestor content depends on insertion order and favors recent material, which the authors measure directly.

Talebirad and colleagues drew the general lesson in 2026, showing that these systems, from RAPTOR to conversational memories to execution-trace managers, implement one pipeline of extraction, coarsening, and traversal, and proving that each lossy coarsening level strictly decreases entropy, so the atom layer caps what any summary can recover. Their coupling claim, that self-sufficient summaries support collapsed search while referential labels demand top-down refinement, would explain why MemTree's retrieval works, but they state plainly it has not been tested in a controlled comparison.

That untested coupling is one open edge. Another is that Talebirad's theory assumes a static hierarchy; insertion, restructuring, and decay break its central argument, which the authors name as the most pressing open problem, and MemTree's measured insertion-order drift shows the gap is real rather than formal. Beneath both sits CoALA's still-standing observation that principled forgetting, deciding what a memory system should unwrite, remains nearly unstudied.

What Does a Tower of Summaries Actually Know?

Wu and colleagues' recursive decomposition, met in the previous section, entered the literature as a training problem before anyone read it as a memory architecture, and it left behind two facts that every later hierarchy inherits. Compression works: their 175B models set state of the art on BookSum. And compression lies in characteristic ways, with the further complication that training on full trees made the models worse as lower-level errors compounded upward.

The next wave measured those lies. Chang, Lo, Goyal, and Iyyer built BooookScore in 2024 on a contamination-resistant corpus of 100 recent books, deriving a taxonomy of eight coherence error types from 1,193 human annotations and automating it as an LLM-judged metric. Their central comparison pits hierarchical merging against incremental updating, the rolling-summary pattern most memory systems use, and hierarchical merging is almost always more coherent; the instructive exception is Claude 2 with an 88K-token window, where fewer update-and-compress steps largely erased the gap. Annotators nonetheless preferred incremental summaries for detail by 83 to 11 percent, so coherence and completeness pull in opposite directions. Kim and colleagues pressed harder in 2024 with FABLES, hiring readers who already knew 26 recent novels to verify 3,158 atomic claims from hierarchically

merged summaries. Even the best model, Claude-3-Opus, produced only 90.9 percent faithful claims, unfaithful claims clustered around character and relationship states, and the sobering result was that verification itself fails: the strongest auto-rater, given the entire book, reached only 58.2 F1 at catching fabrications, and retrieval-based checking did far worse. BooookScore finds omission errors dominating its error distribution, and FABLES finds 33 to 65 percent of summaries omitting key events; both foreground omission, the error a summary's reader cannot see.

Meanwhile the tree was being repurposed from output to index. Sarthi and colleagues introduced RAPTOR at ICLR 2024: cluster chunk embeddings, summarize each cluster, re-embed, recurse, then at query time collapse the tree so nodes from every level compete in one similarity search under a token budget. Because clustering runs on embeddings rather than position, distant passages can share a parent, which distinguishes it from Wu's positional recursion. The hierarchy itself did the work; on one QuALITY story, full-tree search beat any single layer by some fifteen points, and the headline QuALITY result with GPT-4 was 82.6 percent against a prior best of 62.3. Edge and colleagues took the complementary path in 2024 with GraphRAG, extracting entities and relations into a knowledge graph, partitioning it with hierarchical Leiden, and pre-writing summaries for every community at every level. For global sensemaking questions over million-token corpora, root-level summaries beat vector RAG on comprehensiveness while using 97 percent fewer context tokens than map-reduce summarization of source text. The graph is one way to decide what gets summarized together; the load-bearing finding is what nested pre-written summaries buy, and its price, 281 minutes of GPT-4-turbo indexing per million tokens.

Talebirad and colleagues' unifying argument lands hardest here, because these trees are its main exhibits. The data processing inequality makes the bottom layer the information ceiling, so every level above can only lose. Their self-sufficiency measure, the fraction of a group's information its representative preserves, gives the coupling claim its terms, and mismatching coarsening style to traversal style wastes budget. They also derive a depth bound predicting when one summary layer stops sufficing as a corpus grows.

Three questions stay open in the works themselves. Every system here builds its hierarchy in one batch, and RAPTOR, GraphRAG, and the theory paper all name incremental insertion and restructuring as unsolved, the same static-hierarchy limit the previous section closed on. The coupling of coarsening to traversal remains an untested prediction. And FABLES leaves auditing itself unresolved: if no automatic method reliably catches what a summary fabricates or omits, the upper layers of any memory tree become a record no one can cheaply check.

How Does a Graph Store a Fact That Was True Then, Learned Later, and Corrected Since?

The temporal half of this question was answered before the graph half existed. Snodgrass distilled roughly 1600 research papers into a 1999 book for working SQL developers, and its central distinction endures (Fowler 2021; Rost and colleagues 2021): valid time is when a fact held in the modeled world, transaction time is when the database learned it, and a table carrying both is bitemporal. His Danish

mortgage bank case (Nykredit, nine million loans) shows why both clocks matter: when someone reports bad data, staff roll the table back in transaction time, read the valid-time history as it then stood, and repair it, with the repair itself logged because transaction time is append-only. Fowler restated the same pattern for practitioners in 2021, preferring the terms actual and record over the Snodgrass vocabulary he found confused workshop audiences, and adding the essential structural claim: once retroactive change is allowed, actual history stops being append-only, but record history never does, which is what preserves the audit trail. He is equally clear that bitemporality complicates a system significantly and can be skipped when retroactive change is forbidden.

The graph half took shape independently. Angles and colleagues surveyed graph query languages in 2017, formalizing the property graph (labels and attribute maps on both nodes and edges) and cataloging which query features are cheap: fixed small patterns over a growing graph sit in logarithmic-space data complexity, while path queries under simple-path semantics turn NP-complete. Angles then gave the property graph model itself a Codd-style formal definition in 2018, noting that despite commercial dominance in Neo4j and Neptune no standard specification existed. Hogan and sixteen coauthors unified the field in their 2021 survey, and their framing matters most here: graphs let you postpone schema definition while data and scope evolve, and their context machinery (temporal validity, provenance, geographic scope) is the vocabulary for statements true only at a time or per a source.

How to attach that context to a fact is where the real engineering fights happened. Vrandecic and Krotzsch described Wikidata in 2014: statements carry qualifiers (“as of 2010”), references, and preferred or deprecated ranks that let contradictory claims coexist rather than forcing resolution. Hernandez, Hogan, and Krotzsch then tested in 2015 how such qualified statements survive translation into plain RDF, comparing four reification schemes over 57 million Wikidata quads on five engines. The elegant scheme lost: singleton properties, the most concise encoding, broke four of the five engines because millions of unique predicates violated indexing assumptions, while named graphs, n-ary relations, and standard reification split the wins with no clear victor. Rost and colleagues joined the two halves in 2021, defining a bitemporal property graph that carries both Snodgrass intervals on every vertex, edge, and individual property value; the property-level intervals supersede their own earlier TPGM, which had to copy an entire element when one property changed. Their system remains a prototype with no benchmarks, so the design is argued rather than measured.

Getting facts into such a graph from text is its own literature. Weikum, Dong, Razniewski, and Suchanek set out the four-phase pipeline in 2021 (discovery, canonicalization, augmentation, cleaning), with the standing conclusions that entities must be canonicalized before relations mean anything and that construction is a life-cycle with curators permanently in the loop. Zhong and colleagues surveyed over 300 construction methods in 2023, fixing the task vocabulary of entity linking, coreference, and relation extraction, but their coverage stops at BERT-era models. Pan and colleagues extended the map to LLM-based construction in 2024. On the embedding side, Cai and colleagues surveyed temporal knowledge graph representation learning in 2024, though Rossi and colleagues had already shown in 2021 that a cheap rule-mining baseline matches embedding models, which perform worst exactly in the sparse, thin-support regime a personal graph occupies. Rasmussen and colleagues assembled the whole stack in 2025: Zep builds a bitemporal graph continuously from conversation, four timestamps per fact edge,

contradiction handled by invalidation rather than overwrite, beating full-context baselines on 115k-token conversations by 15.2 to 18.5 percent while conceding that below context-window scale the graph buys little.

Each strand of this section leaves a gap its own authors have named. Weikum's group calls the personal knowledge base, built from one user's longitudinal footprint, an open research problem; Rasmussen's team reports entity resolution as the hard ongoing problem and notes no benchmark yet tests fusing conversational with structured data; and Cai's survey concedes that temporal-graph methods have been evaluated only on benchmarks far smaller than real deployments, so whether any of this machinery pays at organizational scale is still unmeasured.

Does the Knowledge Graph Survive Contact with Real Data, and Does It Shrink to One Person?

Wikidata, met in the previous section as a design, is also the longest-running public deployment of that design. Vrandecic and Krotzsch presented it two years after its October 2012 launch as the answer to a duplication problem: the population of Rome lived in hundreds of independently edited Wikipedia articles and disagreed across them. The layered statement carried the load, down to distinguishing "value unknown" and "no value" from mere absence. By August 2014 the system held 15.8 million items and 43.2 million statements, 23.2 million of them referenced, and about 90 percent of its half million daily edits came from bots with human curation layered on top. The authors were candid that these numbers documented adoption, not quality, and that complex query support did not yet exist.

Five years later the industry confirmed both the pattern and its costs. Noy and colleagues, writing in 2019 from Google, Microsoft, Facebook, eBay, and IBM, pooled the experience of five production graphs ranging from Facebook's 50 million entities to Google's roughly 1 billion entities and 70 billion assertions. Their lessons rhyme with Wikidata's design at industrial volume: Facebook keeps provenance on every assertion and defines correctness as always being able to explain why an assertion was made; IBM treats source evidence as a primitive and defers entity resolution to query time so context can settle ambiguous names. But the paper's sharpest finding is negative. Entity disambiguation remained the top industry challenge despite years of research; Bing's single entry for the actor Will Smith is assembled from 108,000 facts across 41 websites, against 200 Will Smiths in Wikipedia alone. This is an experience paper with self-reported mechanisms and no benchmarks, and it says so.

The same year, Balog and Kenter argued in 2019 that shrinking this machinery to one person is not a scaling exercise but a different research object. Their personal knowledge graph is defined by a spiderweb topology, every node connected directly or indirectly to a central user node, and by three properties that invert the industrial assumptions: entities need not be prominent ("my guitar", "Mom's dentist"), entity information can be radically sparse, and relations are often short-lived, the opposite of the stability criterion general graphs use for inclusion. They flagged that data-hungry neural link prediction is unlikely to transfer to this regime, and that linking "Jamie" with no external profile entangles linking, population, and NIL-detection at once. There is no system behind the paper: it is agenda, not artifact, and on evaluation the authors conceded that no open PKG dataset existed and synthetic data might be

the only path. Noy and colleagues, from the other side, had independently flagged personalized on-device graphs as an open direction they invited research on, so by 2019 industry and academia were pointing at the same gap from opposite ends.

What the 2019 agenda lacked in deployed evidence, Xu and colleagues supplied in part in 2024, at organizational rather than personal scale. Their LinkedIn customer-service system replaced flat-text RAG over Jira tickets with a two-level graph: each ticket parsed into a section tree (summary, root cause, steps to reproduce), trees linked by explicit Jira references and by thresholded embedding similarity. Retrieval then returns a coherent subgraph instead of arbitrary chunks, scored only over nodes matching the query's intent. Against a plain-text baseline with GPT-4 and E5 held fixed, MRR rose from 0.522 to 0.927 and Recall@3 from 0.640 to 1.000, and in a randomized six-month split of the support team, median resolution time fell 28.6 percent. The caveats matter for this project: the golden dataset was small and internal, and the graph template was hand-built, exactly the kind of curation Balog and Kenter said a personal graph cannot assume. For a memory system where the graph is one candidate representation among several, the honest reading is that structure demonstrably pays when retrieval must respect it, but every deployed success so far bought its structure with human schema work.

On these papers' own accounting, three things are missing. Disambiguation at scale remained unsolved for the five largest graphs in the world as of 2019, and the sparse, nickname-heavy personal case that Balog and Kenter posed is strictly harder. No open personal knowledge graph dataset existed in 2019, leaving evaluation of any personal representation without a shared benchmark. And Xu and colleagues' own future-work list, automatic template extraction and dynamic graph updates, names precisely the manual steps that would have to disappear before their result transfers to a graph nobody curates by hand.

Can a Schema Be Pulled Out of Raw Dialogue, and How Would Success Be Measured?

The hand-built templates the previous section ended on are exactly what a conversational archive cannot assume. Somewhere in that archive, one exchange was about payroll, another was a support request, and two came from the same person; whether such structure can be induced from the transcripts themselves is the question these four works ask, and their collective answer is a qualified yes, with the qualifications doing most of the work.

The earliest of the four takes the schema as given and asks only how cheaply it can be applied. Kecht, Egger, Kratsch, and Roglinger showed in 2021 that standardized process-mining event logs can be built from customer service conversations with no model training at all. Each message becomes the premise of a natural language inference pair, a candidate activity ("This example asks for iOS version") becomes the hypothesis, and a pre-trained BART model returns an entailment probability. The economics are the finding: roughly 100 to 300 hand-labeled messages, used only to calibrate a per-label decision threshold, were enough to structure corpora of 88,000 to 288,000 tweets, with 55 of 56 classifications clearing an MCC of 0.72. But hypothesis wording proved brittle (rephrasing one hypothesis moved MCC from 0.53 to 0.91, and a grammatically broken one scored 0 where its fix scored 1), and the mined process model

covered only the topics the authors thought to define in advance. For open-ended conversational memory, that last concession is the whole problem: the schema is exactly what is not known ahead of time.

Wegmann, Schraagen, and Nguyen complicated the picture from a different angle in 2022. Their target was style embeddings, but their real contribution is a warning about what conversation-derived representations actually encode. Models trained on authorship verification can succeed by memorizing what an author writes about rather than how; when the authors forced distractors to come from the same conversation as the anchor, so topic could no longer separate authors, base RoBERTa picked style over content only 5 percent of the time on their STEL-Or-Content probe, while their conversation-controlled model reached 0.42. The lesson generalizes past stylometry. Any embedding over a conversation archive silently entangles speaker, topic, and phrasing, and disentangling them requires hard negatives deliberately mined from conversation structure. The paper also models a discipline the induction work that followed would need: build a probe where the shortcut and the target answer disagree, and do not trust the training metric.

The intent-induction line then confronted the missing-schema problem directly, and 2023 produced both a reckoning and a workaround. Gung and colleagues at AWS ran the DSTC 11 track on inducing intents from raw call-center transcripts, arguing that prior work had evaluated on repurposed classification datasets lacking full dialogues and realistic skew. Their three simulated corpora ran 59 to 71 turns per conversation, and their open-induction subtask judged an induced schema not by cluster purity but by the accuracy of a classifier trained on it, a framing that transfers directly to judging induced structure over any archive. Thirty-four teams entered; the winner reached 69.8 accuracy on clustering against a 55.8 k-means baseline. Two of their analyses cut at the evaluation itself: propagating labels to one system's unassigned noise points moved it from 33rd place to 9th, and swapping the downstream classifier's encoder shuffled the middle of the leaderboard while the top ten held.

De Raedt and colleagues, also in 2023, attacked the representation problem Wegmann had diagnosed, though by another route. Rather than training an encoder to ignore surface form, IDAS keeps the encoder frozen and moves disambiguation into text: an LLM abstractively summarizes each utterance into a short label, the labels are embedded alongside the utterances, and the combination is clustered. Every labeling strategy they tried beat raw utterance encodings by 5 to 19 ARI points, and with zero labels IDAS beat semi-supervised methods given labels for half the intents on CLINC. Summarize-then-cluster, in other words, is not a convenience but a correction: the summary strips the syntax and noun overlap that otherwise drive similarity. Yet IDAS assumes the true cluster count is known, pays an LLM call per utterance, and found smaller models produced labels too poor to use; the authors flag the first two as open problems themselves.

The unfinished business is stated plainly in the papers. Gung and colleagues showed that the evaluations themselves are unstable, punishing abstention and wobbling with the choice of downstream encoder, so the instruments that would certify an induced schema cannot yet be fully trusted. De Raedt and colleagues left the cluster count and the per-utterance cost unsolved, and Kecht's zero-shot pipeline still requires the event vocabulary in advance. Between them sits the question a memory system cares about

most: how to discover the schema, not merely apply one, when neither the categories nor their number is given.

What Teaches a Memory, and Who Is Allowed to Read What It Learns?

Left to itself, a store of experience is a filing cabinet. What makes it a memory is that it changes in response to how things went, and the question this literature circles is what signal should drive the change. The answers split into three lineages: outcome feedback, self-judged experience, and shared structured state.

The cleanest evidence that outcomes are a usable teacher comes from outside the memory literature. Brynjolfsson, Li and Raymond reported in 2025 a field deployment of an AI assistant in customer support, measured on resolution outcomes. When a real outcome measure exists, learning from it works. Most conversational systems, though, have no resolution ticket to close. Liu, Zhang and Choi asked in 2025 whether the implicit feedback buried in chat logs could substitute, densely annotating 109 conversations from LMSYS and WildChat. The descriptive half is encouraging: feedback appears in more than half of later user turns, so the signal is abundant. The prescriptive half is a warning. Praise is rare, prompts that elicit positive feedback are slightly more toxic than random ones (users praising successful jailbreaks), and training on feedback-conditioned regenerations won on MTBench but lost on the harder WildBench, with plain distillation beating both. Implicit feedback tells you where the user's intent went unmet; it cannot be consumed raw as a reward.

Ouyang and colleagues took the opposite bet in 2026 with ReasoningBank: drop human feedback entirely and let an LLM judge label each finished trajectory success or failure, then distill both into titled strategy items, successes as validated moves and failures as preventative lessons. On WebArena the loop lifted success from 40.5 to 48.8 percent, and the failure records earned their place: on the WebArena-Shopping ablation, success-only extraction reached 46.5 where adding failures reached 49.7, while the earlier success-only AWM pipeline degraded when fed failures its schema could not hold. The striking part is noise tolerance. The judge was only 72.7 percent accurate there, yet varying simulated accuracy from 70 to 90 percent barely moved results, which suggests curation can run without labels at all. This converges with what Maharana and colleagues found in 2024 building LoCoMo: retrieval over distilled per-turn assertions beat both raw transcripts and session summaries, and precision beat volume. Two very different papers, one about writing memory and one about reading it, arrive at the same representation: small, self-contained, distilled units.

The shared-state lineage is much older. Erman, Hayes-Roth, Lesser and Reddy described in 1980 the Hearsay-II blackboard, independent knowledge sources coordinating solely through shared structured state, and Hayes-Roth extended the idea in 1985 to control, letting the blackboard decide what to do next. Salemi and colleagues revived the pattern for LLM agents in 2026, applying it to data discovery over large file lakes: the main agent posts what it needs, and partition agents that have pre-digested their holdings volunteer only when capable. Against an otherwise identical master-slave design, self-selection won everywhere (file discovery F1 of 0.561 versus 0.513, and 0.247 for embedding retrieval), and the advantage grew with lake size, reaching 211 percent relative gain on a 1,556-file lake. The lesson is that a

coordinator's index of who knows what goes stale; letting the holders of knowledge nominate themselves does not.

Once memory is shared, someone must be kept out of parts of it. Rezazadeh and colleagues proposed in 2025 stamping every memory fragment with immutable provenance (time, contributing user, agents, resources) and checking it against time-varying permission graphs at read time, so revoking an edge retroactively hides fragments that depended on it. Shared memory also paid for itself as a cache, cutting external resource calls by up to 61 percent at high query overlap. But this is a formulation with feasibility evidence, not yet a benchmark, and the authors concede their LLM-based policy enforcement can still occasionally breach policy.

The closing admissions of these papers converge on three doubts. ReasoningBank consolidates by pure addition, with no pruning or supersession, and its streams run hundreds of tasks, not years; Hu, Wang and McAuley showed in 2025 with MemoryAgentBench that this gap is not hypothetical, since on their fact-update tasks no tested system exceeded 28 percent at superseding stale facts. Liu's finding that positive feedback encodes bad behavior as readily as good leaves the polarity of the teaching signal itself in doubt. And access control enforced by the model being controlled has, by its own authors' account, no hard guarantee.

Four observations recur across these sections. First, the field builds structures well and maintains them badly. HNSW deferred deletion to future work in 2016; RAPTOR, GraphRAG, and Talebirad and colleagues' 2026 theory all name incremental insertion and restructuring as unsolved; MemTree, which does insert incrementally, measures its own drift; and on MemoryAgentBench no tested system exceeded 28 percent at superseding stale facts. Second, small distilled units keep winning over raw volume: Maharana and colleagues' per-turn assertions in 2024, De Raedt and colleagues' summarize-then-cluster labels in 2023, ReasoningBank's titled strategy items in 2026; Pollertlam and Kornsuwannawit's 2026 pricing shows why, and also names what the compression costs. Third, selection beats capacity. Wang and colleagues' retrieval gate, Jeong and colleagues' router, Salemi and colleagues' self-nominating partition agents, and Modarressi and colleagues' collapse result all locate the gains in deciding whether and whom to ask rather than in asking harder. Fourth, the instruments trail the systems. Kim and colleagues' best fabrication catcher reached 58.2 F1, Gung and colleagues' leaderboard moved with the choice of evaluation encoder, and Balog and Kenter noted in 2019 that no personal-graph dataset existed, a gap no later paper in this corpus reports closed.

PART 5

Ways the research could elevate the project

Prepared by the assistant. An itemized list of options, not recommendations. The project's goal is a rigorous general system, with the existing prototype serving as evidence and first tenant; read each item accordingly, as a candidate mechanism for the general design, stated here against the prototype because the prototype is where the evidence and the baseline live. Each item traces to the work that suggested it, and each states what the prototype already does, since no item may propose what already exists. Where overlap exists, the item is scoped to the delta, usually the automated, measured, or scaled version of a manual practice. Citations outside the packaged reading list, from the adversarial literature pass, carry arXiv identifiers inline. Effort figures are orders of magnitude (days, a week, weeks), not commitments. Items plainly exceeding the fall budget of 70 to 100 hours are marked spring-scale. Nothing here is ranked or sequenced; selection and ordering belong to another document and the advisor conversation.

Capture

A revived runtime citation log

A PostToolUse hook that appends every tree read to a session log, so a leaf's citations become transcription rather than recollection. The system built exactly this (`cite_log.py`), shelved it on 2026-07-31, and deleted it on 2026-08-21 without it ever running; OPERATIONS.md keeps the argument alive in prose ("reconstructing at session close is a recollection"). CoALA (Sumers et al. 2024) names memory writes as deliberate internal actions worth first-class treatment, and Noy et al. 2019 report provenance-first assertion as the pattern all five industrial graphs converged on. The delta is a rebuild, wired and verified, feeding the cued-versus-read-versus-load-bearing distinction OPERATIONS names but never mechanized. Effort is days, rebuilding from git history. The counterfactual miss instrument below consumes its output.

Outcome-signal mining from correction turns

A pass that scans transcripts for implicit negative feedback, the rephrasings and "no, I meant" turns, and converts recurring ones into failure-log candidates. Liu, Zhang and Choi 2025 show later user turns are dense with correction signal, reliable as a lens on unmet intent even though noisy as training data; ReasoningBank (Ouyang et al. 2026) shows failure records carry real value when the schema stores lessons rather than replays. The overlap is the failure loop: `fail` exists but fires only when someone notices a miss, and nothing mines the archive for the ones nobody logged. This is the wrongness signal his own dossier lists as open space, since `_failure_log.jsonl` records misroutes, not wrong answers. Effort is a week, including a hand-checked precision sample.

A segmentation quality lane in the stale-leaf slice

An audit of whether multi-topic sessions were cut into leaves at the right boundaries, with permission to re-cut. Event segmentation theory (Zacks et al. 2007) treats the cut as an encoding decision, and Reflective Memory Management (2503.08026, ACL 2025) shows rigid granularity fragments

conversational structure. The overlap is real but partial: the one-chat-many-leaves rule governs capture, and the stale-leaf slice re-summarizes in place every pass, yet nothing ever re-segments; a badly cut leaf stays badly cut forever, a gap his own literature dossier already names. The delta is a sampled bad-cut rate plus a re-segmentation lane in the existing burn-down, converting a permanent defect class into a measured, shrinking one. Effort is days for the audit sample, a week for the lane.

Structure and representation

A one-era property-graph trial

A small representation experiment, honoring the standing position that knowledge graphs are one candidate representation, not the destination. Render one era's ledger rows as a typed property graph using the Angles 2018 formalization, whose delta function (an edge-type whitelist per node-type pair) is a vocabulary contract for an extraction pipeline, then answer the same golden questions by graph walk versus sheet read, scoring accuracy and hop count. The overlap is that the ledger is already subject-predicate-object rows with qualifiers, so most of a graph exists; the delta is typed edges and multi-hop query. Rossi et al. 2021 show sparse personal graphs are the worst regime for embedding methods, so the trial stays symbolic, per Hogan et al. 2021 on lightweight schema. Effort is days, and it produces evidence for or against the professor's graph steer, drawn from the student's own corpus.

Recorded currency judgments as dated rank rows

When a read-time judgment decides which of two conflicting rows is current, record that judgment as its own dated, sourced row, in the spirit of Wikidata's preferred and deprecated marks (Vrandecic and Krotzsch 2014) and Fowler's 2021 point that record history stays append-only even under correction. Nothing is overwritten or invalidated at write time, so the founding facts-cannot-be-wrong rule holds; what changes is that an expensive judgment is made once and cached as data instead of remade at every read. The overlap is that the ledger already stores corrections as new rows and imprecision as date_precision qualifiers; the currency call currently lives only in the reading agent's head. Effort is days. The stale-serve measurement below can consume these rows.

Entity resolution measured on his own registry

A labeled set of alias pairs drawn from the corpus (Defy Medical, Defy, the clinic; GAO across four eras) and a measured resolution accuracy over it. Balog and Kenter 2019 frame long-tail personal entity linking as the defining open problem of personal knowledge graphs; Noy et al. 2019 call disambiguation the top challenge across all five industrial graphs; Zhong et al. 2024 supply the task decomposition. The overlap is that the ledger resolves identity through `registry_entities.txt`, and OPERATIONS.md itself flags that the two entity systems produced 70 versus 388 entities with only 17 names in common, so the corpus already demonstrates the problem. Nothing measures it. The delta is a number for a known weakness and a defensible answer when a reviewer asks how identity is managed. Effort is days to a week.

Currency

A stale-serve rate

A measurement of how often a recall surface serves a superseded fact. MemStrata (2606.26511) supplies the metric shape and reports plain RAG serving outdated facts 15 to 40 percent of the time; MemoryAgentBench (Hu, Wang and McAuley 2025) finds no tested system above 28 percent on multi-hop fact updates, so the number is expected to be nonzero. The overlap is drift-debt, which measures whether a leaf is current against its source, a storage property; nothing measures the delivery property, whether answers actually reaching a session were stale. The method: a probe set built from his own documented supersessions (the internship ending, the session renumbering, the topic pivot) run against tree and ledger reads. Effort is days once the probe set exists; the typed gold set item can supply it.

An implicit-conflict probe

A test of whether the system detects conflicts carrying no explicit negation, where a later observation quietly falsifies an earlier row. STALE (2605.06527) defines the task and reports a best-model score of 55.2 percent; Pollertlam and Kornsuwannawit 2026 name update non-propagation, a “twice a week” fact surviving its revision to three, as a leading failure of flat fact extraction. The overlap: the tail-scan rule already catches corrections inside one conversation at capture time, but the validator only re-checks that quotes still appear, never that a claim went false; cross-conversation supersession has no detector at all. This measures the cost of the deliberate design choice against Zep-style invalidation-on-write without proposing to reverse it. Effort is a week; measurement only, no write-path change.

Retrieval and delivery

A counterfactual miss instrument

A measurement of how often the store held the answer and the session never consulted it. The adversarial literature pass rated this the project’s strongest publishable claim; today it is only a limitation note (access is not recall): every published benchmark poses the query, so this failure cannot occur in their setups, and Remember When It Matters (2607.08716) covers only the intra-task horizon. The overlap is substantial and the item scopes around it: the failure mode is named, `recall_cue.py` is the intended fix, and the failure loop logs misses somebody noticed. What is missing is the denominator, the misses nobody noticed. The method seeds sessions with tasks whose answers provably sit in the tree, instruments reads through the citation log, and counts non-consultations. Effort is a week to build, with data accruing on normal use afterward. Depends on the runtime citation log item.

A retrieval-depth router

A cheap per-query decision among no lookup, a one-hop rollup read, and multi-hop descent, made before retrieval starts. Adaptive-RAG (Jeong et al. 2024) matches its always-iterate baseline at a third of the steps and shows usage logs can self-label the router by recording which depth sufficed; Wang et al. 2024 find the decide-whether-to-retrieve gate the single cheapest accuracy and latency win in their module search. The overlap is that current policy is static: the primer’s standing instruction plus the cue hook’s boundary triggers, no per-query decision anywhere. The delta buys cheaper recall and, as a side

effect, a free labeled dataset of query difficulty over his own corpus, which feeds the evaluation work. Effort is a week.

The endorsed escape-hatch index, built and measured

A brute-force exact-cosine embedding index over leaf text, no vector database, consulted only when tree descent lands on the wrong branch. His own records already endorse this role, with the flattened-CBC example as the worked case, and fix the design point: at roughly 1,200 files exact search is correct and index machinery is overhead, consistent with the Faiss sizing guidance (Douze et al. 2024) placing the brute-force crossover far above this scale. NoLiMa (2025) supplies the motivation, showing retrieval must bridge vocabulary before the model sees anything because lexical mismatch is the normal case. The overlap is a decision and a cost model with nothing built. The delta is an afternoon of code plus a probe set of vocabulary-mismatch questions measuring wrong-branch recovery. Effort is days.

Self-selection routing across eras (spring-scale)

The blackboard pattern applied to the tree: partition agents that have each pre-digested one era volunteer answers when a query touches their subtree, instead of a central index deciding who knows what. Salemi et al. 2026 report 13 to 57 percent end-to-end gains over master-slave routing, with the advantage growing as the corpus grows. The overlap is direct: `directory.md` plus the curated boundary lines is exactly the central index whose staleness the blackboard result targets, and the era summaries are already the pre-digestion. It buys an architecture argument for the organizational scale-out his newest statement of intent names. Effort is weeks, and the payoff scales with corpus size he does not yet have: spring-scale on both counts.

Evaluation and measurement

E2, a typed gold set built on published task definitions

An extension of EXPERIMENTS.md: 100 to 150 gold questions typed by the published categories (single-hop, multi-hop, temporal, knowledge-update, abstention) from LoCoMo (Maharana et al. 2024) and LongMemEval, per the standing rule that benchmarks supply task definitions and comparability, never a contest to win. The overlap: `_golden.jsonl` exists as a frozen deterministic set, the adversarial-sampling rule is written, and E1 established the file format, so the delta is size, typed coverage, and category-level reporting a reviewer can read. Agent fleets make the labeling tractable inside the budget; completion accounting is mandatory: five of twelve automated audit checks have previously stopped running without raising an error. Effort is a week of fleet runs plus review. Presented as an option for the advisor to shape, not a finished design.

A calibration set for the faithfulness gate

A measurement of the gate itself. FABLES (Kim et al. 2024) shows automatic faithfulness checking reaches 58.2 F1 at best on the indirect multi-hop claims that rollups are made of, and the cold audit proved the point locally: eight defects, all in the rollup layer, all invisible to entailment checking because a hardened sentence is still entailed by its hedged source. The overlap is that the faithfulness metric already runs every pass, and the 39 hand-verified claims from 2026-08-21 are a free labeled seed. The

delta scores the gate against those labels, reports its precision and recall, and extends the target taxonomy to the omission and salience errors BooookScore (Chang et al. 2024) shows dominate hierarchical summarization. Effort is days; it converts a known blind spot into a calibrated one.

A layer-localized error study of the write path

The 39-claim audit scaled with agent fleets and reported by layer: transcript to leaf, leaf to node, node to era. BooookScore scores only final summaries and cannot localize which level introduced an error; Wu et al. 2021 documented compounding errors at composition points; Talebirad et al. 2026 prove the atom layer sets the information ceiling, which makes the fold step the place losses concentrate. The overlap is that the rotating source audit is the standing mechanism and the cold audit ran this design once by hand, finding 31 confirmed, 4 wrong, 4 unsupported, with every defect in the synthesized layer; audit coverage sits at 2.3 percent. The delta is coverage, statistics, and the by-layer cut. This is the paper's motivating study run on machinery that already exists. Effort is a week of fleet time.

The hash-versus-dirty-flag demonstration

The controlled experiment behind the one mechanism claim that survived his adversarial novelty review. Hand-edit N leaves out of band, then show that a write-time dirty flag, the scheme MemForest (2605.23986) and every build system use, misses all N while content-hash recomputation catches all N, because a flag is only correct if every writer remembers to set it and a recomputed hash is self-checking. The overlap is nearly total and that is the point: `fold_sig` is implemented, tested, and running, so nothing new gets built except the comparison arm and the write-up. It buys the strongest defensible contribution named in his own literature review for the cost of a fixture script. Effort is days. It pairs naturally with the layer study as a short paper's experimental section.

Scale-out

A permission-graph formalization of the existing boundaries (spring-scale)

A recasting of the guard patterns, the holds ledger, and the gitignored eras as the provenance-stamped fragments and retrospective permission checks of Collaborative Memory (Rezazadeh et al. 2025), where revoking an edge in the access graph retroactively hides every fragment derived through it. The overlap is that the single-user version already runs: CUI boundaries, held-local UUIDs, and the guard-your-own-writing rule are hand-enforced policy over the primitives that paper formalizes. It buys the concrete bridge from a personal backend to the organization-scale framing in his newest statement of intent, since multi-tenant access is the one thing the current design cannot claim. Effort is weeks and spring-scale, though a two-page design note mapping current mechanisms onto the formalism would fit a fall afternoon.

A measured cache-tier policy for the primer

Treat the SessionStart primer as the preloaded tier of a two-tier design and set its boundary from data. CAG (Chan et al. 2025) shows that below some corpus size, precomputed context beats retrieval outright, and its own large-corpus result shows where that stops; Pollertlam and Kornsuwannawit 2026 supply break-even arithmetic between resending history and extract-then-retrieve, with memory winning after

roughly ten turns at 100K tokens. The overlap is that the tiering already exists by instinct: the primer preloads the era map, the dossiers load on demand, and everything else sits behind search, with no measurement saying what belongs where. The delta varies primer contents and measures recall against token cost per session. Effort is days; the numbers strengthen the cost section of any write-up.

Three shapes the project could take

Prepared by the assistant from the research package, the design brief, and the constraints on record: roughly 70 to 100 project hours this fall concentrated between September 1 and November 15, PhD applications due December 1 to 15, IT 494 continuing through Spring 2027, and the standing requirement that Justin works through all code himself. One correction governs this document: the goal is a rigorous and defensible general system, not an improvement to the existing prototype. The prototype's role in every shape below is evidence and instrument: two years of operation that reveal the real problems, a corpus, and a first tenant. It is never the deliverable. These are judgments, not options for the sake of options, and a recommendation follows.

Shape 1. Design first, prove the core

The semester's product is a defensible design and a working core. The sequence: read the field until each design question can be argued from the literature rather than from habit; write the architecture of a general knowledge backend, with every load-bearing choice justified by a citation, a measurement, or a named open question; then implement the core of it, small but real, with evaluation built in from the first commit rather than bolted on. The prototype supplies the requirements the literature cannot: the documented failure cases, the corpus, and a live user. Where the new design disagrees with what the prototype does, that disagreement is stated and defended, which is what makes the design an argument rather than a description.

What it pays: this is the shape that matches the stated goal directly. It produces the artifacts a committee and an application both respect: a design that can be defended choice by choice, a core that runs, and numbers behind both. It fits the hour budget because the expensive half, construction at scale, is explicitly deferred to spring.

What it risks: a design semester can slide into a reading semester. The mitigation is the core: code that runs is the proof the design is real, and the December paper is scoped to design plus first measurements, not to a finished system.

Shape 2. Build the general system outright

The semester's product is the backend itself: capture, structure, currency, and retrieval as a general service, installable, multi-tenant in principle, with the prototype's data migrated in as the first tenant and the annual-training multi-chat pattern as the demonstration.

What it pays: the strongest possible demo, and a foundation the organizational use case could stand on immediately. A portfolio artifact for the federal hiring channel as well as the academic one.

What it risks: rigor is the first casualty of a build semester. Under a review-every-line constraint inside 70 to 100 hours, engineering will crowd out both the reading and the evaluation, and December would hold

a partially built system with neither a defended design nor results. Honestly scoped, this is the spring semester, executed against a design that fall produced.

Shape 3. One question, answered generally

The semester's product is a single defensible result about memory systems in general, studied on the prototype's corpus because that is the corpus available. The two candidates the package supports: the counterfactual miss rate, how often a store held the answer and the session never consulted it, which no published benchmark can even express because benchmarks pose the query; and content-derived staleness against write-time flags, which is a general correctness property of any human-editable store. Either is framed as a claim about the class of systems, measured on one instance.

What it pays: the cleanest research identity per hour, and the fourth-band and miss-rate instruments would be contributions other systems could adopt.

What it risks: a narrow result leaves the general-system goal unadvanced except by reputation. If the aim is a defensible system rather than a defensible sentence, this shape is an ingredient, not a semester.

Recommendation

Shape 1, with Shape 3's instruments as its evaluation layer, and Shape 2 as spring. Concretely: September is the reading, closed out by a short written synthesis that turns the reading list into positions. October is the design document, general by construction, defended line by line, with the prototype cited as evidence rather than authority. October into November is the core implementation with the evaluation harness alongside it, the corrected-facts band and the miss rate among its instruments, run first against the prototype's corpus as tenant zero. The December paper reports the design and the first measurements. Spring, in the student's framing, builds the distributable harness others can run and tests it: installation by someone who is not the author, a cold corpus the workflow did not grow up on (the certified-unknown literature test in the testing outline), migration of the student's own two years of raw data out of the prototype and into the harness as the proof that tenant zero moves, and the fall harness riding along as the regression suite. The migration doubles as the strongest single validation available: the mock retires, the product carries its history, and any answer the old system could give and the new one cannot is a regression with a name.

Two commitments make it concrete. First, every fall artifact is written about the general problem, with the prototype as the running example, so nothing has to be reframed later. Second, the paper is scoped to what exists by November 15, aligned with the calendar's dead stretches, so coursework collisions are planned rather than suffered.

PART 7

End product, test data, and what gets recorded

Prepared by the assistant. This outline assumes the framing the student set: the existing prototype is a mock of the intended end state, and the end state is a general workflow built into a product and then tested on data the workflow did not grow up on. Fall produces the defended design, a working core with its evaluation harness, and a short paper; spring builds the product out. If a different shape is chosen, the testing section still applies, since every shape answers to the same measurements.

The end product, concretely

Fall delivers four things. First, a written synthesis that closes the reading phase: the positions taken, with the works that ground them. Second, the design document for the general backend: capture, structure, currency, retrieval, and evaluation as first-class components, each choice defended from the literature or marked as an open question the implementation will test, and each departure from the prototype's way of doing it stated and argued. Third, the core implementation with its evaluation harness built alongside: enough of the design running to measure, populated with the prototype's archive as tenant zero. Fourth, the paper distilled from the design and the first measurements, targeted at a workshop or arXiv by early December.

Spring delivers the system the design describes: installable, documented, separable from any one tenant's data, with the annual-training multi-chat pattern as the demonstration and the fall harness as its regression suite.

Test data, the fork that must be chosen deliberately

Three options, answering different questions, and the general-system goal changes their weights.

The prototype's archive is tenant zero: two years, roughly a thousand distilled summaries over twenty million tokens of transcript, with ground truth available from the one person the corpus is about. It is the only corpus on which the corrected-facts band can be built today, because it carries documented instances of facts that changed. Its limits are real: private, so results are not reproducible by others, and the summaries were written by models that have seen the writer's patterns, so contamination is named in the report. For a general-system claim it has one more limit worth stating plainly: it is one tenant, and nothing measured on it alone demonstrates generality.

A public benchmark is the generality check, not just an anchor. One suite from the package, run in its published configuration, shows the design is not shaped to one man's archive. Scoped as a week.

A second, disjoint corpus is the honest test of the design's portability, ingested cold through the same pipeline. The student's choice, refined 2026-08-25: a long work of fiction with an active community wiki. The novel half is strong on its own terms: a book is a life in miniature, events arriving in order, entities accumulating relationships, and facts superseded by later reveals, which instantiates the corrected-facts

band with ground truth obtainable by reading. The wiki half upgrades the evaluation from answers to structure: the harness induces a tree, entity pages, and dated facts from raw text, and the community spent years hand-building the same artifacts, so induced structure can be scored against theirs for entity coverage, relation recall, and timeline agreement. Two rules govern it. First, a wiki implies popularity and popularity implies training exposure, so certification moves from the work to the question: the question set is run closed-book against every bare model tier, and only questions all tiers fail are kept; long serials make this workable because models know famous plots but fail on depth, minor characters, and chapter-level chronology (the NovelQA finding). Second, the wiki is a reference, never an authority: disagreements between harness and wiki are adjudicated by the text, and cases where the harness is right and the wiki is wrong are reported, not discarded. The candidate class is long serial fiction with a dedicated wiki; three candidates are nominated in Phase 0. One becomes the primary corpus with the full protocol (deep question set, all four bands, structure scored against its wiki); at least one more runs as a lighter replication (a smaller certified set and entity-coverage scoring only), so the claim is about books, not one book. The replication protocol exists the moment the primary protocol does, which is what makes the second work cheap. This is a spring gate for the distributable claim, stated in the paper as future work if fall runs out of hours.

The deliberate choice this outline assumes: design measured on tenant zero with the contamination caveat open, one public suite for generality, and a cold literature corpus, certified unknown to the model, gating the spring release.

What gets recorded

Every run records the same row: date, arm, configuration hash, question identifier and band, verdict against the fixed answer, tokens in and out, dollars, and latency. The harness instruments are general by construction: the four question bands, the counterfactual miss rate, and the stale-serve rate are defined against any conforming store, with tenant zero merely their first subject. Alongside the runs, the standing counters the prototype already computes stay in the record, since the design document will cite them as the evidence base. Incidents get their own log: every case of a store holding an answer a session failed to consult, and every superseded fact served as current, each with enough context to re-derive. These logs start accumulating the day the harness exists, not the day the paper is written.

What gets printed

For the advisor, three artifacts on a cadence. The metric definition sheet, early, so the measurements are agreed before results exist. The design document at its first complete draft, because that is the artifact this project stands on and the one that most needs adversarial reading. And the review trail: which code was walked through line by line, when, and what changed as a result, kept as a running log rather than reconstructed later.

The QA constraint, structurally

The constraint is authorship, not review. The mock was built by directed AI execution that the student never examined, which was the right way to prove the idea and is the wrong way to build the product; in this project the student implements the execution and builds the intermediate steps himself, with AI assistance in drafting and discussion but never as the unexamined author. The design document, the harness runner, the scorer, and every metric definition are build gates: the project does not proceed past them until the student has written or rewritten them and logged it. The weekly rhythm follows the campus calendar: build lands Tuesday through Friday, review happens in the Monday and Wednesday campus gaps, which are reading time by construction. The five dead stretches in the design brief are scheduled as review-and-writing weeks, not build weeks, because reading survives a deadline week and building does not.

Anatomy of the mock: every stage, and what executes it

This document dissects the running claude-archive memory pipeline stage by stage, assembled from six subsystem dissections (capture, distill, fold, entities and facts, retrieval, maintain and verify), each claim carrying its file-and-line receipt. It is the specification baseline for the engineered harness: corpus in, memory tree plus entity knowledge base out, efficient injection at query time. The governing finding: of 49 distinct stages, 26 execute end to end as deterministic stdlib Python, but those 26 are plumbing (parsing, hashing, gating, scheduling, serving, alarming). Every stage that creates or changes meaning runs by hand: 23 stages have a human, a scheduled agent, or an in-session LLM as their generative core (1 human-only, 8 with an LLM or agent as sole executor, 14 mixed), all of them executing prose protocol from OPERATIONS.md rather than code executing a contract. No LLM output in the tree layer is schema-validated at write time; the single write-time contract in the whole system is the fact ledger's `ledger.append()` validator. Quality is checked only after the fact, by sampling.

The pipeline as it exists

Capture

1. Source acquisition. EXECUTED BY: **human**. Two asymmetric channels. Claude Code sessions self-record: the CLI writes append-only JSONL per session to `C:\Users\jhffm\.claude\projects\<project>\<uuid>.jsonl` with no archive code involved. Web-chat corpora (ChatGPT, claude.ai, Gemini) exist only when Justin manually requests a vendor takeout and drops the archive into `C:\Users\jhffm\claude-archive\raw\`. Judgment: whether and when to request a takeout; the only reminder is weekly-pass step 1 ("requesting the takeout itself is manual and his", `OPERATIONS.md` line 105). Fragility: ChatGPT is frozen at the 2026-07-11 takeout and Gemini at 2026-07-31; nothing measures that staleness, and there is no freshness alarm on `raw\`.

2. Nightly trigger. EXECUTED BY: **deterministic-code**. Task Scheduler job ClaudeArchiveNightlySync (02:30, 12:30, 19:15, plus wake-catchup) runs `wscript.exe scripts\nightly_hidden.vbs`, which launches a hardcoded python 3.14 path on `scripts\maintain.py` with a hidden window. Heartbeat is the rewritten `_last_maintain.log` (MARKER.write_text, `maintain.py` line 249). Judgment: none. Fragility: the heartbeat mtime is rewritten even when the push fails, so a "worked but didn't ship" run reads green to the 2-day staleness rule (`OPERATIONS.md` line 506); the hardcoded python path dies silently on a Python upgrade (Task Scheduler still reports 0 because wscript succeeded). Observed 2026-08-25: the task, and AgentMemoryKit, both read DISABLED in Get-ScheduledTask; why is NOT FOUND in any file.

3. Change gating. EXECUTED BY: **deterministic-code**. `raw_changed_since(MARKER)` (`maintain.py` lines 78-83) compares mtimes of `RAW.glob('*')` against `_last_maintain.log`; the heavy export adapters run only on change (lines 94-99). Judgment: none. Fragility: the glob is non-recursive (a takeout in a `raw\` subfolder never triggers); a zip dropped mid-run can be skipped until touched again; only `tail[-1]`

of each child's output is logged (lines 107-108), so a parser traceback survives as one truncated line while the run continues.

4. Parsing transcripts into corpus files. EXECUTED BY: **deterministic-code**. One parser, four adapters (`scripts\parse_sources.py`: `from_claude_code` 177, `from_claude_ai` 235, `from_chatgpt` 263, `from_gemini` 324) feed a shared `emit()` (113). Identity is `<source>:<full id>` from the YAML header via `conv_key()` (65), never the filename; rewrite only when the recorded message/event count changes (`_recorded_count`, 92); drifted filenames are renamed via `os.replace`; two-plus priors for one id report AMBIGUOUS and touch nothing (`conversations\` is immutable by house rule; merging is exported to Justin via the printed report). Current corpus: 1060 files (74 ccd, 560 gpt, 219 gemini, ~207 unprefixed claude.ai). Fragility: each export adapter returns after the first archive whose shape matches, newest mtime first, so an older takeout with unique conversations is silently never parsed (bare `return` at 259/301/369); `_load()` swallows BadZipFile/JSONDecodeError as "no conversations" (218-231); count-equality cannot see same-count edits; Gemini identity is fabricated as date plus session index (line 365), so a re-export would mint duplicates wholesale; ccd messages starting with `[` are dropped as noise (195).

5. Code-artifact extraction. EXECUTED BY: **deterministic-code**. Because stage 4 strips tool blocks, session code exists only in the raw jsonl; `scripts\extract_code_artifacts.py` replays Write/Edit/MultiEdit per session (`reconstruct`, 76) into gitignored `code_artifacts\<id8>\`, then `--programs` rebuilds per-(era, filename) end-states across all chats with a frozen latest-within-85%-of-largest rule, pasted-fence harvesting (`_harvest_pasted`, 223), and a mechanical audit (`_audit_body`: ast.parse authoritative) to `_programs\_AUDIT.md`. 2026-08-24 log: 148 snapshots, 1045 end-states, audit OK 659 / SUSPECT 81 / SYNTAX-ERROR 11. Fragility: `maintain.py` runs the identical snapshot pass twice per nightly (lines 114-120 and 152-158, both visible in the log); files created via Bash heredocs, `python -c`, or generators are invisible; filename attribution for pasted fences has documented misfires; unclassified conversations default to era `misc` via `_conv_era` (205).

6. Guard, commit, push. EXECUTED BY: **deterministic-code** (disposition of holds is human, see the inventory). `git status --porcelain -uall` enumerates changes; allowlisted suffixes are scanned against SECRET_PATTERNS (absolute) and CUI_PATTERNS (diff-aware versus `git show HEAD:<path>`), with site-specific fragments from gitignored `_guard_patterns.txt` (missing file: WARNING and reduced coverage). Quarantined paths go to `_REVIEW_NEEDED.txt` and are unstaged after `git add -A`; then commit and push, push failure logged, never fatal (`maintain.py` 160-245). Fragility: LIVE as of 2026-08-24, the run ended "PUSH FAILED... fast-forwards", local commits stranded, heartbeat still green; secrets inside .docx/.png/.pdf are never scanned; novel credential or CUI phrasing auto-pushes (accepted residual risk, docstring 18-21); `_REVIEW_NEEDED.txt` is only rewritten when something new quarantines, so a drained list reads identically to an ignored one (`OPERATIONS.md` line 540).

Segment, summarize, file (distill)

7. Rule delivery. EXECUTED BY: **deterministic-code**. The session-close rule is not self-executing; it reaches sessions via the SessionStart hook (`mcp_server\session_recall.py`, primer strings 160-167) and the global `~\.claude\CLAUDE.md` standing block, mirrored by hand per `OPERATIONS.md` line 602. Fragility:

OPERATIONS.md line 31 calls this “the single most load-bearing surface and the easiest to lose”; the hook is fail-silent, an unprimed session simply never writes leaves, and nothing checks the CLAUDE.md mirror.

8. Session-close leaf writing. EXECUTED BY: **llm-in-session-per-protocol**. At the end of any substantive session, the in-session LLM writes one leaf per topic segment into eras_staging\: filename YYYY-MM-DD_<slug>_<convid8>.md, frontmatter (date, era, topic, people, decisions, code_artifacts, open_threads), body prose, verb-tagged ## Related, code pointed at rather than inlined (OPERATIONS.md lines 40-48, 44, 75-99). Judgment: substantive or not; where segment boundaries fall (the whole specification is “segment by topic”); era/topic routing; what survives summarization; citation from recollection; whether a mid-conversation figure was later retracted (tail-scan rule). Fragility: nothing enforces this at write time; skips surface only as coverage debt (v2 baseline 67.4%, 244 un-leafed, line 294); the _live-session misname happened three times, two leaves now unkeyable (line 44); the runtime citation layer (cite_log.py) was never wired and was deleted 2026-08-21 (line 450); no validator ever runs on live-session leaf frontmatter.

9. Import digestion (weekly steps 1-2). EXECUTED BY: **scheduled-agent**. The weekly agent diffs conversations\ against eras\MANIFEST.md (PowerShell enumeration, never Glob, per the ripgrep false-negative gotcha), then per unclassified transcript: segment, write staged leaves, add MANIFEST rows (| File | Proposed topic folder | Gist | Source |; held-local rows carry no gist, lines 574-578); leaf-unworthy conversations go to eras_no_leaf.md as reasoned decisions the coverage gate reads. This is the one place raw is legitimately re-read (line 16). Fragility: on 2026-07-30 a pass announced its worklist and died writing nothing, invisible for five days (leaf eras\claude-archive-meta\maintenance-and-review\2026-07-30_weekly-run-died-mid-pass_cbfff1ba.md); nothing catches a bad split; MANIFEST is a large hand-edited table with documented drift (495 rows once named dead folders).

10. Promotion and routing. EXECUTED BY: **mixed**. Clean staged leaves move into era/topic folders; routing is LLM judgment against eras_boundaries.md discriminator lines merged into the regenerated eras\directory.md (rollup_tools.py directory, OPERATIONS.md line 150); cwd never decides era (line 109); a misroute earns a failure-log row and a new boundary line. Fragility: “reviewed/clean” has no checklist; _staging is an unbounded-latency buffer; the boundary parser silently clobbers duplicate keys (its own header warning); routing correctness is only ever corrected reactively.

11. Leaf baselining (stamp-leaves). EXECUTED BY: **deterministic-code**. After promotion, rollup_tools.py stamp-leaves writes <!-- rollup: {"src_sha":..., "src_stamped":...} --> into the leaf (write at ~1324-1330); scan later reads stale/current/UNMEASURED (never “clean”) from it; existing baselines are protected and --force is banned as a drift launderer. Fragility: unstamped leaves stay UNMEASURED indefinitely; doc-import leaves keyed doc-<hash8> sit outside the id-keyed drift machinery; the stamp records currency, not correctness (line 296).

12. Document-import variant (one-off, 2026-08-23). EXECUTED BY: **mixed**. ~1,042 filesystem documents became 73 leaves through a 14-step pipeline (scripts\imports\2026-08-23-doc-import\README.md): deterministic CUI scan, extraction, backup, clustering, validation (validate_leaves.py: REQUIRED frontmatter list, hardcoded KNOWN node map, source_doc resolution), em-dash sweep, promotion, additive fold; LLM cores at triage (13 agents), distillation (13

agents), and adversarial verification (9 agents, 9 cross-doc-bleed findings fixed). Fragility: this is the only mechanical frontmatter/routing validation anywhere, and it is a one-off with an already-stale node map that never runs on live leaves; triage manifests were deliberately not committed, so a re-run reconstructs judgment from scratch.

Fold

13. Scan. EXECUTED BY: **deterministic-code**. `cmd_scan` (`rollup_tools.py` 742-978) rehashes every conversation (`sha`, 88; multi-file ids get a combined hash, 768-773) and every tree file with markers stripped (`sha_nomarker`, 96-101), computes recursive per-node `fold_sig` over (relpath, content-hash) pairs (789-805, duplicated in `_compute_fold_sig` 1124-1137), classifies node and leaf staleness from in-file markers, and computes coverage, refcheck, compression, golden, boundaries, fact-layer metrics, and MCP health into `_ledger.json` and `_metrics.json`. Fragility: two fold_sig implementations could drift apart; coverage percent is coverage of what is measurable only.

14. Node staleness detection. EXECUTED BY: **deterministic-code**. A node is stale iff its marker's fold_sig differs from the recomputed one; no marker means stale; `cmd_idempotency` (1051-1062) always rescans first (the 2026-07-26 stale-ledger bug) and prints the refold worklist. Because markers are stripped from hashes, stamping never cascades; a real child edit does. Fragility: fold_sig proves the children changed, not that the summary is good.

15. Refold (node summary prose). EXECUTED BY: **scheduled-agent**. The weekly agent rewrites each stale node's ERA_SUMMARY/NODE_SUMMARY from child summaries only, never raw (`OPERATIONS.md` steps 4 and 7), curating promoted links under the altitude test. No function in `rollup_tools.py` writes summary prose (verified across all 2906 lines). Judgment: everything (compression, spine, link promotion, split/merge). Fragility: quality checks are after the fact and narrow; faithfulness and retrieval are self-graded by the same agent; the pre-remediation mutation study found 5 of 10 real breakages produced no signal.

16. Stamp. EXECUTED BY: **mixed**. `cmd_stamp` (1140-1167) recomputes fold_sig and writes {fold_sig, n_children, last_folded} into the node marker, trusting the agent's claim that a refold happened. `stamp --all` is banned outside a logged re-baseline: it once marked ~90 rollups current without rewriting any (`OPERATIONS.md` step 6b). Fragility: stamping is a trust declaration; a lazy agent turns every mechanical check green.

17. Leaf drift and the stale slice. EXECUTED BY: **mixed**. Detection deterministic (cmd_scan 821-845, `stale_leaf_ages` 1438-1490, escalation at 21 days, bulk clears logged to `_stale_cleared.jsonl` as "the launderer's signature"); the re-summarise itself is agent work: a dated addendum in place, then clear exactly that leaf's src_sha for re-baselining. Fragility: the clock records first observed stale, understating true age; currency is not correctness.

18. Directory generation and boundary curation. EXECUTED BY: **mixed**. `cmd_directory` (2439-2495) deterministically merges disk structure with hand-kept rules from `eras\_boundaries.md`, rendering stale rules in a labeled section (the 2026-07-31 cutover hid 11 dead rules for two weeks under the old logic). The rules themselves are Justin-adjudicated, grown one line per logged misroute. Fragility:

duplicate-key clobber documented but unenforced; rule quality depends on failures actually being logged.

19. Cross-context links. EXECUTED BY: **mixed**. Agents propose links as JSON; `cmd_links` (1195-1293) validates through a rejection ladder including verbatim-evidence-in-source ("EVIDENCE NOT IN SOURCE (possible fabrication)", the stated anti-fabrication guarantee) and writes `## Related` bullets only with `--apply`; `links promote` nominates, the fold agent decides via the altitude test. Fragility: leaf-birth citations depend on recollection because cite_log.py never ran; link writes churn hashes and cascade refolds.

Entities (retired layer)

20. Entity-page build. EXECUTED BY: **deterministic-code**, RETIRED 2026-08-21. Nightly pages per recurring name in `eras\entities\` from leaf prose plus the curated alias map `eras\_entities.md`, threshold-gated, index-not-copy (recoverable at git 5c84ec8; now a no-op shim at `rollup_tools.py` 1341-1352 because the governance-gated nightly still calls it). Failure mode that killed it: rebuilt nightly, consumed by no recall surface (archive_mcp excluded it; recall_cue's dated-filename filter could never match it). Fragility for a rebuilder: `OPERATIONS.md` lines 275-289 still describe the layer as staying, contradicting the retirement record at lines 50-73; the current text wins.

21. Quote-index invariant. EXECUTED BY: **deterministic-code**, RETIRED with stage 20. Every quoted evidence sentence had to still appear verbatim in its leaf (git 5c84ec8 lines ~830-842); the invariant was dropped, and nothing replaced it for the ledger: fact rows carry sources, not quotes, so "does the source support this row" is checked only by the monthly human review queue. The pattern survives in `cmd_links`' evidence check (1175-1199).

Facts (the ledger)

22. Source registration. EXECUTED BY: **mixed**. `ledger\registry_sources.txt` maps portable prefixes to roots with (excluded) markers (`ledger.py` source_roots/resolve_source/is_excluded, 161-204); inclusion policy is human (health un-excluded 2026-08-23 on Justin's direction, army-reserve stays out). Fragility: the registry is the distiller's only privacy boundary; an exclusion typo silently opens a corpus.

23. Queue planning. EXECUTED BY: **deterministic-code**. `extract.undistilled()` (102-155) orders changed-fingerprint docs first (sha256[:16] vs `_distilled.json`), then never-read docs ranked by mentions of the top-150 known entities. Fragility: .md files only; the rich-get-richer ranking is by design.

24. Prompt build. EXECUTED BY: **deterministic-code**. `extract.brief()` (185-196) assembles RULES contract, predicate vocabulary, entity digest capped at 400 names, source path, and body capped at 24,000 chars. Fragility: LIVE, the registry is ~1,169 names against the 400 cap, so the in-prompt duplicate-minting warning path is active ("Build candidate retrieval before trusting this batch", 179-182); long-document tails are silently dropped.

25. LLM extraction. EXECUTED BY: **scheduled-llm-call** (or a live session via `extract_queue\` plus ingest). A model emits bare pipe rows `asserted_on | subject | predicate | object | qualifiers | source`; `run_api()` (295-364, claude-opus-5, effort low) cleans and hands rows to the validator. This is the only place non-deterministic choice enters the ledger: durability, dating, entity binding versus minting, predicate selection. Fragility: LIVE double stall, the User-scope ANTHROPIC_API_KEY gap left days of "daily left DUE", and the 2026-08-24 heartbeat records a BadRequestError for exhausted API credits with ~1,397 documents undistilled.

26. Validation and append. EXECUTED BY: **deterministic-code**. `ledger.append()` (433-511) with `validate()` (229-296): six fields, full ISO date, predicate registered, entities resolve or mint typed unknown, key:value qualifiers, source resolves and is openable right now, no excluded prefixes, subject != object, byte-duplicate suppression; rejects logged with reasons to `_dropped_rows.txt`, barren docs to `_barren.txt`. "Nothing here trusts the model's output" (`extract.py` docstring). Fragility: minted unknown-typed entities await a review nothing schedules; `OPERATIONS.md` line 261's "no row is appended unattended" is false of the running system (run_api appends directly).

27. Rendering. EXECUTED BY: **deterministic-code**. `ledger.render()` (388-396) rebuilds ~1,120 sheets and project handoffs, each headed "Nothing here is computed. Read the dates to work out what is still true", with a content signature enabling content-keyed staleness (`stale_projects`, 399-417). Conflict handling is deliberate absence: append-only, no supersession engine (an earlier one "produced four separate classes of confidently-wrong answer", `ledger.py` 7-10); later-wins is a read-time judgment.

28. Tick scheduler. EXECUTED BY: **deterministic-code**. AgentMemoryKit runs `ledger\tick.cmd` every 20 minutes into the kit's own `maintain.py tick` (243-290): elapsed-time dispatch so every run retries every missed run, lock recovery, validate-before-touch, split last_tick/last_success heartbeat, LEDGER_AUTOCOMMIT=0 so commits route through the archive's guarded nightly (the 08-20 to 08-21 guard-bypass hole). Fragility: productivity hinges on one machine-local credential; the task currently reads DISABLED.

29. Repair, propose, review. EXECUTED BY: **mixed**. Tier 1 deterministic repairs apply and roll back on regression (`repair.py` run, 118-161); tier 2 drafts split-identity "same as" proposals into `proposals.txt` ("Nothing here appends itself"); tier 3 judgment questions land in audit reports; monthly `review.write_queue()` asks per assertion whether the source supports the claim, verdicts appended, never edited. Fragility: 131 proposals pending with no forcing function beyond the weekly habit, several obvious similarity false positives; review coverage rides on the monthly habit.

30. Fact serving (MCP). EXECUTED BY: **deterministic-code**. `memory_mcp.py` (38-166) serves find_entity, about, assertions_about, resume_project, vocabulary, and remember (one row through the same validator, mint=False). Fragility: no mint tool, so a session cannot record a fact about a new entity without hand-editing the registry; proposals and the review queue are not exposed.

31. Health integration into the archive. EXECUTED BY: **deterministic-code**. The archive watches the ledger at exactly one read-only seam: `fact_layer_metrics()` (566-647), the User-scope key probe (2120-2146), and `append_only_verdict()` (1493-1578), a monotonic byte-plus-sha watermark

chosen because both git-HEAD comparison and silent re-baselining were demonstrated laundering paths on 2026-08-21. Fragility: the watermark file is an untracked trust anchor beside the data it guards; 36h OVERDUE means a two-day stall is the earliest loud signal.

Retrieve and inject

32. SessionStart priming. EXECUTED BY: **deterministic-code**. `session_recall.py` prints, in order: the gate_alert() health block (76-134), the "LONG-TERM MEMORY IS LIVE" standing orders, the fact-layer counts, the dossier line, and one line per era (leaf_count plus a 150-char first-prose gist). Measured live 2026-08-25: 4,860 bytes, roughly 1,100-1,250 tokens, a map plus orders, never leaf content. Fragility: fail-silent by design; registration is machine-local; this primer is also the system's only alerting channel, so losing the hook loses all alarms.

33. Cued recall. EXECUTED BY: **deterministic-code**. `recall_cue.py` fires at boundaries only (new session, 30-min idle, or 2 new salient terms, 120s cooldown) and injects at most a reason tag plus 3 leaf paths from a substring search over dated leaf filenames (constants 38-41, `_search` 98-136). Pointers only. Fragility: "indistinguishable from working" (any error prints nothing, exits 0; the hooks are not gated, only the MCP servers are); a subject cues once per session ever; bare substring matching means vocabulary mismatch yields silent zero.

34. Ranked search and reads (MCP). EXECUTED BY: **mixed**. The engine is deterministic (`archive_mcp.py`: `_terms` 118-132, `_score` 135-146 doing lowercase substring counts, `_importance` hand-weighted 189-209, `_recency` half-life on `_access_log.json` 212-227, relevance-guard warning 336-338); when to call it and how to word queries is in-session LLM judgment steered by docstrings. Fragility: search covers leaf summaries only, never raw, so wrongly summarized or un-leafed material is unfindable; the access log is machine-local; the importance weights are hand-set and unvalidated.

35. Drill-down and altitude navigation. EXECUTED BY: **llm-in-session-per-protocol**. No code decides drill-down: the primer's CONSULT-IT order, tool docstrings, the altitude ladder, and "Following links is ADVISORY, not mandatory" (`OPERATIONS.md` 485-491) are all prose. Fragility: nothing verifies compliance or logs whether a cued leaf was ever opened (the layer that would have, cite_log.py, was deleted); the founding failure (2026-07-26, twenty turns argued without consulting the tree) recurs whenever the model ignores the prose.

36. Browser-Claude surface. EXECUTED BY: **llm-in-session-per-protocol**. `eras\BROWSER_RECALL.md` is paste-in prose for a claude.ai Project with a GitHub connector: recall before responding, era to node to leaf, cite paths, no ranked search. Fragility: "drift, with nothing to detect it", and the drift is live: `OPERATIONS.md` line 35 still describes the file as saying health is off GitHub while the file was rewritten 2026-08-23 to say health syncs.

37. Remote HTTP surface. EXECUTED BY: **deterministic-code**, RETIRED, never verified deployed. `remote_server.py` (surviving only at `code_artifacts\40bacd93\remote_server.py`) wrapped the MCP app in bearer auth that claude.ai's connector flow could not use; carried for weeks as "PRESENT, DEPLOYMENT UNVERIFIED" pointing at a DEPLOY.md that never existed. The kept lesson: the local/cloud serving split is a real security boundary; rebuild deliberately.

38. Registration and bootstrap. EXECUTED BY: **mixed**. `scripts\install_surfaces.py --apply/--status` registers both hooks and both MCP servers (verified 2026-08-25: all four at USER scope, redundant project approvals in dnd-campaign); OPERATIONS mandates verify-by-observation after three wrong prose claims about registration. Fragility: on a fresh clone the tree arrives complete and every surface is off, silently; only the servers are probed, the hooks are not.

Maintain and verify

39. Refcheck. EXECUTED BY: **deterministic-code**. `_refcheck` (981-998) reports dangling [[wikilinks]] and missing backticked `eras/` citations on every scan. Fragility: basename-only slug resolution; prose paths are deliberately skipped.

40. Golden regression. EXECUTED BY: **deterministic-code**. `_golden_check` (1011-1036): 13 frozen rows in `eras\_golden.jsonl`, each passing iff all expect_keywords appear as substrings of the first file matching the source relpath substring; run live in every scan and report; never edit an answer to pass. Fragility: 13 substring probes over ~1000 leaves; `hits[0]` on an ambiguous source name could test the wrong file; growth depends on the add-when-durable habit.

41. Content gates. EXECUTED BY: **deterministic-code**. `hedge_stripping` (217-309) flags a node hardening a date its children only hedge; `coverage_regression` (312-380) flags summaries that stop naming their topics or reaching their leaves. Added 2026-08-21 after the mutation study. Fragility: deliberately narrow; they catch hardened dates and dropped topics, not wrong facts or causality, and faithfulness scoring structurally cannot see hedge-stripping.

42. Invariant gates and the failure log. EXECUTED BY: **deterministic-code**. `_run_invariants` (1901-2227): 24 gates spanning privacy (gitignore, held-local, held-gist), guard invocation, tracked-credential sweep, drift measured, truncated-code pointers, count and topic recomputation, push age under 26h, content gates, pip check, the 21-day stale-leaf clock, alarm-wired, coverage-counts-everything, real MCP handshake-plus-tool-call probes (v5 of a probe wrong four ways), distiller key at User scope, autocommit off, the append-only watermark, freshness, and the GATE_PROOF meta-gate requiring every gate to declare its mutation-test proof (`test_gates.py`). Non-PASS auto-opens `_failure_log.jsonl` rows; `reconcile_failure_log` (1647) auto-closes them; a crash writes a FAIL gate (`run_invariants`, 1876). UNKNOWN is never PASS. Fragility: maintain.py never reads report's exit code (comment 2394-2397); the alerting chain hangs on the SessionStart hook, which is checked by one of these very gates, a partially circular guarantee.

43. Alarm surface and response. EXECUTED BY: **mixed**. `write_gate_state` (1618) persists failures; `session_recall.py` renders them first in every session, including "ARCHIVE HEALTH: STALE" past 36h. A human is the consumer; nothing emails, blocks, or retries. Fragility: the human-fix loop has no deadline (the credential-sweep gate has been failing since 2026-08-22, row still open); the alarm depends on the very nightly it reports on.

44. Weekly maintenance pass. EXECUTED BY: **scheduled-agent**. The named task 'weekly-memory-tree-maintenance' fires an agent whose entire prompt is "run the weekly pass per OPERATIONS.md": pass open, scan, import digestion, promotion, folds and stamps, one rebalance, one

backlog item, the stale-leaf slice, rollups, the rotating audit, one log line, quarantine drain, pass close. Fragility: NOT FOUND in Windows Task Scheduler on this machine; the last two log entries (08-21, 08-23) were interactive sessions, not the agent; the entire quality layer rides on an LLM following a 621-line protocol whose only in-repo backstop is the 7-day log-staleness habit.

45. Rotating source-verification audit. EXECUTED BY: **llm-in-session-per-protocol**. `audit-pick` (1079) suggests the least-recently-audited subtree; the agent verifies its leaves against raw transcripts, actively refuting dates, figures, quotes, and causality, subject to the mandatory self-audit exclusion rule; `audit-record` (1090) writes acc/fid/cmp into `_audit_state.json` and the node marker; `audit-claims` (2498) mechanizes the node-versus-raw packet where 8 of 8 measured defects lived. Fragility: this is the only mechanism that tests whether summaries are TRUE, and coverage is 5.7% (5 subtrees); scores are unverified self-reports; the exclusion rule lives only in protocol.

46. Metric gate and the failure loop. EXECUTED BY: **mixed**. Before a weekly diff stands: rerun scan plus golden; bars are golden 100% and retrieval at least 10/12; on regression, revert the pass and log a failure. Golden is mechanical; faithfulness and retrieval are LLM-judged strings the agent records into `_metrics.json`, which scan merely carries forward (934-936). Safety-critical process fixes are queued for Justin, never self-applied. Fragility: nothing mechanically blocks a bad commit, the nightly commits whatever is staged; a pass that skips the sampled scoring leaves stale scores that look current.

47. Pass open/close marker. EXECUTED BY: **mixed**. `cmd_pass` (2257-2283) records open time in gitignored `_pass_in_progress.json`; scan warns past 6 hours, converting died-mid-run from invisible to visible, but only when the agent remembered to call open. Fragility: nothing forces the call; the marker protects one machine.

48. Selftest. EXECUTED BY: **deterministic-code**. `cmd_selftest` (2604-2859) rebinds state globals to a temp fixture and asserts ~40 named checks pinning every historical defect; mandated after any edit to rollup_tools.py. Fragility: convention, not a pre-commit hook; nothing blocks committing with a red selftest.

49. Staleness watch. EXECUTED BY: **mixed**. Three clocks converge on Justin: the 7-day maintenance-log rule (an LLM-in-session obligation from `~\.claude\CLAUDE.md`), the 2-day nightly heartbeat mtime with its read-the-log-body caveat, and the deterministic gate alarm. Fragility: every surface fails silently by design; the ultimate backstop is an LLM following prose, the least mechanical link in the chain.

Direct answers

Is there semantic search anywhere? No. None. Every retrieval path is lexical substring matching: `archive_mcp.py _score` does `low.count(t) / low.find(t)` over lowercased full text with hand-set importance and recency tiebreaks; `recall_cue.py` does `t in text`; `search_manifest` reuses `_score`. A grep for embed|vector|cosine|semantic|faiss|sentence-transformer|tfidf across `mcp_server\*.py`, `scripts\*.py`, and `agent-memory-kit\prototype\*.py` returns only incidental prose, and `OPERATIONS.md`'s invariants phrase embeddings as hypothetical: “embeddings (if ever added) are an index, never the memory.”

How are documents ingested? Three ways, none unified. (1) Claude Code sessions: continuous free capture to `~\.claude\projects\*\*.jsonl`, swept three times daily by `parse_sources.py` into `conversations\*.md` keyed by conversation id. (2) Web-chat corpora: manual vendor takeouts into `raw\`, parsed only when the mtime gate notices, only the newest archive per format ever read, currently frozen at 2026-07-11 (ChatGPT) and 2026-07-31 (Gemini). (3) Filesystem documents: the one-off 2026-08-23 import pipeline (`scripts\imports\2026-08-23-doc-import\`), 1,042 docs to 73 leaves through scripted scaffolding around three multi-agent LLM workflows, not repeatable without reconstructing judgment.

How are fact rows generated? `registry_sources.txt` declares the corpus; `extract.undistilled()` picks documents (changed fingerprints first, then never-read ranked by known-entity mentions); `extract.brief()` builds the prompt (RULES contract, predicate vocabulary, 400-name entity digest, 24k-char body cap); a model (claude-opus-5 via `run_api()` unattended, or a live session via `extract_queue\`) emits pipe-delimited rows; `ledger.append()` validates every field against the registries and the live filesystem, appends clean rows to `facts.txt`, mints unknown-typed entities, logs rejects with reasons; sheets re-render deterministically. The LLM decides only row content; code enforces everything checkable. Conflicts are never resolved: append-only by design, later-wins at read time, corrections as new dated rows.

Where do the tree layer and the fact layer connect? Almost nowhere, deliberately. They share no code: the retired entity layer lived in `claude-archive\scripts\rollup_tools.py`, the ledger lives in `C:\Users\jhffm\agent-memory-kit\prototype\` parameterized by LEDGER_HOME; they disagreed on what an entity is (threshold-gated 70 versus mint-on-sight 388, 17 names in common, `OPERATIONS.md` 281-285). The system of record was settled for the ledger 2026-08-21 and the entity layer retired outright. Remaining touchpoints are one-directional and read-only: `fact_layer_metrics()` and `append_only_verdict()` read ledger files for health gating, and `registry_sources.txt` excluded `eras\entities\` so the ledger could never cite a derived view of itself. The written rebuild instruction: if a per-entity index is wanted again, build it FROM the ledger and wire it into a recall surface before trusting it (`OPERATIONS.md` 70-73).

The by-hand inventory

One row per stage whose generative core is a human or an LLM improvising against protocol today. This table is the build backlog for the harness.

#	Stage	What the hand does today	What the harness must do instead	Hardest part of automating
1	Source acquisition (S1)	Justin remembers to request takeouts and drop them in raw\	Scheduled export pulls per vendor API where possible, plus a freshness alarm on every source channel	Vendors without export APIs; detecting silent corpus staleness
2	Ambiguous-duplicate merge (S4	Justin merges AMBIGUOUS same-id	Deterministic merge policy (superset wins,	Choosing a merge rule safe for an immutable

#	Stage	What the hand does today	What the harness must do instead	Hardest part of automating
	residue)	transcripts from the printed report	else union with provenance) executed by code	corpus
3	Quarantine drain (S6 residue)	Weekly human disposition: commit deliberately or hold local	Ticketed queue with deadlines, and machine-checkable disposition records replacing the hand-cleared txt file	Judging CUI/sensitivity is inherently human; the harness can only enforce the deadline
4	Session-close leaf writing (S8)	In-session LLM segments, summarizes, routes, cites, entirely from a prose rule it must remember	A SessionEnd-hooked extraction job: contract-schema leaf output, validated frontmatter, machine-supplied conversation id, retraction-aware tail scan	Segmentation has no specification anywhere; defining a testable boundary function is new research, not porting
5	Import digestion (S9)	Scheduled agent leafs the backlog and hand-edits MANIFEST rows	Same extraction contract as row 4 run in batch, MANIFEST as generated artifact not hand-edited table	Same segmentation problem at scale, plus leaf-worthiness (the _no_leaf bar) as a classifier
6	Promotion and routing (S10)	LLM routes each leaf against prose boundary rules; “clean” is undefined	Routing as a constrained classification call validated against the machine-readable boundary set, with a defined promotion gate	Boundary rules are failure-grown prose; making them a testable ruleset without losing nuance
7	Document-import triage/distill/verify (S12)	One-off multi-agent workflows with uncommitted judgment manifests	A repeatable cold-ingest pipeline: any foreign corpus in, triaged and distilled under the same contracts	Cross-doc-bleed prevention (the verify pass caught 9); triage verdicts are judgment
8	Refold prose (S15)	Weekly agent rewrites node summaries from child summaries	Contracted summarize-of-summaries calls with entailment checks against children before acceptance	Verifying faithfulness mechanically; today it is self-graded by the writer
9	Stamp discipline (S16)	Agent honestly stamps only what it refolded	Stamping performed by the refold job itself, atomically with the accepted rewrite	Trivial once refold is a job; the hard part is row 8

#	Stage	What the hand does today	What the harness must do instead	Hardest part of automating
10	Stale-leaf re-summarise slice (S17)	Agent picks and rewrites stale leaves with dated addenda	Drift-triggered re-summarization jobs, ordered by a scoring rule, re-baselined atomically	Deciding what changed in the source and whether the leaf is actually misleading
11	Boundary-rule curation (S18)	Justin adjudicates new discriminator lines after misroutes	Misroute detection feeding proposed rule diffs for one-click approval	The adjudication itself stays human; the harness closes the loop around it
12	Link proposal and promotion (S19)	LLM proposes links; fold agent applies the altitude test	Proposal generation as a batch job through the existing validator; promotion as a scored, thresholded decision	The altitude test ("would a reader stopping here be misled?") resists mechanization
13	Fact extraction (S25)	Scheduled LLM call, currently stalled on a machine-local credential	Same call under harness-managed credentials, retry budgets, and a candidate-retrieval step replacing the 400-name digest cap	Entity binding versus minting at scale; the digest-cap duplicate-minting problem needs retrieval, not a bigger prompt
14	Proposal approval and review verdicts (S29)	Human/session reviews proposals.txt (131 pending) and monthly review queue	Batched review sessions with escalation on backlog age; auto-reject for scored-obvious false positives	"Does the source support this claim" is the same truth problem as row 20
15	Registry curation (S22, S26 residue)	Human edits source inclusion and entity types; minted unknowns await a review nothing schedules	Typed-entity resolution jobs and a scheduled unknown-type triage queue	Identity resolution (same-as) without silent identity blending
16	Query planning (S34)	In-session LLM decides when to search and how to word queries	Automatic retrieval at query time (the RAG arm): the harness retrieves, the model consumes	Query formulation from conversational context; today mitigated only by a warning string
17	Drill-down navigation (S35)	LLM follows the altitude ladder by prose discipline	Harness-side context assembly to a budget: pre-fetch node plus leaf content instead of hoping the model drills	Choosing the stopping altitude per query; nothing today measures whether pointers were followed

#	Stage	What the hand does today	What the harness must do instead	Hardest part of automating
18	Browser surface (S36)	Pasted prose instructions, hand-updated, drifting	Generated surface config from one source of truth, or retire the surface into the served API	Keeping any un-hooked surface in sync is unsolvable; generation is the fix
19	Bootstrap and registration (S38)	Human runs install_surfaces.py and verifies by observation	Idempotent provisioning in setup, with the registration gates as post-conditions	Nothing hard; it is discipline made into code
20	Weekly pass orchestration (S44)	A scheduled agent interprets a 621-line prose protocol	A real orchestrator: each step a typed job with inputs, outputs, and completion records; the protocol becomes code	Decomposing OPERATIONS.md into jobs without losing its accumulated failure wisdom
21	Source-verification audit (S45)	LLM refutes leaves against raw, self-scores acc/fid/cmp	Independent-model verification jobs with claim extraction (audit-claims already mechanizes the packet) and adversarial scoring	Ground truth: verifying summaries are TRUE is the system's only truth check and the least automatable judgment in it
22	Metric gate scoring and revert (S46)	Agent self-grades faithfulness and retrieval, decides revert	Held-out scored evaluation by a non-author model; revert as an automatic transaction on failing bars	Building an honest retrieval benchmark; today the bars have no mechanical enforcement at all
23	Alarm response (S43)	A human reads the primer and fixes what it names	Escalating notification with deadlines per failure class; auto-remediation for the mechanical subset	Repairs like credential revocation and history purges stay human; the harness enforces the clock
24	Staleness habit (S49)	An LLM obeys a 7-day prose rule in CLAUDE.md	Watchdog jobs supervising every scheduler, including the schedulers themselves (both tasks are DISABLED today and nothing said so)	Supervising the supervisor on a single machine with no external monitor
25	Golden-set growth (S40 residue)	Agent/human adds a question when a durable fact lands	Auto-proposed golden rows at leaf acceptance, human-approved	Selecting durable facts worth freezing; 13 probes over ~1000 leaves is the current total

What the mock never had

Stages a rigorous pipeline needs that do not exist today in any form. A semantic retrieval arm: every path is substring matching, so vocabulary mismatch fails silently and no embedding, index, or reranker exists anywhere (proved by grep; `OPERATIONS.md` names embeddings only as hypothetical). Outcome and citation capture: nothing records whether an injected cue or search hit was ever opened or helped; the one attempt (cite_log.py, a PostToolUse capture) was never wired and was deleted 2026-08-21, so leaf citations are end-of-session recollection. Injection budgeting: the primer is a fixed ~1,200-token block and the cue a fixed 3 paths; no component measures a context budget, ranks what deserves injection under it, or adapts payload to query. Write-time contracts on LLM output: the fact validator aside, no leaf, MANIFEST row, node summary, or audit score is schema-checked when written (the only leaf validator is a one-off import script with a stale hardcoded node map); the harness's central move is making every LLM output pass a contract the way fact rows already do. Enforcement of the session-close rule: no hook fires when a session ends without a leaf; omissions surface only as later coverage debt. Repeatable cold ingest of a foreign corpus: takeouts parse only the newest archive per format, Gemini has no stable identity so a re-export mints mass duplicates, and the document import was a one-off whose judgment manifests were not kept. Capture freshness measurement: nothing alarms on a corpus frozen at its last takeout. An honest retrieval benchmark: golden is 13 substring probes, and the two LLM-judged bars are self-graded by the summary author with stale scores carried forward indefinitely. Independent truth verification at speed: the rotating audit covers 5.7% of the tree at one subtree per week, and between it and two narrow content heuristics there is no fast check that any node is true. Supervision of the schedulers themselves: both Windows tasks read DISABLED today, the reason is recorded nowhere, and the only remaining detector is a session-start alarm whose own registration is machine-local and unprovisioned on a fresh clone.

The plan, in phases

Prepared by the assistant. This is the operative sequence for the harness project and supersedes the recommendation section of the project-shapes document where they differ. Scope of record: the prototype is a mock; the build target is a bottom-to-top harness, corpus in, memory tree plus entity knowledge base plus dated facts out, with budgeted injection for RAG, model-agnostic by construction, in Python, at the rigor bar of the EES work. The calendar constraints come from the design brief: two usable work blocks (September 1 to 27 and October 19 to November 15), five dead stretches owned by coursework, applications December 1 to 15, and roughly 70 to 100 project hours in total. Each phase below states its entry condition, its steps in order, its exit deliverable, and its scope valve, the thing that gets cut first if hours run short. Gates marked QA are authorship gates: the student implements the stage himself (AI assists in drafting and discussion, never as unexamined author, which is the mock's model and stays the mock's), and the gate is the logged completion of his own build; design and reading land in campus gaps and dead stretches.

Phase 0. Decisions and scaffold (now through August 31, about 4 hours)

Entry: the advisor meeting.

1. Resolve the five [CHECK] flags in the proposal and hand the corrected proposal to Dr. Fang.
2. Take three outcomes from the meeting: agreement on the harness scope, a reaction to the December paper target, and a ruling on the GAO EES preprint question (one paper or two this fall).
3. Pick the model tiers for the sensitivity axis: one local 7B via Ollama, one mini-class API model, one frontier model. Install Ollama and confirm the local model runs.
4. Nominate three candidate works for the cold-corpus test. Criteria, revised: long (a serial of several hundred thousand words is fine and better than a doorstopper standalone), fiction with entity structure and reveals, and carrying an ACTIVE COMMUNITY WIKI to score induced structure against. Popularity is acceptable because certification is per-question (bare tiers must fail each kept question closed-book), not per-work. No commitment yet; certification happens in Phase 3.
5. Scaffold the build repo inside the project repo: `harness/` with the two interfaces stubbed (`embed(texts)`, `generate(prompt, schema)`), the run-row schema from the testing outline as a dataclass, and pytest wired. QA gate: the interfaces, about two pages, walked through.

Exit: decisions logged in the repo, empty harness that imports and passes a trivial test. Valve: none; this phase is small on purpose.

Phase 1. Read with the build in mind (September 1 to 21, about 18 hours)

Entry: Phase 0 decisions logged.

1. Week one: consolidation and hierarchy (the reading list's sections in that order), read against the question "what does the fold stage need to do." Output: one page of positions, each position a sentence plus the work that grounds it.
2. Week two: knowledge graph representation, temporal and bitemporal modeling, and open information extraction, read against the fact-row schema. Output: the fact-row schema v1 with each field justified by a citation, and the supersession rule stated.
3. Week three: retrieval, injection, and evaluation (RAG best practices, Lost-in-the-Middle-class work, LLM-as-judge validity, narrative QA benchmarks), read against the three injection arms and the scorer. Output: the metric definition sheet v1, the artifact the testing outline says gets agreed with Fang before results exist.
4. Throughout: the gap-list sources on schema-constrained generation and model cascades, read against the contracts design. Output: the summary and extraction contracts v0, with named rejection criteria.

Exit: a synthesis memo (the three outputs above merged, five to eight pages) sent to Fang, and the metric sheet on his desk for agreement. QA gate: the memo is the gate; it is the reading made checkable. Valve: week three compresses to the judge-validity and narrative-QA reads only; injection-ordering literature can be read during the October dead stretch.

Phase 1.5. Design and dry scaffold (September 22 to 27, about 8 hours)

1. Write the pipeline design as one document: every stage from the anatomy's 25-row by-hand table mapped to a component, each with its contract, its inputs and outputs, and its executed-by target (deterministic code, or a model call through the interfaces). The anatomy found 26 of the mock's 49 stages already deterministic; those port, they are not redesigned. 1a. Write the segmentation specification as its own artifact. The anatomy's sharpest finding: segmentation has no specification anywhere in the mock, so this is new design work, not porting. The spec defines a testable boundary function with ground-truth metrics (the Pk and WindowDiff framing from the segmentation reading), with TextTiling as the deterministic baseline arm and the dialogue-segmentation work governing the chat case.
2. Implement the corpus adapters dry: transcript-file reader and plain-text book reader emitting a common document stream. No model calls yet.
3. Freeze contracts v1 (segment, summarize, extract-entities, extract-facts) as JSON schemas with validators and rejection logging.
4. QA gate: design document plus adapters plus contracts, one sitting each.

Exit: the design document, adapters passing tests on real files, contracts that validate. Valve: none; Phase 2 cannot start without this.

Dead window (September 28 to October 18)

Coursework owns these weeks (Test 1, HW2, the midterm window, MP1). Project work is reading-only: leftover gap-list sources, the paper's related-work section drafted from the digest and narrative, at most two hours a week. Nothing lands unreviewed later because nothing lands.

Phase 2. Build the pipeline (October 19 to November 15, about 32 hours)

Entry: design document QA'd; contracts frozen; Fang has the metric sheet.

Build order is corpus order, one stage per step, each step ending in a QA gate and a pytest suite. Hour boxes are budgets, not estimates of luck.

1. Segmentation (4h): boundary detection over the document stream, model-called against the segment contract, with the cheap-tier model as default. Test: boundaries on three known transcripts and one book chapter, compared against one alternative segmenter, differences explained in a note.
2. Summarization and filing (5h): segment to leaf against the summary contract; filing into the fixed two-level shape by model call with the routing rule in the prompt; rejection rate logged per model tier.
3. Fold (3h): rollup generation from children only, staleness by content hash ported conceptually from the mock's fold_sig, refold only stale nodes. Test: idempotence, a second run folds nothing.
4. Entities (4h): extraction against the entity contract, alias resolution by explicit map plus model proposal, pages built as indexes with quote validation, the mock's rule kept.
5. Facts (4h): dated assertion rows per the Phase 1 schema, subject-predicate collision detection, later-assertion-wins read rule, rejection rate logged. This is the stage the OpenIE and Eywa-style reading directly governs.
6. Injection arms (5h): the three strategies (top-k chunks as control; tree-routed summaries; tree plus facts hybrid) under one token-budget accountant, ordering per the injection reading, every run emitting the standard run row.
7. Harness and scorer (5h): question runner, LLM-judge scorer with the bias mitigations from the judge reading, calibration against a small hand-labeled set (the agreement reference from the gap list), per-band reporting. One rule the anatomy makes non-negotiable: the judge model is never the writer model. The mock's refold prose and audit scores are self-graded by the model that produced them, which is exactly the unverifiable-self-report failure the harness exists to close; in the harness, every acceptance check (summary faithfulness, fact support, answer scoring) runs on a different model tier than the one that generated the artifact, and the sensitivity table reports judge-model effects alongside writer-model effects.
8. Integration pass (2h): corpus in, answers out, one command.

Exit: the harness runs end to end on a slice of the archive as tenant zero. Valves, in cut order: the entity layer reduces to extraction-only (no pages) saving ~2h; injection arms drop from three to two (control plus hybrid) saving ~2h; the sensitivity axis defers the frontier tier and reports cheap-versus-mini only.

Phase 3. Measure and write (November 16 to December 7, about 12 hours, deliberately low-intensity)

Entry: harness end-to-end. This window contains Test 2, Thanksgiving, HW4, and the application deadlines; everything here is runs and prose, no construction.

1. Certify the question set per question: every bare tier runs the candidate set closed-book, and only questions all tiers fail survive; then ingest and run all arms (runs are cheap once built; evenings suffice). Score induced entities and timelines against the community wiki, adjudicating disagreements by the text.
2. Run the archive slice with the corrected-facts band and the miss-rate instrument.
3. Stage-wise sensitivity table: swap tiers per stage on the fixed question set.
4. Write the paper from the design document, the synthesis memo, and the run tables; the related-work section already exists from the dead window. Target: arXiv by December 5, cited in the applications.
5. Brag sheets to the three letter writers when portals open, the paper named.

Exit: preprint posted, applications out. Valve: if runs expose a broken stage, the paper narrows to the corpus actually working (the novel alone is a complete story) and says so.

Phase 4. Spring: the distributable harness

Scoped now, planned in January against the spring syllabus: packaging and installation by someone who is not the author; the cold-corpus protocol run in full on the primary work and as a lighter replication on at least one more (smaller certified set, entity-coverage scoring), so portability is shown across works; migration of the full two-year raw archive through the harness as tenant-zero proof, with any answer the mock could give and the harness cannot logged as a named regression; multi-tenant separation; and the second paper if fall's ruling was two. The fall harness rides along as the regression suite. Entry condition: the fall paper exists; this phase never starts early at the cost of Phase 3.

Standing across all phases

Weekly rhythm: build lands Tuesday through Friday; review happens in the Monday and Wednesday campus gaps; dead stretches are review-and-writing only. Writer and judge are separate models everywhere, per the anatomy's finding that the mock self-grades. The build backlog of record is the anatomy's by-hand inventory; a stage not in that table and not in this plan does not get built without being added here first, in writing. Every model call goes through the two interfaces and lands in a run row with its model id. Every stage has a contract, a rejection count, and a test. Anything cut by a valve is cut in writing, in the log, the day it is cut.

One-page summaries of the sources

66 summaries, alphabetical by first author. Each states at its foot how much of the source it was written from.

Foundations of Modern Query Languages for Graph Databases

Angles et al. 2017, ACM Computing Surveys 50(5), arXiv:1610.06264

This survey organizes graph querying by feature rather than by language, aiming to bridge database theory and the practical languages SPARQL, Cypher, and Gremlin. It fixes two data models: edge-labelled graphs (the RDF style, edges as label triples) and property graphs, for which the paper gives a new formalization as a tuple (V, E, rho, lambda, sigma) with labels and attribute maps on both nodes and edges. On top of these it builds a small feature hierarchy: basic graph patterns (equivalent to conjunctive queries), complex graph patterns adding projection, join, union, difference, optional, and filter, then regular path queries (RPQs) for arbitrary-length navigation, and their combinations into navigational graph patterns. Each feature is illustrated with worked queries in all three languages.

The load-bearing content is the semantics catalog and the complexity that follows from it. Pattern matching can run under homomorphism semantics (SPARQL, Gremlin), isomorphism variants (Cypher uses no-repeated-edge), or cheaper simulation semantics; path queries can return arbitrary, shortest, simple, or edge-distinct paths, each as bags or sets. Evaluating basic patterns with projection is NP-complete in combined complexity under both homomorphism and isomorphism semantics, but polynomial under simulation, and only logarithmic space in data complexity, where the query is fixed and only the graph grows. Adding optional or difference lifts complex patterns to PSPACE-complete. RPQs returning node pairs under arbitrary-path semantics run in linear time O(|G| times |L|), yet the same query under simple-path semantics is NP-complete even in data complexity. The authors also flag what is unknown: Gremlin is Turing-complete in full, and Cypher's no-repeated-edge semantics and path unwinding (already NP-hard) had, as of 2017, essentially no formal analysis.

For a personal knowledge backend over conversational history, the practical lesson is that the store will be a property graph in all but name (entities, typed relations, provenance attributes on edges), and that the expensive parts are choices, not fate. Fixed, small query templates over a growing graph sit in the logspace data-complexity regime, which is the scaling case an org-wide memory would actually hit; tree-shaped patterns stay tractable where cyclic ones do not (treewidth); and multi-hop recall queries (who told whom, transitively) are the linear-time RPQ case so long as the system returns endpoints rather than enumerating simple paths. The semantics table also explains why memory layers built on Neo4j versus RDF stores can return different answers to the same conceptual query.

Read from: pages 1-25 and 37-41 of 59

The Property Graph Database Model

Renzo Angles, AMW 2018, CEUR Vol-2100 paper 26

This short paper (ten pages) gives a formal definition of the property graph database model, the abstraction behind Neo4j, Amazon Neptune, Azure Cosmos DB, and Titan. Angles observes that despite the commercial dominance of property graphs there is no standard specification of the model, which fragments the market and blocks research that would apply across systems. Following Codd's framing of what a database model must contain, the paper supplies the three components in turn: a data structure (Section 2), integrity constraints (Section 3), and a query language with a Datalog-based semantics (Section 4). It is a definitional paper, not an implementation or evaluation; nothing is benchmarked and no system is built.

The data structure is a tuple G = (N, E, rho, lambda, sigma): finite node and edge sets, an incidence function mapping each edge to an ordered node pair, a partial labeling function assigning sets of labels to nodes and edges, and a partial property function assigning sets of atomic values to (object, property name) pairs. The definition deliberately permits multiple labels per object and multiple values per property. A schema is a second tuple (TN, TE, beta, delta) naming node types, edge types, typed properties, and which edge types are allowed between which node-type pairs; schema-instance consistency is then four validity conditions checked per node, edge, property, and endpoint pair. Mandatory versus optional properties, unique attributes, and cardinality constraints are sketched as extensions rather than fully developed. The query language combines SQL, SPARQL, and Cypher syntax (SELECT, FROM, MATCH, WHERE, plus UNMATCH for safe negation and UNION), and its semantics is given by translation to non-recursive safe Datalog with negation: a graph becomes Node, Edge, Label, and Prop facts (property identifiers exist specifically to support multivalued properties), and each query becomes a set of rules. The translation is shown by worked example only, not proved in general, and the language omits recursion, so path queries, the signature graph operation, are out of scope.

For a personal knowledge backend over conversational history, the schema machinery is the useful part: the delta function, which whitelists edge types between node-type pairs, is exactly the shape of a vocabulary contract for an extraction pipeline writing entities and relations into a store, and the optional-schema stance (all functions partial) matches how such a store grows before its ontology settles. The Datalog encoding also shows that a property graph reduces cleanly to four relational predicates, a useful mental model for judging whether a graph database is buying anything over a relational one at small scale. As a reading of how knowledge graphs are represented, this is the cited formalization of the property graph side, though its query fragment is weaker than what Cypher or the paper's own G-CORE successor provide.

Read from: full text

Survey Article: Inter-Coder Agreement for Computational Linguistics

Ron Artstein and Massimo Poesio, Computational Linguistics 34(4), 2008; DOI 10.1162/coli.07-034-R2 (ACL Anthology J08-4004)

This survey is the standard treatment of how to measure whether human annotators agree with each other enough for their labels to count as reliable data. Working from a decade of computational linguistics practice after Carletta's 1996 kappa proposal, the authors expose the mathematics and assumptions behind the main chance-corrected coefficients: Bennett's S, Scott's pi, Cohen's kappa, weighted kappa, and Krippendorff's alpha. All follow one formula, observed agreement minus expected agreement over one minus expected agreement; they differ only in how chance is modeled. S assumes a uniform category distribution (and can be gamed by adding categories nobody uses), pi estimates one shared distribution from the pooled data, and kappa gives each coder an individual distribution, which folds systematic coder bias into the chance term. Alpha generalizes pi through distance functions, handling multiple coders, missing values, and graded disagreement, and for nominal data it converges to multi-pi as items or coders grow.

The concrete guidance matters as much as the algebra. Raw percent agreement misleads: with a 95/5 category split, two coders agree 90.5 percent of the time by pure chance, so an observed 90 percent is worse than chance. On skewed data, reliability reduces to agreement on the rare category, roughly delta over delta plus epsilon in their two-parameter analysis. The pi versus kappa debate is settled by purpose, not by marginal divergence: single-distribution coefficients (pi, alpha) measure reproducibility of the annotation procedure and are right for reliability claims, while kappa speaks to trustworthiness of the specific coders; the numerical gap shrinks as coders are added, so they recommend more than two coders. On interpretation they are candid about limits: their experience says values above 0.8 mark reasonable quality, some useful discourse corpora sit near 0.7, and no single cutoff fits all purposes, so report the methodology and the confusion matrix, not just the score.

For the memory harness this paper governs the judge-calibration step inside LLM-judge scoring, after the three injection arms are run and answers need grading. Before trusting a schema-validated judge call to score arm outputs at scale, the judge must be checked against Justin's hand labels on the small calibration set, and that check should be reported as chance-corrected agreement, not raw match rate, since correct labels will be common and inflate percent agreement. Krippendorff's alpha is the right coefficient there (it tolerates a small item count via its unbiased estimator and admits partial credit for near-miss verdicts), with 0.8 as the working reliability bar and anything near 0.7 flagged as provisional.

Read from: pages 1-20 and 37-42 of the 42-page PDF (full coefficient development, bias and prevalence analysis, interpretation guidance, and references; the annotation-task case studies on pages 20-36 were skipped).

Personal Knowledge Graphs: A Research Agenda

Balog and Kenter, ICTIR 2019, DOI 10.1145/3341981.3344241

This is a four-page position paper from two Google researchers that defines the personal knowledge graph (PKG) as a distinct research object and lays out an agenda for studying it. The paper contains no system, dataset, or experiment; its method is conceptual. It defines the PKG, contrasts it with prior work (Montoya et al.'s PIM knowledge base, Gyrard et al.'s personalized health graph, Yen et al.'s life-event extraction from tweets, and personalized views over general KGs), then works through four problem themes, each anchored by an explicit research question, and closes with notes on evaluation, implementation, and utilization.

The definition it establishes: a PKG is structured knowledge about entities and relations of personal rather than general importance, with a mandatory "spiderweb" topology in which every node connects, directly or indirectly, to one central user node. Three properties separate PKGs from general KGs: entities need not be globally prominent (a friend, "my guitar", "Mom's dentist"); entity information can be radically sparse, sometimes just a type plus the relation to the user; and relations are often short-lived, the opposite of the stability criterion general KGs use for inclusion. The four research questions cover representation under sparse, ephemeral predicates (RQ1); entity linking when the target entity has little or no structured information and may not be a proper noun, which the authors frame as long-tail linking where linking, population, and NIL-detection intertwine (RQ2a, plus RQ2b on when to link against the PKG versus a general KG); automatic population and maintenance without human curators, where they note that data-hungry neural link prediction is unlikely to transfer (RQ3); and continuous, one-to-many, potentially two-way integration with external sources, with the user in the loop for conflicts (RQ4). On evaluation they are blunt: no open PKG dataset exists, logging real user interactions is costly and privacy-fraught, and synthetic data may be the only path.

For a personal knowledge backend over conversational history, the paper is a checklist of the hard parts rather than a solution: first-mention population, resolving "Jamie" or "my guitar" without any external profile, deciding what to store (only user-relevant attributes, not full entity records), and expiring ephemeral relations all map directly onto conversation-derived memory. The user-centered spiderweb constraint is a concrete schema decision a backend can adopt, though the paper says nothing about multi-user or organizational graphs, where the single central node no longer holds. As a representation reference it is a compact framing of how KGs are used (entity linking, population, verification, completeness prediction) with pointers into that literature, not a treatment of any mechanism in depth.

Read from: full text

A Survey on Temporal Knowledge Graph: Representation Learning and Applications

Cai, Mao, Zhou, Long, Wu, and Lan; arXiv preprint 2403.04782, 2024.

This is a survey of temporal knowledge graph representation learning (TKGRL): methods that learn low-dimensional embeddings for entities, relations, and timestamps in graphs whose facts are quadruples (head, relation, tail, timestamp) rather than static triples. The authors position it as the first broad survey of the area; prior surveys covered static KGs with a short temporal section, or only the completion subtask. The paper proceeds from problem formulation, datasets, and metrics, to a ten-category taxonomy of methods, to three application areas, and closes with future directions. The taxonomy chapters catalog roughly forty models from 2017 to 2024 at the level of representation space, encoder, and scoring function, without new experiments or head-to-head benchmark numbers, so the survey establishes structure rather than empirical rankings.

The standing infrastructure it documents is concrete. Four benchmark families dominate: ICEWS event data (ICEWS14 has 7,128 entities, 230 relations, 365 timestamps, 90,730 facts), GDELT (2,278,405 facts over 2,751 timestamps), Wikidata, and YAGO; evaluation is by mean reciprocal rank and Hit@k. The ten method categories are transformation (translation and rotation, extending TransE and RotatE with time hyperplanes or temporal rotations), tensor decomposition (order-4 CP and Tucker variants such as TComplEx and TuckERT), graph neural networks, capsule networks, autoregression (RE-NET, RE-GCN: snapshot sequences encoded by relational GCNs plus GRUs), temporal point processes for irregular intervals (Know-Evolve, Graph Hawkes), interpretability via subgraph reasoning or reinforcement learning, language-model methods (in-context forecasting, retrieval plus fine-tuning), few-shot learning for rare entities and relations, and a residual class including copy-generation and neural ODEs. A useful task split runs throughout: interpolation (completing missing past facts) versus extrapolation (forecasting future ones), which demand different machinery.

For a personal knowledge backend built over conversational history, the relevant contribution is the framing itself: facts that hold only during an interval, the quadruple representation, and the interpolation versus extrapolation distinction map directly onto memory that must record when something was true and predict what still is. The autoregressive snapshot view suggests treating accumulated conversation state as a timestamped graph sequence, and the few-shot chapter addresses exactly the sparse new entities a personal graph accumulates. The scalability discussion is candid that current benchmarks are far smaller than real-world graphs and that most methods ignore scaling, so organizational use remains an open problem; the LLM-integration section (using language models for relation descriptions and semantic enrichment) is directional rather than settled. As a map of how temporal knowledge is represented and scored, the survey is a serviceable entry point and reference index rather than a source of design-ready results.

Read from: full text, pages 1 through 25 of 36 (everything before the references), with the per-model embedding sections in chapter 3 skimmed rather than studied.

Structured Prompt Interrogation and Recursive Extraction of Semantics (SPIRES): a method for populating knowledge bases using zero-shot learning

J. Harry Caufield, Harshad Hegde, et al. Bioinformatics 40(3), 2024. DOI 10.1093/bioinformatics/btae104 (arXiv:2304.02711)

SPIRES is a zero-shot method for populating knowledge bases from raw text, built on one commitment: every LLM output must conform to a user-defined schema, and every named entity must ground to an identifier in an external ontology or vocabulary. The user writes the schema in LinkML, a formal language whose classes carry typed attributes, cardinality constraints, per-attribute prompts, and value sets defined by ontology queries. Given a schema, an entry-point class, and a text, SPIRES generates a pseudo-YAML template prompt, sends it to the model, parses the completion heuristically (line splitting on colons, since responses are not guaranteed valid YAML), and recurses: any attribute whose range is a nested class triggers a fresh SPIRES call on that value alone. Leaf entities are then grounded through the Ontology Access Kit against resources like FOODON, MeSH, and MONDO, and validated against identifier-prefix and value-set constraints. The implementation ships as the open-source OntoGPT Python package.

The grounding numbers make the argument. On 100 random Gene Ontology terms, SPIRES with GPT-3.5-turbo returned 98 correct identifiers; the same model prompted directly returned 3, hallucinating plausible-looking IDs for nearly all 100. GPT-4-turbo often refused to ground at all, and on MONDO disease terms SPIRES with GPT-4-turbo managed only 18 of 100 against 97 for GPT-3.5-turbo, so a newer model tier is not uniformly better at this stage. On the BC5CDR chemical-disease relation benchmark, with no fine-tuning, SPIRES scored F 41.16 (chunked, GPT-3.5) and 43.80 with GPT-4 unchunked, just below the average of the original challenge's 18 trained systems and near BioGPT's 44.98; the trained state of the art reached 0.57, so zero-shot extraction trades several F points for zero training data. Hallucinated relations were rare but real, including a technically true statement (lisinopril induces angio-oedema) that the source text never made, and the authors insist LLM output be validated before entering a KB.

For the memory harness, SPIRES is the published precedent for the fact-extraction stage, where the corpus becomes schema-shaped entity and dated-fact rows. It shows that schema validation plus external grounding converts silent hallucination into countable rejections, exactly the measured-rejection design of the harness's schema-validated calls; that model tiers are swappable behind one prompt-completion interface but must be measured per stage rather than assumed; and that recursive per-class prompting handles nesting that flat triple extraction cannot. Its scope is single-document extraction only: it says nothing about tree consolidation, injection arms, or judge scoring.

Read from: pages 1 through 20 of 21 (the full body and conclusion end on page 11; the rest is references), extracted with PyMuPDF.

Don’t Do RAG: When Cache-Augmented Generation is All You Need for Knowledge Tasks

Chan, Chen, Cheng, and Huang; WWW Companion 2025; arXiv:2412.15605

The paper proposes cache-augmented generation (CAG) as a replacement for retrieval-augmented generation when the knowledge base is small enough to fit in a long-context model’s window. Instead of retrieving passages per query, the whole document collection is preprocessed once into the model’s key-value cache, which is stored on disk or in memory; at inference time only the query is appended, and the model answers with the entire corpus already resident in its inference state. The method has three phases: offline preloading (a one-time encoding cost regardless of query count), inference over the cached context, and a cache reset that truncates the appended query tokens rather than reloading from disk. The claimed gains are zero retrieval latency, no retrieval errors, and a simpler architecture with no retriever to tune.

Experiments use Llama 3.1 8B on subsets of SQuAD and HotPotQA sized from 21k to 85k tokens (3 to 64 documents), against BM25 sparse retrieval and OpenAI dense embeddings via LlamaIndex at top-1 through top-10, scored with BERTScore. CAG wins in most configurations: on HotPotQA-small it scores 0.7951 against 0.7676 for the best sparse RAG setting, and on SQuAD-large 0.7734 against 0.7658. The margin narrows as the corpus grows; at HotPotQA-large CAG’s 0.7407 falls below sparse RAG top-5 at 0.7535, which the authors connect to known long-context degradation. Timing on HotPotQA shows CAG generation at 0.85 to 2.26 seconds across sizes, while plain in-context learning (encoding the reference text per query) takes 9.3 to 92.1 seconds, a roughly 40x gap at the large size. Sparse RAG beating dense RAG throughout suggests the benchmarks were not retrieval-hard. All results are for one 8B model on corpora under 100k tokens; the authors state plainly that the method is impractical for significantly larger datasets, naming small-company knowledge bases, FAQs, and call centers as the intended fit.

For a personal knowledge backend over conversational history, CAG marks one endpoint of a design spectrum: below some corpus size, retrieval infrastructure is pure overhead and precomputing the context is both faster and more accurate. A personal archive might start under that threshold; an organizational one will not, and the paper’s own large-corpus result shows the accuracy cost of stuffing everything into context. Its concluding suggestion, a preloaded foundation context with retrieval reserved for edge cases, is the more durable idea for memory systems: a tiered design where hot, stable knowledge lives in cache and the long tail stays behind a retriever. The precomputed-KV technique itself is memory-adjacent engineering worth tracking, since it treats an LLM’s inference state as a persistable artifact.

Read from: full text

BooookScore: A Systematic Exploration of Book-Length Summarization in the Era of LLMs

Chang, Lo, Goyal, and Iyyer, ICLR 2024, arXiv:2310.00785

The paper studies how well LLMs summarize books longer than 100K tokens, a task that forces the document to be chunked and the chunk summaries combined. The authors compare two prompting workflows: hierarchical merging, where chunk summaries are recursively merged until one remains, and incremental updating, where a running global summary is updated and compressed as the model steps through the book. Because existing benchmarks like BookSum sit in LLM pretraining data, they build a contamination-resistant corpus of 100 recently published books (average 190K tokens, no public summaries found for any of them) and collect 1,193 span-level human annotations on GPT-4 summaries, at a cost of about $3K and 100 annotator hours. From these they derive a taxonomy of eight coherence error types, including two not seen in earlier taxonomies for fine-tuned summarizers: causal omissions and salience errors. They then automate the annotation as BooookScore, a GPT-4 prompt that checks each summary sentence against the taxonomy; the metric is the fraction of error-free sentences.

Validation shows the automatic annotations have 78.2 percent precision against human judgment versus 79.7 percent for the human annotations themselves, and system-level scores computed from human and LLM annotations nearly coincide. Applying the metric across models (a $10K API study), Claude 2 and GPT-4 lead (hierarchical scores of 91.1 and 89.1 at chunk size 2048), Mixtral-8x7B roughly matches GPT-3.5-Turbo, and LLaMA2-7B-Instruct trails badly at 72.4 with 36 percent repeated trigrams and cannot perform incremental updating at all. Hierarchical merging is almost always more coherent than incremental updating, but the exception is instructive: Claude 2 with an 88K chunk jumps from 78.6 to 90.9 on incremental updating, meaning fewer update-and-compress steps largely erase the gap. Annotators still preferred incremental summaries for detail (83 to 11 percent) while preferring hierarchical ones overall (54 to 44), and overall preference did not correlate with the coherence score. The taxonomy and metric are both derived solely from GPT-4 outputs on English trade books, a scope limit the authors flag themselves.

For a personal knowledge backend built over conversational history, this is essentially a study of the two compression strategies such a system must choose between. Incremental updating is exactly the rolling-summary pattern most memory systems use, and the paper shows it degrades coherence through repeated compression unless each update window is large; hierarchical merging is more coherent but loses long-range dependencies and detail. The finding that omission errors (entity, event, causal) dominate the error distribution tells a memory designer what to audit for, and the reference-free, taxonomy-grounded LLM-judge protocol is a workable template for evaluating memory summaries where no gold answer exists.

Read from: pages 1-12 of 35 (full main text; pages 13-35 are references and appendices)

FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance

Lingjiao Chen, Matei Zaharia, James Zou; arXiv (cs.LG), 2023 (later TMLR); arXiv:2305.05176

Chen, Zaharia, and Zou survey the pricing of twelve commercial LLM APIs from five providers and find fees that differ by two orders of magnitude: 10M input tokens cost $0.20 on Textsynth's GPT-J but $30 on GPT-4. From this they lay out three families of cost-reduction strategy. Prompt adaptation shrinks what gets sent (prompt selection, query concatenation); LLM approximation substitutes cheaper infrastructure for the expensive model (completion caches, fine-tuning a small model on the big model's outputs); and LLM cascade sends each query through an ordered list of models, stopping at the first answer a learned scorer deems reliable. They implement the third strategy as FrugalGPT: a DistilBERT regression scorer rates each generation, and a specialized optimizer (pruning candidate lists by answer disagreement, interpolating the objective from samples) picks the three-model chain and per-model thresholds under a user-set budget.

The empirical results are specific. On HEADLINES, a financial news classification task, the learned cascade (GPT-J, then J1-L, then GPT-4, with score thresholds 0.96 and 0.37) beats GPT-4's accuracy by 1.5 points at one fifth its cost; matching GPT-4's accuracy costs $0.60 against GPT-4's $33.10, a 98.3 percent saving. Savings on OVERRULING and COQA are 73.3 and 59.2 percent, with accuracy gains up to about 4 to 5 percent at matched cost. The mechanism is generation diversity: for roughly 6 percent of HEADLINES queries GPT-4 errs where GPT-J succeeds. The results come with real caveats stated by the authors themselves: the cascade needs labeled training examples drawn from the same distribution as the test queries, learning it is an upfront cost that only pays off on large query volumes, and the evaluation covers short-answer classification-style tasks, not long-form generation.

For the memory harness this is the prior art governing the model-tier assignment across pipeline stages, the stage where schema-validated LLM calls (tree building from the corpus, entity extraction, dated-fact extraction, the three injection arms, LLM-judge scoring) get routed through swappable model tiers. FrugalGPT frames the exact question the stage-wise sensitivity experiment answers: which stages a mini-class model can own outright and where escalating to a strong model buys accuracy per dollar. Its distribution-match caveat carries over directly, since a routing rule tuned on one corpus may not transfer, and its labeled-data requirement suggests the harness's judge scores could double as the reliability signal a cascade would need.

Read from: full text of all 13 pages, extracted with pypdf.

IDAS: Intent Discovery with Abstractive Summarization

De Raedt et al. 2023, IDAS: Intent Discovery with Abstractive Summarization, NLP4ConvAI at ACL 2023

The paper tackles intent discovery: partitioning a pile of unlabeled utterances into clusters that each share a conversational goal, the usual first step in building a chatbot without paying for annotation. Prior methods train an unsupervised sentence encoder on the domain data, then cluster; the authors argue that what makes two encodings similar is opaque and often driven by syntax or noun overlap rather than intent. IDAS instead keeps the encoder frozen and does the disambiguation in text: an LLM (GPT-3 text-davinci-003, temperature 0) abstractively summarizes each utterance into a short label such as “when is next payday”. Because the task has no labeled examples for in-context learning, IDAS bootstraps them. It runs an initial K-means, takes the utterance nearest each centroid as a prototype, has the LLM label the prototypes with a five-word-max instruction, then labels every remaining utterance using its 8 nearest already-labeled neighbors as ICL demonstrations, growing the demonstration pool as it goes. Utterance and label encodings are averaged, smoothed over n’ nearest neighbors (n’ picked by silhouette score), and clustered with K-means at the true K.

Against MTP-CLNN, the unsupervised state of the art, IDAS gains an average of 3.94 ARI, 2.86 NMI, and 3.34 accuracy points across Banking (77 intents), StackOverflow (20 topics), and a private 1,257-utterance transport dataset (42 intents), with the largest single gain +7.42 ARI on Transport; on Banking the two methods are roughly tied when compared in matched settings. On CLINC (150 intents) IDAS reaches 79.02 ARI and 85.48 accuracy with zero labels, beating two semi-supervised methods that were given labels for 50% of the intents, and trailing only at the 75% ratio. Ablations show every strategy using generated labels beats raw utterance encodings (by 5 to 19 ARI points), nearest-neighbor demonstration selection clearly beats random, and doubling K in the prototype step barely hurts. The results all assume the true intent count is known, and the test sets are small (1,000 to 3,080 utterances); smaller models (curie, babbage, Flan-T5-XL) produced labels too poor to use.

For a personal knowledge backend over conversational history, the transferable idea is that summarize-then-embed beats embed-raw-text when you need clusters organized by meaning rather than surface form, and that a seed-and-grow labeling loop turns an unlabeled corpus into its own demonstration set. The K-must-be-known and per-utterance-LLM-call costs are exactly the constraints that bite at organizational scale, and the paper flags both as open problems.

Read from: full text

The Faiss Library

Douze et al., arXiv preprint, 2024, arXiv:2401.08281

This paper is the design document for Faiss, Meta's open-source C++ library (with Python bindings) for approximate nearest neighbor search over embedding vectors. Rather than proposing a single new algorithm, it explains how the library organizes a dozen index types around two tools, vector compression and non-exhaustive search, and how a user should trade off speed, memory, and accuracy under a fixed budget. The authors frame the whole problem as an "embedding contract": the embedding model makes distances meaningful, and the index's only job is to approximate exact search under the agreed metric. They are equally explicit about what Faiss is not: it extracts no features, runs as no service, and provides no database machinery such as sharding, transactions, or concurrent writes.

The benchmarks establish practical selection rules. On datasets from SIFT features (BIGANN, 128 dimensions) up to 768-dimensional Contriever text embeddings of English Wikipedia, additive quantizers win at small code sizes (around 8 bytes per vector), product-additive hybrids take over at larger budgets, and plain scalar quantizers suffice for long codes. For non-exhaustive search, graph indexes (HNSW, NSG) are recommended below roughly 1M vectors when memory is unconstrained, while inverted-file (IVF) indexes are the only option once compression is needed to fit in RAM; graph quality peaks at 64 edges per node and degrades beyond. Fitting a scaling model on BigANN at 1B vectors, search time grows as N to the 0.29 at 50 percent recall@1 and N to the 0.45 at 99 percent, so cost stays below the square root of N but climbs steeply with the accuracy target. On Deep10M under ANN-benchmarks, Faiss beats SCANN and roughly matches DiskANN's Vamana. A production case study indexes 1.5 trillion 144-dimensional vectors at 54 bytes each (PCA to 72 dimensions plus a 6-bit scalar quantizer), sharded across hundreds of servers into an 83 TiB memory map, reaching about 1 second per query.

For a personal memory backend over conversational history, the field guide here is the sizing logic: a personal corpus sits far below the 10k-vector threshold where indexing beats brute force, while an organizational one crosses into graph-index territory, so the same Faiss code covers both ends without redesign. The gaps matter too. Faiss stores no metadata, filters only through id-bit tricks, and its graph indexes handle deletion poorly; a memory system needs a database layer on top, which is exactly the Faiss-plus-DBMS split that Milvus and Pinecone embody. The retrieval-augmented language model work it cites (Atlas, kNN-LM, RA-DIT) is the direct lineage of persistent AI memory.

Read from: full text

From Local to Global: A GraphRAG Approach to Query-Focused Summarization

Edge et al. 2024, From Local to Global: A Graph RAG Approach to Query-Focused Summarization, arXiv:2404.16130

The paper targets a failure mode of conventional vector RAG: questions that require global understanding of a whole corpus ("what are the main themes here?") are query-focused summarization tasks, not retrieval tasks, and retrieving a handful of similar chunks cannot answer them. GraphRAG's answer is to spend indexing-time compute building structure. An LLM extracts entities, relationships, and factual claims from 600-token chunks, aggregates the duplicates into a weighted knowledge graph, then partitions that graph with hierarchical Leiden community detection. Each community, at every level of the hierarchy, gets a pre-written LLM summary, with higher levels built from the summaries below them. At query time the community summaries are shuffled, chunked, mapped in parallel to partial answers with self-rated helpfulness scores, and reduced into one global answer.

The evaluation compared four community levels (C0 root through C3 leaf) against direct map-reduce summarization of source text (TS) and vector RAG (SS), on two roughly one-million-token corpora: podcast transcripts (1,669 chunks, graph of 8,564 nodes and 20,691 edges) and news articles (3,197 chunks, 15,754 nodes). With 125 persona-generated sensemaking questions per dataset and GPT-4 as judge, every global condition beat vector RAG on comprehensiveness (72 to 83 percent win rates, $p<.001$) and diversity (62 to 82 percent), while vector RAG won the directness control criterion, as expected of shorter answers. A second experiment counting and clustering extracted factual claims (47,075 claims via Claimify) broadly confirmed the LLM-judge results, agreeing on 78 percent of non-tied comprehensiveness comparisons. The efficiency finding matters most: root-level C0 answers used 97 percent fewer context tokens than TS (26,657 versus about a million for podcasts) while still beating vector RAG at 72 and 62 percent. The authors state plainly that all of this is established only for these two corpora, this token scale, and GPT-4.

For a personal knowledge backend over conversational history, the transferable idea is the shape of the index: entities and relationships distilled once, then summarized at nested levels of abstraction, so that broad questions read a few cheap high-level summaries instead of re-digesting raw text. That is essentially a mechanized version of a summarize-once, query-summaries-forever memory tree, and the C0 token numbers quantify what the hierarchy buys. The paper also contributes evaluation machinery (persona-driven question generation, comprehensiveness and diversity criteria, claim-based validation) that applies directly to measuring any persistent memory system where no gold answers exist. Indexing cost is the caveat: 281 minutes of gpt-4-turbo calls for one million tokens, so incremental update strategy is left open.

Read from: pages 1-16 of 26 (full main text and references; appendices skipped)

Bitemporal History

Martin Fowler, martinfowler.com, April 2021, https://martinfowler.com/articles/bitemporal-history.html

This is a pattern essay, not an empirical study. Fowler works a single payroll example through the whole piece: on February 25 the company pays Sally her recorded $6000 monthly salary; on March 15 HR reports that she was actually raised to $6500 back on February 15; on April 5 HR corrects the raise to $6400. He uses the example to motivate treating time as two dimensions. Actual history is what happened in the world given perfect information; record history is what the system believed at each moment. He prefers the terms actual and record over the Snodgrass valid and transaction terminology used in SQL:2011, on the grounds that workshop audiences found the latter confusing. The essay proceeds through tables and plots (actual time on one axis, record time on the other), then generalizes and closes with implementation options.

The concrete machinery is small but exact. A bitemporal query takes two time parameters, in his notation sally.salaryAt('2021-02-25', '2021-03-25'), read as what we believed on March 25 her February 25 salary was. Horizontal slices of the two-axis plot give the actual history as known at some record time; vertical slices give how belief about one date evolved. The key structural claim is that while actual history stops being append-only once retroactive changes are allowed, record history stays append-only: corrections are layered on rather than overwritten, which yields a reliable audit trail of the modifications themselves. He is explicit about scope limits: bitemporality complicates a system significantly, is avoidable when retroactive change is forbidden or when actions capture their inputs (the check records the salary it used), and by itself cannot compute the downstream consequences of a correction, such as back pay or tax effects. For storage he sketches two schemes, a relational table with paired record and actual date ranges, and event sourcing where each event carries both an actual and a record timestamp and simpler read models are projected from the log.

For a knowledge backend built over conversational history, the bearing is direct. Facts extracted from conversations get corrected later, and the record dimension is what lets the system answer both what is true and what was believed when an action was taken, without destroying the audit trail. Fowler's generalization of record time into perspectives, where HR and Payroll hold different record histories of the same fact and a perspective can even be a counterfactual scenario, maps onto multi-agent or multi-user ingestion at organizational scale, though he cautions that stacking many perspective dimensions is rarely worth the complexity. This is design guidance from one practitioner's example, with no benchmarks or formal treatment, so it constrains schema choices rather than settling them.

Read from: full text

Grammar-Constrained Decoding for Structured NLP Tasks without Finetuning

Saibo Geng, Martin Josifoski, Maxime Peyrard, Robert West; EMNLP 2023; DOI: 10.18653/v1/2023.emnlp-main.674

Geng and colleagues argue that grammar-constrained decoding (GCD) is not a niche trick for parsers and code generators but a unified framework for structured NLP. The idea is to describe a task's entire output space as a formal grammar, then run an incremental parser (Grammatical Framework's, in their implementation) as a completion engine during decoding: at each step the parser reports which tokens can legally extend the partial output, and the model's probability distribution is pruned to that set. They write character-level grammars, compile them into tokenizer-specific token-level grammars, and show fourteen tasks expressible this way, from information extraction to coreference to extractive summarization. Their main extension is the input-dependent grammar (IDG), where the legal output space is rebuilt per input; thirteen of the fourteen tasks need one. The method works with any autoregressive model that exposes its vocabulary distribution, which excludes logit-hiding APIs.

The experiments use few-shot LLaMA and Vicuna models (7B to 33B) with no finetuning. On closed information extraction over SynthIE-text, constrained to 2.7 million Wikidata entities and 888 relations, unconstrained LLaMA-33B scores 17.5 F1 while the constrained version reaches 36.0, beating the finetuned GenIE T5-base at 34.8. On entity disambiguation across six benchmarks, constrained LLaMA-33B averages 80.3 accuracy against 54.1 unconstrained, and the input-dependent grammar beats the input-independent one by twelve points; the finetuned state of the art still leads at 89.4. Constituency parsing shows the limit: IDG guarantees 100 percent valid trees but LLaMA-33B tops out at 54.6 bracketing F1 against 95-plus for supervised parsers, since the task needs syntactic understanding no constraint can supply. GCD adds negligible per-token latency for two tasks and roughly 69 ms on cIE, and it induces a likelihood misalignment where the empty string becomes the top beam, fixable by length normalization.

For the memory harness, this is direct prior art for the extraction stages: pulling entity records and dated fact rows out of a corpus is exactly the extraction-shaped, restricted-vocabulary work the paper tests, and it grounds the claim that a cheap constrained model can approach a stronger or finetuned one there. It governs the schema-validated LLM call layer beneath the swappable model tiers: when a local model serves tree-node, entity, or fact extraction, decode-time constraints make schema-invalid output structurally impossible rather than merely counted, which cleans up the stage-wise sensitivity comparison across tiers. Its scope limits transfer too: constraints repair format and vocabulary, not comprehension, so stages needing genuine reasoning, and the LLM-judge scoring of the three injection arms, should not expect constraint-driven gains, and API-tier models cannot use the mechanism at all.

Read from: pages 1-9 of the 21-page PDF (full main text, introduction through conclusion and best practices), plus citation metadata from gap_survey_entries.json.

Intent Induction from Conversations for Task-Oriented Dialogue Track at DSTC 11

Gung et al. 2023, Intent Induction from Conversations (DSTC 11), SIGDIAL 2023, arXiv:2304.12982

This paper, from AWS AI Labs, describes a shared-task benchmark rather than a single method: the DSTC 11 challenge track on inducing customer intents from raw conversation transcripts. The authors argue that prior intent-mining work evaluated on repurposed classification datasets (BANK77, CLINC150, SNIPS) that lack full dialogues and realistic intent skew. They release three simulated call-center corpora: Insurance (948 conversations, 22 intents, used for development) and Banking (1,000 conversations, 29 intents) and Finance (2,000 conversations, 39 intents) for evaluation, averaging 59 to 71 turns per conversation, three to five times longer than MultiWOZ or SGD. Two subtasks are defined. Task 1 is intent clustering: assign a cluster label to each pre-tagged intentful turn, with the number of reference intents withheld (only a 5 to 50 range given). Task 2 is open intent induction: no turn labels, only noisy automatic InformIntent predictions; systems must produce a labeled training set for an intent classifier, and evaluation trains a logistic regression classifier on frozen all-mpnet-base-v2 embeddings and scores its predictions on an independent test set. Clustering accuracy via Hungarian matching is the primary metric.

Thirty-four teams entered Task 1 and 19 entered Task 2, all using clustering-based pipelines. The winner, T23, reached 69.8 ACC on Task 1 and 76.3 on Task 2, against baselines (k-means over sentence embeddings, with silhouette-based selection of k) at 55.8 and 63.6. Deep embedded clustering variants, particularly SCCL, dominated the top ten; nine of ten top systems used them, and DSE-RoBERTa-large plus supervised pre-training on public intent data marked the winner. Two analyses qualify the numbers. HDBSCAN systems that left noise points unassigned were punished by clustering metrics: propagating labels to T11's noise instances moved it from 33rd to 9th, exposing a weakness of clustering-style evaluation itself. And rerunning Task 2 scoring under nine different classifier encoders left the top ten ranks stable but shuffled ranks 11 through 15, so prediction-based evaluation carries measurable noise. All results are scoped to simulated (not genuinely live) English conversations in three financial-adjacent domains.

For a personal knowledge backend over conversational history, the relevant lesson is methodological. Task 2's framing, judge an induced schema by the downstream system it trains rather than by cluster purity on the source turns, is directly transferable to evaluating induced topic or era structures over an archive: a smaller, cleaner distillation can beat exhaustive coverage. The corpus statistics also matter at organizational scale: real request distributions are long-tailed, so any memory taxonomy induced from usage will concentrate on a few head topics, and the granularity trade-off the metrics here are built around (too many fine categories versus too few coarse ones) is the same one a schema over conversation history must manage.

Read from: pages 1-9 of 18 (full main text and results; remainder is references and appendices)

TextTiling: Segmenting Text into Multi-paragraph Subtopic Passages

Marti A. Hearst, Computational Linguistics 23(1), 1997, ACL Anthology J97-1003

TextTiling divides expository text into contiguous, nonoverlapping multi-paragraph segments, each covering one subtopic under the document's main topic, using nothing but patterns of word repetition. The algorithm runs in three stages. Tokenization lowercases the text, strips 898 stop words, reduces tokens to morphological roots, and regroups them into fixed-size pseudosentences of w tokens (default 20), because real sentences vary too much in length for fair comparison. Scoring assigns a lexical score to every pseudosentence gap; the strongest variant, block comparison, computes normalized inner products of raw term-frequency vectors over adjacent k-unit blocks (default k = 10) slid one unit at a time. Boundary identification smooths the score curve, computes a depth score at each valley (the summed drop from the flanking peaks), and marks boundaries where the depth exceeds a cutoff derived from the mean and standard deviation of the document's own depth scores, snapped to the nearest paragraph break.

The evaluation is small but careful. Seven readers segmented 12 magazine articles of 1,800 to 2,500 words; interjudge kappa averaged .647, at the low end of acceptability, which caps how precisely any algorithm can be scored. Against a three-of-seven majority gold standard, the blocks version reached precision .66 with recall .78 at the liberal cutoff, beating a 39 percent random baseline (.44/.42) but trailing the judges themselves (.81/.71). An earlier tf.idf weighting hurt rather than helped, as did thesaural relations; plain within-block frequency proved more dependable. On 44 concatenated Wall Street Journal articles the blocks version hit a .67 precision/recall break-even for article boundaries, though Hearst notes this task violates the algorithm's single-document assumption. Known failure modes include summary paragraphs that echo earlier vocabulary, quoted speech, and topic shifts carried by stop-listed pronouns.

For the memory harness, TextTiling governs the corpus-to-tree segmentation stage, the first cut before entity extraction, dated-fact capture, and the schema-validated LLM calls that build the tree. It is the natural deterministic arm to run beside the model-tier segmenters: fully reproducible, free per document, and parameterized by exactly four knobs, so any LLM-based chunker (and the downstream three-arm injection comparison scored by an LLM judge) must beat this baseline to justify its cost. Its named weakness, cohesion troughs missing topic shifts that share vocabulary, predicts where an LLM arm should win: narrative or dialogue-heavy sources rather than the expository prose the algorithm was designed and validated on.

Read from: full text of all 32 pages of the Computational Linguistics article (introduction, applications, discourse model, related methods, algorithm, evaluation, and the summary and future work section).

Reifying RDF: What Works Well With Wikidata?

Hernandez, Hogan, Krotzsch; SSWS workshop at ISWC 2015

The paper asks how best to represent Wikidata, whose statements carry qualifiers and references, in plain RDF so that off-the-shelf SPARQL engines can query it. Since a Wikidata statement annotates an entire subject-predicate-object relation, the authors first reduce the problem to encoding quads (s, p, o, i), where i is a statement identifier that functionally determines the triple; qualifiers then attach to i as ordinary triples. They compare four established encodings: standard reification, which spends three triples per quad on r:subject, r:predicate, and r:object; n-ary relations, the Erxleben et al. scheme Wikidata itself was using, which needs 2(n + m) triples for n quads over m distinct properties; singleton properties, which mint a fresh predicate per statement and need only 2n triples; and named graphs, which store the quad directly. They build all four datasets from a February 2015 Wikidata dump (57.1 million quads plus 237.6 million triples common to every format, so between 57 and 171 million model-specific tuples) and run 14 benchmark queries, expanded into model-specific versions, against 4store, BlazeGraph, GraphDB, Jena TDB, and Virtuoso.

The empirical picture is uneven in an instructive way. Singleton properties, the most concise encoding, broke four of the five engines: 4store and GraphDB could not index the millions of unique predicates, and Jena timed out on every query. 4store also failed to load the named graphs dataset. Only n-ary relations and named graphs could express the SPARQL property paths two queries required. Virtuoso handled all four models reliably; across all engines, named graphs was fastest in 17 of 70 engine-query cases, with standard reification and n-ary relations at 16 each, so no clear winner emerged among those three. The authors note the caveats themselves: cold-cache runs only, a 60-second timeout, one hardware setup, and a preliminary scope. They also observe that 108 two-triple encodings are possible in principle; only the well-known four were tested.

For a personal knowledge backend over conversational history, the load-bearing problem here is the same one: facts need provenance, temporal scope, and confidence attached to them, and the choice of how to reify those annotations determines which stores and query features remain usable. The concrete lessons transfer directly. Elegant-on-paper schemes can violate engine assumptions (predicate cardinality being the killer for singleton properties), quads with a statement identifier are a clean conceptual core, and the identifier join is highly selective, so the extra joins that reification adds cost less than expected but do complicate query planning. The paper is also a compact tour of how a real large knowledge graph, Wikidata at 65 million statements in 2015, actually structures qualified claims.

Read from: full text

Knowledge Graphs

Hogan et al., ACM Computing Surveys 54(4), 2021, arXiv:2003.02320

This is a tutorial survey by seventeen authors, grown out of a Dagstuhl seminar, that gives the field of knowledge graphs its first unified introduction. The authors adopt a deliberately inclusive definition: a knowledge graph is a graph of data meant to accumulate and convey knowledge of the real world, with nodes as entities and edges as relations. From there the paper walks through the full stack: graph data models (directed edge-labelled graphs such as RDF, property graphs as in Neo4j, heterogeneous graphs, and named-graph datasets), query languages (SPARQL, Cypher, Gremlin, G-CORE) and their shared primitives from basic graph patterns up through regular path queries, then schema, identity, and context, deductive reasoning with ontologies and rules, inductive methods, and the lifecycle of creation, quality assessment, refinement, and publication. A running example, a Chilean tourism knowledge graph, carries the formalism.

The paper's substance is organizational rather than experimental, but it fixes useful distinctions and facts. It separates three kinds of schema: semantic (RDFS and OWL, which license inferences under the open-world assumption), validating (ShEx and SHACL shapes, which check completeness of specified portions of the graph), and emergent (quotient graphs computed from latent structure). On identity it treats IRIs, owl:sameAs links, datatypes, and blank nodes as the machinery for saying when two nodes are the same thing. Its practice section quantifies the landscape as of 2021: Freebase held over three billion edges when it went read-only in March 2015, largely migrated to Wikidata; Cyc represents over 900 person-years of manual effort yielding 2.2 million facts and rules; NELL extracted 120 million edges from web text. Drawing on Noy et al., it lists six traits enterprise graphs (Google, Amazon, Facebook, LinkedIn, Bloomberg) share, including billion-edge scale, continuous refinement, and deliberately lightweight taxonomic ontologies rather than heavy formal ones.

For a personal knowledge backend over conversational history, the load-bearing ideas are the ones the paper opens with: graphs let you postpone schema definition while data and scope evolve, which is exactly the position of a system ingesting unpredictable conversations; the identity machinery answers when two mentions are one entity; and the context section (temporal validity, provenance, geographic scope) is the vocabulary for statements true only at a time or per a source. The framing of a knowledge graph as an ever-evolving shared substrate of knowledge within an organisation or community, plus the enterprise-scale trait list, is a direct sketch of what a personal store scaled to an organization would have to become, and the note that industry favors lightweight taxonomies over expressive ontologies is a useful caution against over-engineering the schema.

Read from: pages 1-20 and 74-78 of 135

Adaptive-RAG: Learning to Adapt Retrieval-Augmented Large Language Models through Question Complexity

Jeong et al., NAACL 2024, arXiv:2403.14403

Adaptive-RAG routes each incoming question to one of three answering strategies based on a predicted complexity level: no retrieval at all (the LLM answers from parametric knowledge), a single retrieve-then-read step, or an iterative multi-step loop that interleaves chain-of-thought reasoning with repeated retrieval. The router is a small T5-Large classifier (770M parameters) that maps a query to label A, B, or C before any answering begins, so the LLM and retriever themselves never change. Since no query-complexity dataset exists, the authors build training labels automatically two ways: run all three strategies on sampled queries and assign the label of the cheapest strategy that got the answer right, then fill remaining gaps using dataset provenance (single-hop benchmarks get B, multi-hop benchmarks get C). They train on about 400 queries per dataset and evaluate on a mixed pool of three single-hop sets (SQuAD, Natural Questions, TriviaQA) and three multi-hop sets (MuSiQue, HotpotQA, 2WikiMultiHopQA), 500 test queries each, with BM25 as the retriever throughout.

The headline result, on GPT-3.5-Turbo-Instruct averaged over all six datasets, is 50.91 F1 at 1.03 retrieval-and-generate steps per query, essentially matching the always-iterate multi-step baseline (50.87 F1) at roughly a third of its 2.81 steps, and beating binary adaptive baselines like Mallen et al.'s Adaptive Retrieval (48.20) and Self-RAG (22.98, though trained on a different base model). The pattern holds for FLAN-T5-XL (3B) and XXL (11B). The efficiency stakes are concrete: 0.35 seconds per no-retrieval query versus 27.18 for multi-step. Two caveats travel with these numbers. The classifier is only 54.52 percent accurate over the three labels, and an oracle router reaches 62.80 F1 at half a step per query, so routing quality, not the strategies, is the current ceiling. Classifier size barely matters; a 60M model performs within about one F1 point of the 770M one.

For a personal knowledge backend over conversational history, the transferable idea is that retrieval depth should be a per-query decision made before retrieval starts, by a component far cheaper than the answering model. Most lookups against a memory tree are single-hop ("what did we decide about X"), some need iterative traversal across sessions, and many need no lookup at all; paying multi-hop cost uniformly is exactly the waste this paper quantifies. The self-labeling trick also matters: usage logs can label their own routing data by recording which strategy sufficed, no annotation budget required. The scope limit is real, though: results cover factoid QA over Wikipedia-style corpora with BM25, not dense retrieval over informal conversational text.

Read from: pages 1-11 of 15

LongLLMLingua: Accelerating and Enhancing LLMs in Long Context Scenarios via Prompt Compression

Huiqiang Jiang et al., ACL 2024 (arXiv Oct 2023), arXiv:2310.06839

LongLLMLingua is a question-aware prompt compression method for long-context use of black-box LLMs. It starts from the observation that long prompts create three coupled problems: cost, degradation from irrelevant content, and position bias (the lost-in-the-middle effect). Building on LLMLingua, which prunes low-perplexity tokens using a small language model, the authors add question awareness at two granularities. Coarse-grained compression scores each retrieved document by the perplexity of the question conditioned on that document, with a restrictive statement appended to curb small-model hallucination, and keeps only high-scoring documents. Fine-grained compression then scores individual tokens by contrastive perplexity, the shift in a token's perplexity when the question is added to its conditioning context, which the appendix shows is equivalent to conditional pointwise mutual information. Two further pieces complete the pipeline: the surviving documents are reordered by their coarse importance scores so the most relevant land at the edges of the prompt, and a subsequence recovery step restores entity spans (names, dates, places) that token-level compression mangled, by matching response substrings back to the original prompt.

The numbers are strong within a specific regime. On NaturalQuestions with 20 documents and GPT-3.5-Turbo, LongLLMLingua beats the original uncompressed prompt by up to 21.4 points at roughly 4x fewer tokens, and its coarse scoring metric alone out-recalls BM25, OpenAI embeddings, SBERT, and reranker baselines. On LongBench at a 2,000-token budget it scores 48.3 versus 44.0 for the full 10k-token prompt, and end-to-end latency improves 1.4x to 2.6x at 2x to 6x compression. The ablations show every component earns its place; removing question awareness collapses performance to near-LLMLingua levels. The limitations are stated plainly: compression is question-specific, so the context cannot be cached across queries, the method costs about twice the compute of LLMLingua, and the coarse question-aware scoring may miss subtler context-question relationships than the tested QA benchmarks exercise.

For the harness, this paper governs the budgeted injection stage, the point where scored memory rows from the tree (entities, dated facts) are packed into a fixed token window before the schema-validated call goes out through a model tier. It supplies two cited design choices the injection arms can adopt or test: select what enters the budget by question-conditioned relevance rather than raw entropy, and order the survivors by score with the best at the edges. It is also the natural compression baseline for the model-sensitivity comparison across tiers, with the caching caveat carried alongside: per-question compression trades reuse for density, which matters if the same tree serves many questions.

Read from: full paper text extracted via pdfplumber, pages 1 through 11 of 20 (complete main text, conclusion, and limitations; remaining pages are references and appendices).

Evaluating Open-Domain Question Answering in the Era of Large Language Models

Ehsan Kamalloo, Nouha Dziri, Charles L. A. Clarke, Davood Rafiei; ACL 2023 (long paper, 2023.acl-long.307); arXiv:2305.06984

This paper asks whether lexical matching, still the default scoring method for open-domain QA, tells the truth about model quality once answers come from generative systems. The authors take 12 open-domain QA models, spanning retriever-reader pipelines (DPR, several FiD variants), end-to-end systems (EMDR2, EviGen), and closed-book InstructGPT in zero-shot and few-shot modes, and have their answers to a random 301-question subset of NQ-OPEN (1490 question/answer pairs) judged four ways: exact match, the supervised semantic matcher BEM, an InstructGPT-based judge prompted with question, gold answer, and candidate, and human annotation with a third-rater tiebreak (Fleiss kappa 72.8 percent). A parallel study on CuratedTREC 2002 adds regex-specified gold answers and 20-year-old TREC systems as baselines.

The headline finding is severe underestimation. InstructGPT zero-shot scores 12.6 percent under exact match but 71.4 percent under human judgment, a gap of nearly 60 points, and the few-shot variant reaches 75.8 percent, a new state of the art on the subset. Rankings shift, not just magnitudes: EMDR2 loses its top spot only under human evaluation. Semantic equivalence accounts for 50.3 percent of exact-match failures, with list-style questions (20.6 percent) and granularity mismatches (15.0 percent) next. Model judges correlate far better with humans than lexical metrics (Kendall tau 0.75 for the LLM judge and 0.70 for BEM, against 0.23 for EM), yet they fail exactly where it matters most for generative output: of 47 hallucinated long-form InstructGPT answers, the LLM judge wrongly accepted 30, BEM 18, and GPT-4 as judge still 9. Scope is explicitly limited to factoid questions with short gold answers; multi-hop, discrete, and causal reasoning are untested, and the human annotators were the authors themselves.

For the memory harness this paper governs the LLM-judge scoring stage that scores the three injection arms against the fixed gold answers of the certified-unknown novel's question set. It justifies the design directly: put the gold answer in the judge prompt rather than asking for open-ended grading, expect judge scores to run a few points below human truth, and treat any arm that emits long, elaborated answers as a known blind spot, since attribution failures slip past every automated judge tested. It also argues for regex or alias expansion of gold answers as a cheap first-pass filter before the model judge runs, since most mismatches are shallow syntactic variation that patterns catch.

Read from: full extracted text of the 16-page ACL PDF, pages 1 through 11 (body, conclusion, limitations); remaining pages are references and appendix.

Event Log Construction from Customer Service Conversations Using Natural Language Inference

Kecht, Egger, Kratsch, and Roglinger; ICPM 2021; doi:10.1109/ICPM53251.2021

The paper builds standardized process-mining event logs out of raw customer service conversations, using natural language inference instead of a trained classifier. Each customer inquiry or agent reply becomes the premise of an NLI pair; the hypothesis is a short sentence naming a candidate topic or activity, such as “This example asks for iOS version.” Facebook’s pre-trained BART model, called through the transformers library, returns an entailment probability for every message-hypothesis pair. The workflow has five steps: define domain topics and activities, hand-label a small sample (100 to 300 messages) with binary tags, run five-fold stratified cross-validation to pick a per-label decision threshold from candidates between 0.70 and 0.999 by maximizing the Matthews correlation coefficient, optionally rewrite hypotheses that score poorly, then classify the full corpus and export it as an IEEE XES log via PM4Py. No model training happens anywhere; this is zero-shot classification in the sense of Yin et al. 2019.

Evaluation uses English Twitter support conversations from AmazonHelp (288,828 tweets), AppleSupport (231,683), and SpotifyCares (88,774) after filtering, with a keyword matcher as baseline. With tuned hypotheses, 55 of 56 topic and activity classifications reached an MCC above 0.72 on the labeled samples; on 100 AppleSupport outbound tweets, four of seven activities scored a perfect 1.00 across all metrics. Hypothesis wording mattered a great deal: “This example is about prime” hit 0.91 MCC where the default phrasing managed 0.53, and one grammatically broken default hypothesis scored 0 where the fixed version scored 1. Keywords sometimes sufficed on their own (URL detection was perfect because every Twitter link starts with the same prefix). Three discovery algorithms (Alpha, Inductive, Heuristics Miner) recovered the same coherent process model from the resulting log, so the logs are usable, though the authors concede the mined model covers only the topics they thought to define and analysis stopped at each conversation’s first message.

For a personal knowledge backend over conversational history, the transferable finding is the labeling economics: roughly 300 hand-labeled examples plus threshold tuning were enough to structure 200,000-plus messages, because the pre-trained model carries the semantics and only the decision boundary needs calibration. That pattern (define a schema of event types, verify on a tiny sample, sweep the archive) maps directly onto turning a conversation archive into a queryable timeline of typed events, and adding a new event type never degrades existing ones since each label is inferred independently. The caveats transfer too: results come from short English tweets in three consumer-support domains, hypothesis phrasing is brittle and opaque, and the schema must be known in advance, which is exactly the part a memory system over open-ended conversations cannot assume.

Read from: full text

FABLES: Evaluating faithfulness and content selection in book-length summarization

Kim et al., COLM 2024, arXiv:2404.01261

The paper reports the first large-scale human evaluation of faithfulness in summaries of book-length documents (mean 121K tokens). The authors sidestep two chronic problems at once: data contamination, by using 26 fictional books published in 2023 or 2024, and annotation cost, by hiring 14 Upwork annotators who had already read the books for pleasure. Five models (GPT-3.5-Turbo, GPT-4, GPT-4-Turbo, Mixtral, Claude-3-Opus) each summarize every book via hierarchical merging; GPT-4 decomposes each summary into atomic, decontextualized claims; annotators label each claim faithful, unfaithful, partially supported, or unverifiable, with quoted evidence. The result is 3,158 annotated claims across 130 summaries, collected for $5.2K, about $40 per summary.

Three findings stand out. First, Claude-3-Opus was the most faithful summarizer at 90.9% faithful claims, ahead of GPT-4 and GPT-4-Turbo (about 78%), GPT-3.5-Turbo (72%), and Mixtral (69%); on this corpus of 26 recent novels, only five of 130 summaries were entirely accurate. Second, unfaithful claims cluster around character or relationship states (38.6%) and events (31.5%), and half require indirect, multi-hop reasoning over the narrative to refute, which distinguishes the task from entity-centric fact checking. Third, automatic verification largely fails: on a seven-book subset (723 claims), the best auto-rater, Claude-3-Opus given the entire book as context, reached only 58.2 F1 on detecting unfaithful claims, and it beat both BM25 retrieval (24.9 F1) and human-annotated evidence spans. Beyond faithfulness, coding of the summary-level comments yielded a taxonomy of omission errors: 33% to 65% of summaries omit key events, every model makes chronology errors, and the long-context models systematically over-weight book endings (at least 20% of Claude-3-Opus summaries).

For a personal knowledge backend built over conversational history, the negative results matter most. Retrieval-then-verify, the backbone of most memory and RAG validation schemes, collapses when the query is a claim about states, relationships, or arcs distributed across a long record rather than a localizable fact; even full-context reading by the strongest model of early 2024 could not reliably catch fabrications. A memory system that summarizes long histories into durable records will inherit exactly these failure modes: silent omission of key events, recency bias toward the end of the record, and confident state claims that no single passage can confirm or deny. The paper argues, credibly, that claim-level verification over long documents is itself a benchmark for long-context understanding, and its human-annotation protocol (readers who already know the source) is a workable template for auditing any summarization layer in a memory pipeline. Scope caveats: fiction only, English, 26 books, February 2024 model checkpoints.

Read from: pages 1-14 of 41 (full main text and conclusion; appendices skipped)

The NarrativeQA Reading Comprehension Challenge

Tomáš Kočiský, Jonathan Schwarz, Phil Blunsom, Chris Dyer, Karl Moritz Hermann, Gábor Melis, Edward Grefenstette; TACL 2018 (arXiv Dec 2017); arXiv:1712.07040

NarrativeQA is a reading comprehension benchmark built from 1,567 full stories, split evenly between Project Gutenberg books and scraped movie scripts, each matched to a human-verified Wikipedia plot summary. The central design move is annotation from the summary alone: Mechanical Turk workers wrote about 30 question and answer pairs per story while never seeing the full text, with copying blocked by instruction and by JavaScript limits on the annotation site. Because the questions were written against an abstractive retelling, answering them from the full story cannot reduce to span lookup; the reader must locate and integrate narrative information about entities, events, and their relations, sometimes across passages separated by hundreds of pages. The paper opens with a survey showing why earlier datasets (CNN/Daily Mail, CBT, SQuAD, NewsQA, MS MARCO, SearchQA) reward shallow salience or local span extraction instead.

The dataset holds 46,765 question and answer pairs; questions average 9.8 tokens and answers 4.73. Stories average around 62,000 tokens with a maximum near 430,000, against summaries averaging about 650. Only 29.57 percent of answers appear verbatim in the stories, versus 44.05 percent in the summaries. The experiments make the gap concrete: on the summaries task a simplified BiDAF span predictor reaches 36.30 test Rouge-L, within reach of the 57.02 human score, but on full stories the same model collapses to 6.22 while a retrieve-then-read pipeline (200-token chunks selected by question similarity, then AS Reader) tops out around 14, barely above a no-context Seq2Seq baseline. Retrieval itself is the bottleneck: the authors found chunk selection from a short question hard even for humans unfamiliar with the story. Scope limits apply in the same breath: the story count is modest, all evaluation is n-gram overlap or MRR rather than semantic judgment, human scores were measured on summaries only, and the 2017 baselines predate transformers, so the absolute numbers are floors, not ceilings.

For the Python memory harness this is the charter document for the question set and the injection experiment. It supplies the cited method for the certified-unknown novel: write questions from a summary-level view so answers demand narrative memory, then run the bare-model spot check that must fail without support, exactly the failure NarrativeQA demonstrates at book scale. The corpus-to-tree stage with entities and dated facts is the harness answer to the retrieval bottleneck the paper names, the three injection arms replay its no-context, retrieved-chunk, and summary-context contrast under schema-validated LLM calls across swappable model tiers, and the LLM-judge scoring replaces the overlap metrics the paper itself strains against with short abstractive answers.

Read from: all 12 pages of the PDF, full text extracted with pdfplumber.

Text Segmentation as a Supervised Learning Task

Omri Koshorek, Adir Cohen, Noam Mor, Michael Rotman, Jonathan Berant; NAACL 2018; arXiv:1803.09337

This short paper reframes text segmentation, the job of cutting a document into topically coherent spans, as supervised learning rather than the unsupervised clustering and graph methods that dominated prior work. The blocker had always been data: existing benchmarks were tiny (Choi's 920 synthetic concatenations, five Manifesto documents, a hundred-odd Wikipedia pages), and the synthetic ones do not resemble natural prose. The authors mine English Wikipedia tables of contents to build Wiki-727K, a corpus of 727,746 documents with hierarchical segment boundaries, split 80/10/10, then train a two-level model: a bidirectional LSTM builds each sentence embedding by max-pooling over word2vec inputs, and a second bidirectional LSTM reads the sentence sequence and emits a per-sentence probability that it ends a segment. Inference is greedy, cutting wherever the probability exceeds a threshold tuned on the validation set.

The results establish that natural data separates real methods from artifact-fitters. Under the Pk metric (Beeferman et al.'s sliding-window boundary error, with k set to half the mean segment length), the model scores 22.13 on the full Wiki-727K test set and 18.24 on the Wiki-50 subsample, against a random baseline near 53 and human performance of 14.97 on Wiki-50. GraphSeg, which beats everyone on the synthetic Choi data (5.6 to 7.2), collapses to 63.56 on Wiki-50, worse than random; word-similarity cues that distinguish concatenated unrelated documents fail on topic shifts inside one document. Scope limits ride along with the wins: the model is trained on top-level Wikipedia sections only, loses to Chen et al. on the chemistry Elements set (33.82 vs 20.1 word-level Pk, blamed on GoogleNews embeddings), and the human ceiling itself is soft since annotators saw few documents. Runtime is linear, 1.6 seconds per document on CPU against GraphSeg's 23.6.

For the memory harness this paper governs the corpus-to-tree stage, the very first cut that decides what a leaf node even is before entity extraction, dated-fact capture, or any schema-validated LLM call sees the text. Its contribution to that stage is less the LSTM than the evaluation contract: Pk against ground-truth boundaries gives the chunking step a real metric instead of eyeballed chunk quality, so an LLM-based segmenter routed through the swappable model tiers can be scored the same way as the cheap TextTiling arm, and segmentation error can be held apart from downstream injection-arm and LLM-judge effects. The GraphSeg collapse is also the standing warning that any segmenter must be validated on natural transcripts, not synthetic splices.

Read from: all 5 pages of the PDF (full paper, pypdf text extraction).

PDX: A Data Layout for Vector Similarity Search

Kuffo, Krippner, Boncz 2025, SIGMOD 2025, arXiv:2503.04422

The paper proposes PDX (Partition Dimensions Across), a storage layout for embedding collections that groups vectors into blocks and, within each block, stores each dimension's values contiguously, the way Parquet rowgroups hold columns. On top of it sits PDXearch, a search framework that computes distances dimension-by-dimension across 64 vectors at a time in tight scalar loops that compilers auto-vectorize. The design targets a specific weakness of the standard vector-by-vector layout: pruning algorithms such as ADSampling and BSA, which stop reading a vector's dimensions once a partial distance shows it cannot reach the top-k, interleave bounds checks with distance math and lose their advantage against plain SIMD linear scans. PDXearch scans an IVF bucket in three phases (a full scan of the first block to set a threshold, a warmup that fetches dimensions in exponentially growing steps, and a prune phase that skips discarded vectors once fewer than about 20 percent remain), keeping the code branchless.

On four CPUs (Intel Sapphire Rapids, AMD Zen4 and Zen3, Graviton4) and ten datasets from 16 to 1536 dimensions, the auto-vectorized PDX distance kernels averaged 2.0x over hand-written SIMD kernels on the horizontal layout, and 5.5x when 8 or fewer dimensions are read, the case pruning creates. In IVF index search on Zen4 at the highest recall, ADSampling on PDX ran 4.6x faster than its own SIMD horizontal version and 2.2x faster than FAISS IVF_FLAT, reaching 7.2x over FAISS on the OpenAI/1536 dataset on Intel. The paper also shows that horizontally stored ADSampling can lose to FAISS outright, so the pruning idea only pays off given the right layout. A companion algorithm, PDX-BOND, orders dimensions by how far the query sits from each dimension's block mean; it needs no preprocessing or transformation of the vectors and is exact, and it beat FAISS, USearch, and Milvus on exact search by 2.5x, 3.7x, and 4.0x on average. All results are single-threaded, in-memory, float32, on collections of 0.3 to 10 million vectors; graph indexes like HNSW are discussed but untested.

For a personal knowledge backend over conversational history, the relevant findings are the exact-search numbers and PDX-BOND's no-preprocessing property. A memory store that grows by appending new conversation embeddings daily cannot afford PCA retraining on every ingest, and at personal scale (well under a million vectors) exact search stays feasible, which sidesteps recall tuning entirely. At organizational scale the IVF results apply, and the layout composes with existing indexes rather than replacing them. The broader lesson for persistent AI memory work is that retrieval cost is partly a systems problem: the same algorithm swung from slower than a linear scan to 4.6x faster on the strength of data layout alone.

Read from: full text

Lost in the Middle: How Language Models Use Long Contexts

Nelson F. Liu et al., TACL 2024 (arXiv 2023), arXiv:2307.03172

This paper asks how well language models actually use the long contexts they can now accept, and it answers with controlled position experiments rather than aggregate benchmarks. The authors build two tasks where exactly one item in the context matters: multi-document question answering over NaturalQuestions-Open (2,655 queries, one answer-bearing Wikipedia passage among k minus 1 Contriever-retrieved distractors) and a synthetic key-value retrieval task over JSON objects of random UUID pairs. In both, they vary two knobs independently, the total context length (10, 20, or 30 documents; 75, 140, or 300 key-value pairs) and the position of the relevant item, then measure accuracy as a function of position across MPT-30B-Instruct, LongChat-13B (16K), GPT-3.5-Turbo, and Claude-1.3, each in base and extended-context variants.

The central finding is a U-shaped performance curve: accuracy is highest when the relevant item sits at the very beginning or very end of the context and drops sharply in the middle. The drop is not small; GPT-3.5-Turbo loses more than 20 points on multi-document QA, and in the 20- and 30-document settings its worst-case mid-context accuracy falls below its 56.1 percent closed-book score, meaning the documents actively hurt. Extended-context variants track their base models almost exactly when the input fits both windows, so a bigger window does not mean better use of it. Mechanistic probes sharpen the picture: encoder-decoder models stay flat only within their training-time sequence length, query-aware contextualization (repeating the query before the data) fixes the synthetic task but barely moves multi-document QA, and even non-instruction-tuned MPT-30B shows the same U shape. A retriever-reader case study finds reader accuracy saturating far before retriever recall; going from 20 to 50 retrieved documents buys roughly 1.5 percent. The scope is 2023-era models at modest lengths (roughly 2K to 16K tokens), with an extractive accuracy metric, so absolute numbers will not transfer to current models even if the protocol does.

For the memory harness, this paper governs the injection arm, the stage that packs scored memory rows (tree summaries, entities, dated facts) into a budgeted context before the schema-validated answer call. It dictates the ordering policy: the highest-scored rows go to the edges of the assembled prompt, never buried mid-block, and it supplies the baseline the ordering experiment must beat or replicate. It also cautions against the reflex of injecting more rows just because the model tier’s window allows it, since marginal rows past saturation add cost and can subtract accuracy. The position-sweep protocol itself is reusable as a harness diagnostic under LLM-judge scoring.

Read from: full extracted text of pages 1 through 12 of the 18-page PDF (all main sections, conclusion, and references; appendices skimmed only via in-text mentions).

User Feedback in Human-LLM Dialogues: A Lens to Understand Users But Noisy as a Learning Signal

Liu, Zhang, Choi 2025, User Feedback in Human-LLM Dialogues, EMNLP 2025, arXiv:2507.23158

The paper asks whether implicit user feedback in chat logs (a user saying “could you label the y-axis?” rather than clicking a thumbs-down) can be detected reliably and then used to improve a model. Working with two real interaction corpora, LMSYS-chat-1M and WildChat, the authors densely annotate 109 conversations (75 from LMSYS, 34 from WildChat), labeling every user turn after the first with a six-way ontology inherited from Don-Yehiya et al. 2024: positive feedback, four negative types (rephrasing, make aware with or without correction, ask for clarification), and no feedback. Inter-annotator agreement is Cohen’s kappa 0.70 for binary and 0.60 for fine-grained labels. They then build a GPT-4o-mini classifier with in-context examples and, in the second half, test whether using the content of negative feedback, not just its polarity, produces better training data: a stronger model regenerates the unsatisfying response conditioned on the user’s complaint, and the result is distilled into 7B models (Vicuna and Mistral) by supervised fine-tuning on 20K conversations.

The descriptive findings are the cleaner half. Feedback is common and skews negative: in later turns it appears in more than half of user utterances, while praise is rare. Refusals explain almost none of it (under 3 percent of sampled responses in either corpus). More troubling for naive harvesting, prompts that elicit positive feedback are slightly more toxic and lower quality than random ones; manual inspection traces this to users praising successful jailbreaks. Their detection prompt roughly doubles prior accuracy on the dense setting (41.6 versus 31.5 percent binary). The training results are mixed and scope-bound: feedback-conditioned regeneration beats regeneration from scratch on MTBench (80 short human-written questions; Vicuna-7b reaches 6.86 versus 6.38), but on WildBench (500 longer, more complex real-user questions) it loses (23.38 versus 27.97 for the same setup), and plain distillation from the stronger model on random conversations does best of all.

For anyone building a personal knowledge backend over conversational history, the annotation study is the useful part: it establishes that later turns are dense with correction signal marking unmet intent, exactly the turns a memory system should weight when inferring what a user actually wants, while the training results warn that this signal cannot be consumed raw. Positive reactions encode bad behavior as well as good, and inferring unspoken intent can erode adherence to well-specified requests. Both lessons rest on two 2023-era public corpora and 7B models, one of which (LMSYS) reflects evaluation play rather than real work, so organizational logs may behave differently.

Read from: full text

Evaluating Very Long-Term Conversational Memory of LLM Agents

Maharana et al. 2024, LoCoMo: Evaluating Very Long-Term Conversational Memory, ACL 2024, arXiv:2402.17753

The paper builds LoCoMo, a dataset of 50 very long two-person conversations, each averaging about 300 turns and 9,200 tokens across up to 35 sessions, roughly nine times the length of MSC, the previous standard. Rather than collect real conversations over months, the authors generate them: two gpt-3.5-turbo agents, each seeded with an expanded persona from MSC and a temporal event graph of up to 25 causally linked life events spanning 6 to 12 months, converse using a memory architecture borrowed from Park et al.'s generative agents (per-session summaries as short-term memory, per-turn observations as long-term memory). Agents can also share web-searched images and react to them. Human annotators then edited about 15 percent of turns and replaced or removed about 19 percent of images to fix long-range inconsistencies. On top of the dataset sit three tasks: question answering across five reasoning types (single-hop, multi-hop, temporal, open-domain, adversarial), event summarization scored with an adapted FactScore, and multimodal dialogue generation.

The headline finding is that every tested configuration sits far below the human QA benchmark of 87.9 F1. GPT-4-turbo with a 4K window is the best base model at 32.1 overall. Extending context helps ordinary recall (gpt-3.5-turbo-16k climbs from 24.1 to 37.8 as its window grows from 4K to 16K) but collapses on adversarial questions, falling to 2.1 F1 against 70.2 for GPT-4-turbo at 4K: given more context, the model hallucinates answers to unanswerable questions and misattributes events to the wrong speaker. Temporal reasoning stays weak everywhere. On event summarization the long-context model actually underperforms the 4K base model (39.9 versus 45.9 FactScore F1), and the common failure modes are missed causal links, hallucinated details, and wrong speaker attributions. All of this is on 50 English, LLM-generated-then-edited conversations, so the numbers describe that synthetic corpus, not real chat logs.

The most transferable result concerns memory representation. RAG over raw dialogue logs helps modestly, but RAG over "observations," per-turn assertions distilled about each speaker's life, is the strongest configuration: top-5 observations reach 41.4 overall F1, and performance degrades as more are retrieved, which the authors read as a signal-to-noise problem. Session summaries retrieve well but answer poorly, suggesting summarization loses the facts QA needs. For a personal knowledge backend over conversational history, that ordering (distilled assertions beat both raw transcripts and summaries, and retrieval precision beats retrieval volume) is a directly usable design finding, and the adversarial collapse is a warning that scale of context is not the same as memory.

Read from: pages 1-10 of 19 (body complete; remainder is references and appendix)

Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs

Malkov & Yashunin, Hierarchical Navigable Small World graphs, TPAMI, arXiv:1603.09320

The paper introduces HNSW, a graph index for approximate K-nearest-neighbor search that needs no auxiliary coarse-search structure. Elements are inserted one at a time; each is assigned a maximum layer drawn from an exponentially decaying distribution, and a proximity graph is built at every layer it occupies. Search starts at the sparse top layer, greedily descends to a local minimum, then drops a layer and repeats, so the structure behaves like a probabilistic skip list with proximity graphs in place of linked lists. The second contribution is a neighbor-selection heuristic: rather than linking a new element to its M closest candidates, it keeps only candidates closer to the new element than to any already-selected neighbor, which recovers a relative neighborhood subgraph and preserves connectivity across cluster boundaries.

The authors argue O(log N) search and O(N log N) construction, with the caveat that the analysis assumes exact Delaunay graphs and constant node degree; empirical support comes from low-dimensional data (d=4 and d=8 random vectors up to 10M elements), and they state that verifying the resilience of the approximation at d=128 would need infeasibly large datasets. Practical settings fall out of the experiments: M between 5 and 48, ground-layer cap Mmax0 = 2M, level normalizer mL = 1/ln(M), efConstruction near 100, and index overhead of roughly 60 to 450 bytes per object. On the ann-benchmarks datasets (1M SIFT, 1.2M GloVe, 2M CoPhIR, 1M DEEP, 60k MNIST) HNSW beats Annoy, FLANN, FALCONN, and VP-tree by wide margins, and on the wiki-8 dataset under Jensen-Shannon divergence it improves on plain NSW by almost three orders of magnitude in distance computations. Against Faiss product quantization on a 200M SIFT subset, HNSW reaches higher recall at faster query times and builds in 42 minutes versus 11 to 12 hours, but needs 64 Gb of RAM against Faiss's 23.5 to 30, and its query-time scaling there visibly deviates from pure logarithm.

For a conversational-memory backend, HNSW is the algorithm inside most current vector stores, so its tradeoffs are the field's tradeoffs. Insertion is truly incremental with no reshuffling, which fits an append-only history; it handles arbitrary metrics, including edit distance on a 1M DNA dataset. Two limits matter at organizational scale: the paper defers element update and deletion to future work, and distributed search is awkward because every query funnels through the top layer, with skip-graph techniques proposed but not implemented. RAM residency of the full graph is the cost of its speed.

Read from: full text

A Survey on Open Information Extraction

Christina Niklaus, Matthias Cetto, André Freitas, Siegfried Handschuh, COLING 2018, arXiv:1806.05599

This survey maps the first decade of Open Information Extraction, the task of pulling relational tuples of the form (arg1; rel; arg2) from text without a predefined relation inventory. It opens with the three founding requirements from Banko et al. 2007 (unsupervised extraction in a single corpus pass, shallow features that survive heterogeneous text, efficiency at Web scale), then organizes the systems that followed into four families: learning-based (TextRunner, WOE, OLLIE, ReNoun), rule-based (ReVerb, KrakeN, Exemplar, PropS, PredPatt), clause-based restructurers (ClausIE, Stanford Open IE), and systems that capture inter-proposition relationships (OLLIE's modifiers, CSD-IE, NestIE, MinIE, Graphene). The through-line is drift from the founding constraints: later systems lean on dependency parsers, trading the original speed and domain-independence claims for precision.

The mechanisms it establishes are concrete. ReVerb's POS-based regular expression covers about 85 percent of verb-mediated relational phrases in English while filtering incoherent and uninformative extractions. OLLIE bootstraps pattern templates from ReVerb seed tuples and adds attribution and clausal-modifier fields so hypothetical or reported claims are not stored as asserted fact. MinIE annotates polarity, modality, and attribution outside the tuple to keep extractions minimal. The evaluation critique is the survey's sharpest contribution: most systems were hand-scored on a few hundred sentences of news, Wikipedia, or Web text, with precision-style metrics and recall proxies like yield, so cross-system numbers rarely compare. Stanovsky and Dagan's benchmark (over 10,000 extractions across 3,200 sentences, built on assertedness, minimal propositions, and completeness principles) and RelVis with its six error classes are the proposed fixes, and at writing time almost no one had adopted them. The scope limits are stated in the same section: nearly all systems are English-only, and canonicalization and coreference are named as open problems, not solved ones.

For the memory harness, this paper governs the fact-row generation stage, where segmented conversation text becomes dated entity-and-fact rows before tree placement. It supplies the design vocabulary (assertedness, so beliefs and hypotheticals never enter the tree as facts; minimality, so each row carries one claim; attribution fields for who said what) that the schema-validated LLM extraction calls should enforce regardless of which model tier runs them. It equally warns the evaluation design: yield-style counts and hand-scored spot checks are the documented traps, so the LLM-judge scoring of the three injection arms needs a fixed gold slice and explicit error classes, RelVis-style, rather than raw extraction counts.

Read from: the full text of all 13 pages, extracted with PyMuPDF (sections 1 through 5 plus references).

NoLiMa: Long-Context Evaluation Beyond Literal Matching

Modarressi, Deilamsalehy, Dernoncourt, Bui, Rossi, Yoon, Schütze; ICML 2025 (PMLR 267); arXiv:2502.05167

The paper builds a needle-in-a-haystack benchmark in which the question and the hidden fact share almost no words, so a model cannot find the needle by pattern matching and must instead follow a latent association. A needle like “Actually, Yuki lives next to the Semper Opera House” answers the question “Which character has been to Dresden?” only if the model knows the opera house is in Dresden; two-hop variants stretch the chain further (Dresden to Saxony). The authors quantify the gap they are closing with ROUGE precision between question and relevant context: vanilla NIAH scores 0.905 R-1, NoLiMa scores 0.069. The benchmark uses 58 question-needle pairs built from 5 needle groups, haystacks assembled from snippets of 10 open-license books and filtered (with Llama 3.3 70B plus manual review) to remove accidental distractors and false answers, and 26 needle placements per length, giving 7,540 tests per context length.

Across 13 models claiming at least 128K context, short-context performance is strong (most base scores above 84 percent) but decays fast. At 32K tokens, 11 of 13 fall below half their base score; GPT-4o, the best performer, drops from 99.3 to 69.7. Defining effective length as the longest context holding 85 percent of the base score, most models sit at 2K or below; GPT-4o reaches 8K against a claimed 128K. The same Llama 3.1 70B that reaches 32K effective length on RULER manages 2K here, which isolates literal matching as the crutch. Two-hop questions and inverted fact order (keyword before character name) degrade further, and an aligned-depth analysis of the last 2K tokens shows the drop tracks total context length rather than needle position, implicating attention capacity rather than position encoding. CoT prompting helps (two-hop at 16K goes from 42.7 to 56.7 for Llama 3.3 70B) but does not fix it; on a hard subset, GPT-o1 and DeepSeek-R1 still fall below 50 percent of base at 32K. Restoring literal matches via multiple choice lifts two-hop 32K from 25.9 to 87.2, while a single distractor sentence sharing words with the question cuts GPT-4o’s effective length to 1K.

For a conversational-history backend, the message is that dumping months of transcripts into context and asking associative questions (“when did I decide X”) will fail well before the advertised window, especially when the query is phrased differently than the original conversation, which is the normal case. Retrieval must do semantic bridging before the model sees anything, keep contexts short, and avoid surfacing lexically similar but irrelevant passages, since those actively mislead. The scope caveat: results come from synthetic single-fact needles in book-text haystacks up to 32K (128K for two models), not from real dialogue data.

Read from: full text

Industry-scale Knowledge Graphs: Lessons and Challenges

Noy, Gao, Jain, Narayanan, Patterson, Taylor; ACM Queue 17(2) / CACM 62(8), 2019; DOI 10.1145/3331166

This is an experience paper, not an empirical study: practitioners from Google, Microsoft, Facebook, eBay, and IBM (expanding on a 2018 ISWC panel) each describe their production knowledge graph, then pool the challenges that recur across all five. It proceeds company by company through design decisions, then through shared open problems: entity disambiguation, type membership, changing knowledge, extraction from unstructured sources, and operating at scale.

The scale numbers frame everything. Google holds roughly 1 billion entities and 70 billion assertions; Microsoft's Bing graph about 2 billion entities and 55 billion facts; Facebook about 50 million entities and 500 million assertions; eBay expects 100 million products and over a billion triples; IBM's framework is proven past 100 million documents and 5 billion relationships. All five use strongly typed entities and relations under a schema or ontology; correctness at this size depends on machinery rather than curation. Bing's single entry for the actor Will Smith is assembled from 108,000 facts across 41 websites, against 200 Will Smiths in Wikipedia alone, which is why the authors name disambiguation the top industry challenge despite years of research. The paper offers mechanisms rather than solutions, and mostly self-reported ones with no benchmarks. Google bootstraps its schema from a small set of low-level structures plus metatypes (types as instances of types) to validate invariants at scale, such as painters predating their paintings. Facebook keeps provenance on every assertion, defines correctness as always being able to explain why an assertion was made, and rebuilds the whole graph from sources daily through an idempotent build. eBay funnels all writes through a replicated log feeding a low-latency document store and a separate analytical graph store. IBM uses polymorphic stores, treats source evidence as a primitive, and defers entity resolution to query time so that context can settle ambiguous names.

For a personal knowledge backend built over conversational history, the transferable material is architectural. Provenance-first assertions, identity managed separately from surface forms, late-binding disambiguation, and the log-plus-specialized-stores pattern all apply directly to conflicting, evolving conversational facts, and IBM's layering of general, domain, and private customer knowledge is essentially the personal-versus-shared split an organizational rollout would need. The authors also flag personalized on-device graphs, small enough to ship to a phone, as an open direction they invite research on. As a representation reference the paper shows the dominant industrial pattern, typed property graphs with schema-level reasoning, but it describes no query languages, storage internals, or evaluation, so it orients rather than specifies.

Read from: full text

RouteLLM: Learning to Route LLMs with Preference Data

Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M Waleed Kadous, Ion Stoica; arXiv (cs.LG), 2024 (v4 Feb 2025); ICLR 2025; arXiv:2406.18665

RouteLLM is a training framework for learning routers that decide, per query and before any generation happens, whether a strong expensive model or a weak cheap one should answer. The router is a win prediction model estimating the probability that the strong model beats the weak one on a given query, paired with a threshold that converts that probability into a routing decision; sweeping the threshold traces a cost-quality curve. The authors train four router architectures (similarity-weighted Bradley-Terry ranking, matrix factorization over model and query embeddings, a BERT classifier, and a Llama 3 8B causal classifier) on 65k pairwise human preference labels from Chatbot Arena, with models clustered into ten tiers to fight label sparsity, and evaluate on decontaminated MMLU, MT Bench, and GSM8K against a random-routing baseline.

The headline mechanism is data augmentation. Trained on Arena data alone, routers do well only on MT Bench (matrix factorization needs half the GPT-4 calls of random to recover 50 percent of the quality gap) and sit at chance on MMLU and GSM8K, which the authors trace to distribution mismatch via a benchmark-dataset similarity score. Adding roughly 1500 golden-labeled MMLU validation questions, under 2 percent of the training data, lifts every router about 20 percent past random on MMLU; adding 120K GPT-4-judge labels (about $700 to collect) raises MT Bench APGR improvement past 50 percent. Best routers cut cost 3.66x on MT Bench at 95 percent of GPT-4 quality, though only 1.4-1.5x on MMLU and GSM8K, and gains depend on augmentation data resembling the target distribution. The strongest result for reuse: the same routers, untouched, route Claude 3 Opus/Sonnet and Llama 3.1 70B/8B pairs at comparable quality, so the learned signal is query difficulty rather than model identity. Scope is binary strong-versus-weak routing only, and router rankings shift across benchmarks without a clean explanation.

For the memory harness, this governs the model-tier assignment stage: the layer where every schema-validated LLM call (segmentation of the corpus into the tree, entity extraction, dated-fact extraction) is bound to a model tier behind a swappable interface. The harness's planned sensitivity experiment fixes one tier per stage and measures degradation; RouteLLM supplies the natural extension, a per-call router in front of the same interface, and its model-pair transfer result argues the router would survive a tier swap without retraining. The catch carries over too: routing quality tracks how well training data matches the harness's actual call distribution, so per-stage judge-labeled samples would be the price of entry. The three injection arms and LLM-judge scoring sit downstream and are untouched by routing.

Read from: full extracted text of all 16 pages, including appendices.

ReasoningBank: Scaling Agent Self-Evolving with Reasoning Memory

Ouyang et al. 2026, ICLR 2026, arXiv:2509.25140

ReasoningBank is a memory framework for LLM agents that face a stream of tasks with no ground-truth feedback. Instead of storing raw trajectories (Synapse) or success-only workflows (AWM), it distills each finished episode into structured memory items, each with a title, a one-sentence description, and content holding the abstracted strategy or pitfall. An LLM-as-a-judge labels each trajectory success or failure without reference answers, and both outcomes feed extraction: successes yield validated strategies, failures yield preventative lessons. At each new task the agent retrieves the top-k items by embedding similarity and injects them into its system instruction; new items are consolidated back by simple addition, closing the loop. The authors also propose memory-aware test-time scaling (MaTTS), which spends extra compute per task (k parallel rollouts with self-contrast, or k sequential self-refinements) and aggregates across the redundant attempts to curate memory, rather than converting each rollout independently.

On WebArena (684 tasks, Map excluded) with Gemini-2.5-flash, ReasoningBank lifts overall success rate from 40.5 (no memory) to 48.8, and MaTTS at k=5 reaches 51.8; gains of +7.2 and +4.6 hold for Gemini-2.5-pro and Claude-3.7-sonnet. On SWE-Bench-Verified, resolve rate rises from 34.2 to 38.8 (flash) and 54.0 to 57.4 (pro), and Mind2Web improves across cross-task, cross-website, and cross-domain splits. Steps per task drop as well, up to 1.4 versus no memory, with the largest cuts on successful runs (2.1 fewer steps on Shopping, a 26.9 percent reduction), so memory shortens good paths rather than truncating bad ones. Ablations on WebArena-Shopping with flash show failures matter: success-only extraction gives 46.5, adding failures 49.7, while AWM degrades from 44.4 to 42.2 when fed failures. The judge itself is only 72.7 percent accurate there, yet simulated accuracy from 70 to 90 percent barely moves the results. A case study shows memory items maturing from procedural rules into compositional checking strategies over the task stream.

For a personal knowledge backend over conversational history, the transferable finding is representational: distilled, self-contained strategy units with title, description, and content beat both raw transcripts and success-only procedures, and failure records carry real signal if the schema is built to hold lessons rather than replays. The robustness to a noisy self-judge suggests curation can run without human labels, which is what organizational scale requires. Scope limits apply: benchmarks are web browsing and repository-level software fixes, streams are hundreds of tasks, consolidation is pure addition with no pruning or deduplication, so behavior over years of accumulation is untested.

Read from: pages 1-12 of 30

Unifying Large Language Models and Knowledge Graphs: A Roadmap

Pan et al. 2024, IEEE TKDE, arXiv:2306.08302

This is a survey and roadmap, not an experimental paper. The authors organize research on combining LLMs and knowledge graphs into three frameworks: KG-enhanced LLMs (graphs injected during pre-training, at inference, or used to probe and interpret models), LLM-augmented KGs (language models applied to KG embedding, completion, construction, graph-to-text generation, and question answering), and Synergized LLMs + KGs, where the two are trained or operated jointly for representation and reasoning. Under each framework they build a fine-grained taxonomy and tabulate representative methods, roughly a hundred systems spanning 2018 to 2023, from ERNIE and K-BERT through RAG, KEPLER, DRAGON, StructGPT, and Think-on-Graph, each tagged by technique and by the kind of graph or model involved.

Because it synthesizes rather than measures, its findings are the trade-offs it documents. Pre-training injection yields the strongest downstream performance on knowledge-intensive tasks but never permits knowledge updates without retraining; retrieval at inference handles frequently changing open-domain facts but the underlying model was never trained to exploit them, so results can be sub-optimal. For KG completion, encoder-style methods (KG-BERT and successors) must score every candidate entity and cannot handle unseen entities, while generator-style methods generalize and skip ranking but may produce entities that do not exist in the graph. The probing work it collects is sobering: LAMA-style cloze tests show LLMs recall popular facts, but a Wikidata study of low-frequency entities found scaling fails to appreciably improve memorization of tail facts, and a causal analysis (Shaobo et al. 2022) found models complete facts from positionally close words rather than knowledge-bearing ones.

For a personal knowledge backend over conversational history, two threads matter most. First, the LLM-augmented KG construction pipeline (entity discovery, coreference resolution, relation extraction, plus end-to-end systems like Grapher and PiVE, which uses a small T5 to iteratively correct a larger model's extracted triples) is exactly the machinery for distilling structured, queryable memory from raw transcripts, and the survey's own trade-off analysis argues for inference-time retrieval over parameter injection whenever the store must stay editable. Second, the agent-style reasoning line (StructGPT, Think-on-Graph) shows LLMs traversing a graph step by step without retraining, giving interpretable answers that cite paths, which is the pattern an organizational memory would need for auditability. The future-directions section flags the open problems such a system inherits: hallucination detection against the graph, the ripple effects of editing one fact, and injecting knowledge into black-box models where prompts are bounded by context length.

Read from: pages 1-24 of 28

QuALITY: Question Answering with Long Input Texts, Yes!

Richard Yuanzhe Pang et al., NAACL 2022, arXiv:2112.08608

QuALITY is a multiple-choice question answering dataset over English passages averaging about 5,000 tokens, mostly Project Gutenberg science fiction plus Slate and other nonfiction, all CC-BY or more permissive. Its central move is procedural. Twenty-two hired writers each read a full passage and write ten four-option questions; every question then passes through two validation gates. Untimed validation, with three to five annotators who read the whole text, certifies that a single answer is correct and unambiguous (6,737 of 7,620 questions survive, 88.4 percent). Speed validation gives five annotators only 45 seconds with the passage; questions that a majority of these skimmers get wrong, while untimed readers get them right, are labeled HARD. That subset is 3,360 questions, 49.9 percent of the dataset, and it is the paper's evidence that the questions cannot be answered by search or skimming.

The numbers earn the design. Human majority-vote accuracy is 93.5 percent overall and 89.1 percent on HARD, while the best baseline (DeBERTaV3-large with DPR sentence extraction, after intermediate training on RACE) reaches only 55.4 percent, 46.7 on HARD. Question-only baselines hit 43.3 percent, and even oracle extraction using the correct answer as the retrieval query tops out at 78.3 on dev, so relevant-excerpt retrieval alone does not solve the task. Predicting the option with highest lexical overlap scores 26.6 percent, near chance. Scope limits sit next to these results: the corpus is mainstream US English written by a relatively privileged population, passages cap near 8k tokens, the baselines predate modern long-context models, and the pipeline cost $9.10 per question, so the method is expensive to replicate at scale.

For the memory harness, QuALITY governs the question-set construction stage, the point where the certified-unknown corpus is turned into probes before any injection arm runs. Its two-gate protocol translates directly: an unambiguity check (schema-validated LLM calls can play the untimed annotators, with the judge tier swappable) and a shallow-access check, where a bare model or a snippet-only pass stands in for the 45-second skimmers. A question enters the counterfactual-miss or corrected-facts band only if it fails under shallow access and succeeds under full-narrative access, which is exactly what makes the three injection arms comparable and keeps LLM-judge scoring honest, since four discriminating options give the judge a closed decision rather than a free-text similarity call. The 26.6 percent lexical-overlap floor is the target any generated distractor set should match.

Read from: full extracted text of pages 1 to 8 (abstract through conclusion, ethics, and contributions) of the 23-page arXiv PDF; remaining pages are references and appendices, not read.

LLM Evaluators Recognize and Favor Their Own Generations

Arjun Panickssery, Samuel R. Bowman, Shi Feng; NeurIPS 2024 (oral), Advances in Neural Information Processing Systems 37; arXiv:2404.13076

The paper asks whether self-preference, the tendency of an LLM evaluator to score its own outputs above equal-quality alternatives, is driven by self-recognition: does the model prefer a text because it can tell the text is its own? Working on news summarization with 1,000 articles each from XSUM and CNN/DailyMail, the authors have GPT-4, GPT-3.5, and Llama-2-7b-chat each play generator, evaluator, and authorship identifier. They measure both properties in pairwise form (choose the better summary, or pick which of two is yours) and individual form (rate one summary, or answer yes/no on authorship), converting output token probabilities into confidence scores and averaging over both option orderings to cancel position bias, which is severe: GPT-4, GPT-3.5, and Llama reverse pairwise preferences under reordering 25, 58, and 89 percent of the time.

Out of the box, all three models show self-preference, and all three exceed chance at self-recognition; GPT-4 reaches 73.5 percent accuracy distinguishing its summaries from two other LLMs and humans, while weaker models mostly cannot separate themselves from stronger ones. Fine-tuning on as few as 500 pairwise examples pushes GPT-3.5 and Llama 2 above 90 percent self-recognition, and across all fine-tuned variants, including control models trained on length, vowel count, readability, and degenerate tasks, self-preference strength falls on a single linear trend against self-recognition score, transferring across datasets. The authors invalidate the reverse causal story (evaluators slightly disfavor fine-tuned models' generations, average preference 0.46), yet they are explicit that this is evidence toward causation, not proof: results cover one task family, three models, one prompt design, and lack ground-truth quality controls, so disproportionate preference is shown only indirectly, through paired self-preference scores summing above one.

For the memory harness, this paper governs the LLM-judge scoring stage that grades answers from the three injection arms on the query set. The stage-wise model-sensitivity experiment swaps the model tier behind extraction and answering, from a local 7B through frontier; if the judge were drawn from the same family as the arm under test, this result predicts inflated scores for that arm, corrupting the comparison. The design consequence is to pin one fixed strong judge across every arm and never let judge identity covary with the system under test, to strip authorship cues from judged transcripts where feasible, and to aggregate judge calls over both answer orderings, since the ordering-bias numbers here are as large as the effect being measured. The tree-building and extraction stages are untouched by this result; it constrains scoring only.

Read from: full paper text, pages 1 to 10 of the PDF (all body sections and conclusion) plus appendices A and B, extracted via pypdf; citation metadata from gap_survey_entries.json.

Generative Agents: Interactive Simulacra of Human Behavior

Park, O'Brien, Cai, Morris, Liang, and Bernstein, UIST 2023, arXiv:2304.03442

The paper builds twenty-five LLM-driven agents, each seeded with one paragraph of natural language identity, and runs them in Smallville, a Sims-like sandbox town, to test whether an architecture wrapped around a language model (gpt-3.5-turbo; GPT-4 was invitation-only at the time) can produce believable long-horizon behavior. The architecture has three parts. A memory stream records every experience as a timestamped natural language object; retrieval scores each record as an equally weighted sum of recency (exponential decay, factor 0.995 per game hour since last access), importance (the LLM rates poignancy 1 to 10 at creation), and relevance (cosine similarity of embeddings against a query), min-max normalized. Reflection fires when summed importance of recent events passes a threshold of 150, roughly two or three times per agent day: the model proposes three salient questions from the 100 most recent records, retrieves against each, and writes cited higher-level inferences back into the stream, forming trees whose leaves are raw observations. Planning drafts a day in five to eight chunks, then recursively decomposes to hour and then 5 to 15 minute actions; plans and reflections re-enter the stream so retrieval mixes all three.

In a controlled evaluation, 100 Prolific raters ranked interview responses from the full architecture, three ablations, and a crowdworker-authored baseline. The full system scored highest (TrueSkill mu 29.89) and each removed component cost believability, down to mu 21.21 with no memory at all; the fully ablated condition, which stands in for prior first-order prompting work, trailed the full system by a standardized effect of d = 8.16, and it was statistically indistinguishable from the human crowdworkers. In a two-game-day end-to-end run, knowledge of a party spread from one agent to thirteen (4% to 52%) and a mayoral candidacy from one to eight, with zero hallucinated claims of knowing; network density rose from 0.167 to 0.74 with 1.3% hallucinated acquaintance claims; five of twelve invitees attended the party. All of this holds only at this scale: 25 agents, two simulated days, one hand-built town, at a cost of thousands of dollars in tokens.

For a personal knowledge backend, the transferable finding is that a flat log plus scored retrieval is not enough; the ablations show believable use of history required the synthesized layer too, and the failure catalog (missed retrievals, embellished recall, one agent conflating a neighbor named Adam Smith with the economist) is a checklist of what such a backend must guard against. This paper fixed the memory-stream, reflection, retrieval-triple vocabulary that most subsequent persistent-memory work inherits. The authors also flag memory hacking, planted false events, as an open robustness problem, which becomes an access-control question at organizational scale.

Read from: full text

Beyond the Context Window: A Cost-Performance Analysis of Fact-Based Memory vs. Long-Context LLMs for Persistent Agents

Pollertlam & Kornsuwannawit (Bricks Technology), arXiv preprint, 2026, arXiv:2603.04814

The paper asks a deployment question: for an assistant that must remember a user across sessions, is it better to resend the whole conversation history to a long-context model every turn, or to extract facts once into a memory store and retrieve only what each query needs? The authors build the memory side on the open-source Mem0 pipeline (GPT-5-nano extracts atomic flat-typed facts, text-embedding-3-small embeds them into pgvector, top-20 retrieval feeds a GPT-5-mini reader) and compare it against long-context inference with GPT-5-mini and GPT-OSS-120B on three benchmarks: LongMemEval (500 conversations, about 101,600 tokens each), LoCoMo (10 dialogues, 1,986 questions), and PersonaMem v2 (222 conversations, 589 questions). Accuracy is scored by GPT-5-mini as judge with a three-vote majority; cost is modeled from actual API spend with a 90 percent prompt-caching discount applied to the long-context side from turn two onward.

Long-context GPT-5-mini wins decisively on factual recall: 92.85 percent versus 57.68 on LoCoMo and 82.40 versus 49.00 on LongMemEval, gaps of 35.2 and 33.4 points. On PersonaMem v2 the gap shrinks to 7.3 points, and the memory system edges out long-context GPT-OSS-120B (62.48 versus 60.50), because persona attributes are exactly the stable facts flat extraction captures well. The cost picture inverts with use. Extraction compresses a 101,600-token history to about 2,909 retrieved tokens (roughly 35:1), a one-time write of $0.0430 per user, after which each query costs about $0.0013; the cached long-context turn still scales with context length. At 100k tokens the memory system becomes cheaper after about ten turns and saves 26 percent by turn twenty; the break-even falls from 13 turns at 30k to 9 at 500k. An error analysis names what compression loses: precise temporal markers, implicit coreferences, and updates that never propagate (a “twice a week” fact surviving a later revision to three). All accuracy results are scoped to Mem0’s flat extraction; the authors caution they do not generalize to structured memory architectures like Zep or temporal semantic memory.

For a personal knowledge backend over conversational history, this paper supplies the economic argument and its price: at organizational scale, per-query cost stays near-flat only if you accept extraction loss, and the three failure modes are a concrete checklist of what a fact schema must handle (timestamps, role resolution, update propagation). The finding that answer-model capability, not context length, limited GPT-OSS-120B is a useful caution against attributing retrieval failures to the memory layer. The static-snapshot protocol, with no incremental updates during querying, marks the gap between these benchmarks and a live system.

Read from: full text

Zep: A Temporal Knowledge Graph Architecture for Agent Memory

Rasmussen, Paliychuk, Beauvais, Ryan, Chalef (Zep AI), arXiv preprint, 2025, arXiv:2501.13956

The paper describes Zep, a commercial memory service for LLM agents, and its core engine Graphiti, a temporally aware knowledge graph built continuously from conversation. Incoming messages become episode nodes, a non-lossy raw store; an LLM pipeline (gpt-4o-mini, with a reflection step to curb hallucination) extracts entities and facts from each message plus the last four for context, embeds them in 1024 dimensions, and resolves duplicates against existing nodes through hybrid search and an LLM judgment. Above the entity layer, communities of strongly connected entities carry map-reduce style summaries, maintained incrementally by label propagation rather than Leiden so the graph can extend without full recomputation. The distinctive piece is the bi-temporal model: every fact edge carries four timestamps, two for when the system learned and expired it and two for when it held true in the world, and new edges that contradict old ones invalidate them rather than overwrite them. Retrieval runs cosine similarity, BM25, and graph breadth-first search in parallel, reranks (RRF, MMR, mention frequency, node distance, or cross-encoder), and formats roughly 1.6k tokens of facts with validity ranges plus entity summaries.

On MemGPT's Deep Memory Retrieval benchmark (500 conversations of only about 60 messages each), Zep scores 94.8% with gpt-4-turbo against MemGPT's 93.4%, and 98.2% with gpt-4o-mini; the authors themselves note that simply stuffing the full conversation into context scores 94.4% and 98.0%, so DMR barely discriminates. The stronger evidence is LongMemEval, with conversations averaging 115k tokens: Zep beats the full-context baseline by 15.2% with gpt-4o-mini (63.8% vs 55.4%) and 18.5% with gpt-4o (71.2% vs 60.2%), while cutting latency about 90% (2.58s vs 28.9s with gpt-4o). Gains concentrate in temporal reasoning, multi-session synthesis, and preference questions; single-session-assistant questions actually drop 17.7% under gpt-4o, an admitted failure. MemGPT could not be run on LongMemEval at all.

For a personal memory backend over conversational history, this is close to a reference design: episodic raw storage under a derived semantic layer, entity resolution as the hard ongoing problem, and explicit fact validity intervals as the answer to knowledge that changes, which flat retrieval handles badly. The retrieval numbers also argue that a graph layer earns its cost mainly at scale; below context-window size it buys little accuracy. The paper doubles as a critique of the field's evaluation habits: DMR is too small and too fact-retrieval shaped, and no benchmark yet tests fusing conversation with structured business data, which is exactly the organizational-scale case. All results depend on chat-style corpora and OpenAI models.

Read from: full text

Collaborative Memory: Multi-User Memory Sharing in LLM Agents with Dynamic Access Control

Rezazadeh, Li, Lou, Zhao, Wei, Bao (Accenture Center for Advanced AI), arXiv preprint (under review), 2025, arXiv:2505.18279

The paper asks how a system serving many users with many LLM agents can share memory across user boundaries without leaking anything a given user is not entitled to see. Its answer is a framework, not a learned model: permissions are encoded as two time-varying bipartite graphs (user-to-agent and agent-to-resource), and memory is split into a private tier scoped to the originating user and a shared tier open to cross-user retrieval. Every memory fragment carries immutable provenance: creation time, contributing user, contributing agents, and resources touched. A fragment is readable in a given user-agent context only if its contributing agents fall within the user's current agent permissions and its source resources fall within the agent's current resource permissions, so revoking an edge in the graph retroactively hides fragments that depended on it. Read and write policies, configurable globally, per agent, or per user, filter and transform fragments on the way in and out; the implementation runs everything on gpt-4o with text-embedding-3-large retrieval, a coordinator that routes subqueries to specialist agents, and an aggregator that composes the final answer.

Three scenarios carry the evaluation, all at small scale. On MultiHop-RAG (609 news articles across six domains, 2,556 multi-hop questions, five users, six domain agents), fully shared memory holds accuracy above 0.90 while cutting external resource calls by up to 61 percent at 50 percent query overlap and 59 percent at 75 percent overlap, relative to per-user isolated memory. A second scenario uses 200 synthetic business queries under asymmetric roles (only a Strategy Director sees all four agents); with no ground truth available it reports only that partial sharing reduces resource calls for every role. The third uses SciQAG science QA (five categories, five users, 100 queries) with permissions granted through step t4 and revoked through t8: accuracy rises and falls with the access graph, resource calls decline over time as memory substitutes for retrieval, and logged access matrices show no fragment crossing a permission boundary. The authors concede the settings are simulated, the user counts modest, and that LLM-based policy enforcement can still occasionally breach policy.

For a personal knowledge backend built over one person's conversational history, the relevant machinery is the provenance-stamped fragment plus retrospective permission check: it is exactly the primitive needed if a single-user memory tree ever grows organizational tenants, since access can change after fragments exist without rewriting the store. The efficiency result also matters, as it quantifies memory's role as a cache that displaces repeated retrieval. Within the field, the paper marks the point where LLM memory work imported access-control formalism (RBAC and ABAC lineage) rather than assuming one global store, though it establishes a formulation and feasibility evidence, not a benchmark others can yet be measured against.

Read from: full text

From Isolated Conversations to Hierarchical Schemas: Dynamic Tree Memory Representation for LLMs

Rezazadeh, Li, Wei, Bao (Accenture Center for Advanced AI), ICLR 2025, arXiv:2410.14052

MemTree is an online memory system that stores an LLM agent's accumulating history as a dynamically grown tree rather than a flat embedding table. Each node holds aggregated text, an embedding of that text, and links to parent and children; depth corresponds to abstraction, with general summaries near the root and specifics at the leaves. New information is inserted by traversing from the root: at each node, cosine similarity against the children is compared to a depth-adaptive threshold (theta grows exponentially with depth, so deeper, more specific nodes demand closer matches). A sufficiently similar child is descended into; otherwise a new leaf is attached and traversal stops. Every ancestor on the path then has its summary rewritten by an LLM call to fold in the new content, and its embedding refreshed. Insertion is O(log N), and the authors connect the scheme to online top-down hierarchical clustering, proving a beta/3 approximation of the optimal Moseley-Wang revenue under a well-separatedness assumption. Retrieval collapses the tree: the query embedding is compared against every node at once, filtered by a threshold, and the top-k nodes are returned.

The evaluation, run with GPT-4o plus text-embedding-3-large and separately Llama-3.1-70B, covers four benchmarks. On the 15-round Multi-Session Chat sessions, simply feeding GPT-4o the full history wins (95.6 percent accuracy) since each dialogue is under a thousand tokens; among memory systems MemTree edges MemoryStream (84.8 vs 84.4) and beats MemGPT (70.4). The interesting result comes from MSC-E, the authors' 70-session extension to 200 dialogue rounds: there MemTree (82.5 overall) beats both full-history prompting (78.0) and MemoryStream (80.7), and holds accuracy steady across evidence positions where the naive baseline degrades. On QuALITY single-document QA with Llama-3.1-70B, MemTree (59.8) roughly matches the offline RAPTOR (59.0) despite building its index incrementally. On MultiHop RAG (609 news articles, 2,556 questions), MemTree reaches 80.5 overall, within 0.5 points of RAPTOR and above GraphRAG (78.3), and on temporal queries (68.4) it beats every baseline including human-annotated evidence. The learned tree over those 609 documents has 3,164 nodes, depth 13, branching factor 2.1. Per-item insertion averages 10 seconds; cumulative cost runs about 1.4x the offline methods.

For a personal knowledge backend over conversational history, the central lesson is that the online-versus-offline distinction matters more than the specific structure: RAPTOR and GraphRAG require full corpus rebuilds to absorb new material, while MemTree pays a modest ongoing tax to stay current, which is the right trade for a system ingesting conversations continuously. Two caveats travel with the results: parent summaries are LLM-rewritten on every insertion, so ancestor content drifts with the insertion order and inherently favors recent material (the authors show accuracy rising from 82.1 to 84.2 for late-positioned evidence), and the corpora tested top out at hundreds of documents, well short of organizational scale.

Read from: pages 1-12 of 34 (full main text; remainder is references and appendices)

Knowledge Graph Embedding for Link Prediction: A Comparative Analysis

Rossi, Firmani, Matinata, Merialdo, and Barbosa, ACM TKDD, 2021, arXiv:2002.00819

The paper retrains and re-evaluates 16 embedding-based link prediction models from scratch, spanning three families (tensor decomposition, geometric, and deep learning), against AnyBURL, a rule-mining baseline, on the five standard benchmarks: FB15k, WN18, FB15k-237, WN18RR, and YAGO3-10. Beyond the usual global metrics (Hits@K, mean rank, mean reciprocal rank), the authors define per-fact structural features of the training graph and correlate each with prediction accuracy: the number of "peers" (alternative valid answers seen in training), a TF-IDF style Relational Path Support score for paths up to three hops, relation properties such as symmetry and transitivity, and the degree of the original reified node for facts derived from FreeBase hyperedges.

Three findings carry the paper. First, the rule-based baseline holds its own: AnyBURL ranks at or near the top on almost every dataset and metric while learning its rules in 100 to 1,000 seconds, against roughly 1 to 300 hours of training for the embedding models; among those, only ComplEx with N3 regularization is consistently comparable. Second, graph structure largely determines difficulty: more source peers help (they are worked examples of the answer), more target peers hurt, and higher path support helps nearly every model. On FB15k and WN18 the leakage from inverse relations is so severe that the path-support score alone, used as a scoring function with two-hop paths on a 512-fact sample, reaches 93.55 percent H@1 on WN18. Third, evaluation plumbing matters: the policy for breaking score ties, rarely even reported, produces wildly incomparable numbers. CapsE scores 73.01 H@1 on FB15k under the min policy and 1.93 under average, and a perfect 100 versus 0 on YAGO3-10; the authors trace the ties to saturating activations (ReLU, and sigmoid in its near-flat regions) and show a leaky-ReLU variant mostly closes the gap.

For a personal knowledge backend over conversational history, the transferable lessons are less about any single model than about method. A graph distilled from one person's conversations will be sparse and skewed, with few peers and thin path support per fact, which is precisely the regime where every tested model performs worst; the cheap, interpretable rule-based approach is the sensible starting point before any embedding machinery. And the tie-policy result is a standing caution for persistent-memory evaluation generally: retrieval benchmarks can flatter or bury a system through unstated protocol details. Scope limit throughout: all results come from graphs of 14k to 123k entities sampled from FreeBase, WordNet, and YAGO, well below organizational scale.

Read from: full text

Bitemporal Property Graphs to Organize Evolving Systems

Rost, Fritzsche, Schons, Zimmer, Gawlick, Rahm; arXiv preprint, 2021; arXiv:2111.13499; a summary report on a one-year Oracle and University of Leipzig cooperation.

This paper reports the outcomes of a joint project between the University of Leipzig and Oracle on organizing relationships within multi-dimensional time-series data, with IoT sensor networks (an aircraft's instrumented components is the running example) as the motivating case. Rather than a single deep result, it presents four connected artifacts: TPGM+, a bitemporal property graph model; T-PGQL, a temporal extension of Oracle's PGQL query language; CGN, a continuous event-detection mechanism; and BiTeGra, a prototype database that stores the graphs in a relational system. The paper walks through each in turn, with the query language treated in the most detail.

The core of TPGM+ is bitemporality carried down to the property level. Every vertex, edge, and property value gets two close-open time intervals, valid-time (when the fact held in the world) and transaction-time (when the database learned it), plus five integrity constraints covering uniqueness, referential integrity of edges and of properties, constant edge endpoints, and constant labels. The property-level intervals are the delta over the authors' earlier TPGM, which had to copy an entire element whenever one property changed. T-PGQL exposes the intervals through dot-notation accessors (VAL_FROM, TX_TIME, and so on), SQL:2011 period predicates (CONTAINS, OVERLAPS, PRECEDES), a FOR TX_TIME clause for snapshot and range time travel with AS OF, FROM TO, BETWEEN AND, and ALL variants, and FIRST/LAST aggregates. CGN adapts Oracle's Continuous Query Notification to graphs, distinguishing notifications on any change to matched elements from notifications only when the query result changes. BiTeGra maps graphs to bitemporal relational tables and weighs schema choices explicitly: one big vertex-and-edge table pair versus a table per label, and properties as columns versus a key-value property table, with hybrid schemas (HyVE, HyPe) that record the per-label choice in metadata. The paper presents no benchmarks or evaluation; the language covers retrieval only, not data manipulation, and the system is a prototype.

For a personal knowledge backend over conversational history, the transferable idea is the two-clock design: separating when something was true from when the system recorded it is exactly what distinguishes "he changed jobs in May" from "I learned this in July," and doing it per property rather than per entity avoids duplicating a whole record when one fact changes. The FOR TX_TIME AS OF construction is a ready-made vocabulary for asking what the system believed at a past moment, useful for auditing an agent's evolving beliefs. The schema-mapping section is also a compact survey of how property graphs sit on relational storage, though the absence of any performance data means the design tradeoffs are argued, not measured.

Read from: full text

LLM-based Multi-Agent Blackboard System for Information Discovery in Data Science

Salemi et al. 2026, LLM-Based Multi-Agent Blackboard System, arXiv:2510.01285

The paper revives the classic blackboard architecture (Hearsay-II lineage, Erman et al. 1980) as a communication paradigm for LLM multi-agent systems, targeting data discovery: answering data science questions when the relevant files sit unidentified inside a large, heterogeneous data lake. Instead of a master agent assigning subtasks to workers it must already understand, the main agent posts a request describing what it needs to a shared blackboard. Helper agents, each responsible for one cluster of the data lake plus one web search agent, monitor the board and volunteer responses only when they judge themselves capable. The data lake is partitioned offline by giving file names to Gemini 2.5 Pro for clustering; each file agent then studies its cluster's structure in advance. The main agent runs a ReAct loop with five actions (plan, reason, execute code, request help, answer) capped at 10 steps, and its final output is a Python program that loads the right files and computes the answer.

Across KramaBench plus discovery-augmented versions of DSBench and DA-Code, the blackboard beats DS-GRU (all files in context), RAG over the lake (E5-large, top 5 files), and a master-slave variant that is otherwise identical, with 13 to 57 percent relative end-to-end gains over the best baseline. The pattern holds across four backbones (Gemini 2.5 Pro and Flash, Claude 4 Opus, Qwen3-Coder 30B): with Gemini 2.5 Pro, macro average rises from 24.0 (master-slave) to 28.5. File discovery F1 reaches 0.561 versus 0.513 for master-slave and 0.247 for RAG, again Gemini 2.5 Pro only. The advantage grows with lake size: the two largest lakes, DA-Code (145 files) and KramaBench Astronomy (1,556 files), show 70 and 211 percent relative gains over master-slave. Costs are real: roughly 2.3 times RAG's per-question spend, though runtime stays comparable (132 to 145 seconds across all three systems on a 50-question sample). Content-embedding clustering outperforms filename clustering, and more clusters generally help.

For a personal knowledge backend over conversational history, the transferable finding is architectural: routing by self-selection beats routing by a central index of who knows what, precisely because a coordinator's model of each partition goes stale. Partition the corpus, let each partition's agent pre-digest its holdings offline, broadcast queries, and let holders volunteer. That pattern sidesteps the retrieval weakness the authors document (embedding retrieval collapses to 0.247 F1 on this heterogeneous data) and scales in the direction an organization's archive grows. The caveats travel with it: results cover data lakes of at most 1,556 files, tabular and scientific domains, and a fixed two-level hierarchy.

Read from: pages 1-14 of 44 (full main text; remainder is references and appendices)

RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval

Sarthi, Abdullah, Tuli, Khanna, Goldie, Manning; ICLR 2024; arXiv:2401.18059

RAPTOR replaces the flat chunk store of standard retrieval augmentation with a summary tree built from the bottom up. The corpus is split into 100-token chunks (whole sentences preserved), embedded with SBERT, and then soft-clustered with Gaussian Mixture Models over UMAP-reduced embeddings; cluster count is chosen by BIC, and a chunk can join more than one cluster. Each cluster is summarized by gpt-3.5-turbo, the summaries are re-embedded, and the cycle repeats until clustering is no longer feasible, yielding a tree whose leaves are raw text and whose upper nodes are progressively broader abstractions. Because clustering runs on embeddings rather than document position, distant but related passages can share a parent, which is what separates RAPTOR from adjacency-based hierarchies like LlamaIndex or the recursive summarization of Wu et al. (2021). Construction cost scales linearly in tokens. At query time, the authors' preferred strategy simply collapses the tree: every node at every layer competes in one cosine-similarity search, and nodes are added until a token budget is reached (2,000 tokens, roughly the top 20 nodes, chosen after collapsed tree beat layer-by-layer traversal on 20 QASPER stories).

In controlled comparisons with UnifiedQA-3B as reader, adding the tree improved every retriever tested (SBERT, BM25, DPR) on all three benchmarks: NarrativeQA (1,572 book and movie transcripts), QASPER (5,049 questions over 1,585 NLP papers), and QuALITY (multiple-choice over passages averaging 5,000 tokens). With stronger readers the gains held: on QASPER, RAPTOR reached F-1 of 53.1 with GPT-3 and 55.7 with GPT-4, beating DPR by 1.8 to 4.5 points and edging past long-context models like CoLT5 XL (53.9). The headline result is QuALITY with GPT-4: 82.6 percent accuracy against a prior best of 62.3, and a 21.5-point lead over CoLISA on the hard subset. All corpora are single long documents or paper collections, not open-domain web scale, and an annotation study found about 4 percent of generated summaries contained minor hallucinations, though these did not propagate upward or measurably hurt QA. A layer ablation on one QuALITY story showed full-tree search (73.68) beating any single layer (about 58), evidence that the abstraction hierarchy itself, not just better chunking, does the work.

For a memory backend over conversational history, the transferable idea is that recall should span abstraction levels: some queries need the raw utterance, others need the season-of-work summary, and a collapsed search over both lets the query pick its own granularity. RAPTOR is also a batch-built, static index; conversations accrete, so an incremental variant of its cluster-summarize loop is the open engineering question, and its 4 percent summary hallucination rate is a caution for any system whose upper layers become the record people trust.

Read from: pages 1-12 of 23 (full main text; references and appendices skipped)

Developing Time-Oriented Database Applications in SQL

Richard T. Snodgrass, Morgan Kaufmann (Series in Data Management Systems), 1999, book, 528 pp. in this PDF

This is an orientation to a book, not a summary of a paper, and it draws on the parts that state the argument: the preface, chapter 1, and the opening of the bitemporal chapter. Snodgrass translates roughly 1600 research papers on temporal databases into guidance for people writing ordinary SQL. The preface opens with a development story that reads like a confession. A team adds one DATE column, finds the primary key no longer holds, adds a second DATE column and inherits a crop of off by one bugs where a comparison should have been less than or equal, or where a query needs a "+ 1 DAY" correction, then learns that two reports run days apart cannot be reconciled at all. The claim is that this is a category error, not sloppiness. The teaching runs through case studies from real installations: oil field records, cattle location data, a Danish mortgage bank.

The organizing scheme is three orthogonal triples. The temporal data types are instant, interval, and period. The kinds of time are user defined time (an uninterpreted value), valid time (when a fact was true in the modeled world), and transaction time (when a fact was stored). The kinds of statement are current, sequenced, and nonsequenced, and that split applies to queries, modifications, views, and constraints. Snodgrass is blunt about tool support as of 1999. SQL-92 offers DATE, TIME, TIMESTAMP, and INTERVAL but nothing for valid or transaction time; sequenced statements, which he calls the most useful case, get no support whatsoever; SQL3's Part 7, SQL/Temporal, which adds a PERIOD constructor, was not expected to reach ballot before the next millennium. The rest falls to the developer, with code fragments tested against DB2 UDB, Informix, Access, SQL Server, Sybase, Oracle8, and UniSQL, none fully compliant.

The bitemporal chapter is what bears on memory systems. Nykredit, a Danish mortgage bank carrying nine million loans against eight million customers and seven million properties, tracks both times on its property ownership table, giving each row six columns: two foreign keys, VT_Begin and VT_End, TT_Start and TT_Stop. The payoff is a repair procedure. When someone reports bad data, staff find when the error was stored, roll the table back to that transaction time, read the valid-time history as it then stood, and fix it; because transaction time is append only, the fix is itself logged. That is the operation a conversational memory backend needs when a stored belief turns out to be wrong. The costs are equally plain: a sequenced primary key cannot be expressed as a PRIMARY KEY constraint and needs an assertion, modifications multiply into nine forms, and the discipline lives in the application, not the engine.

Read from: pages 17-33 and 301-308 of 528

Cognitive Architectures for Language Agents

Sumers, Yao, Narasimhan, Griffiths 2024, Cognitive Architectures for Language Agents, TMLR, arXiv:2309.02427

This is a conceptual paper, not an experimental one. The authors argue that LLM-based agents recapitulate a sixty-year lineage running from Post's production systems through Markov algorithms to cognitive architectures like Soar, with an LLM playing the role of a probabilistic production system: where a production rewrites string XYZ to XWZ, an LLM defines a distribution over completions of a prompt. On that analogy they build CoALA, a framework that describes any language agent along three dimensions: memory (working memory plus long-term episodic, semantic, and procedural stores), an action space split into external grounding actions and internal retrieval, reasoning, and learning actions, and a decision cycle that plans (propose, evaluate, select) and then executes one grounding or learning action per loop.

The paper's product is the organizing power of that scheme rather than any benchmark number. Table 2 casts prominent agents into the framework and exposes real structural differences: SayCan is evaluation-only over a fixed set of 551 robot skills with no internal actions; ReAct adds reasoning but has no long-term memory or learning; Voyager exercises all four action types, learning by writing code skills into procedural memory; Generative Agents write LLM reflections over episodic memory into semantic memory; Tree of Thoughts has elaborate propose-evaluate-select decision-making but no persistence at all. From the gaps the survey exposes, the authors derive directions: learned retrieval procedures are essentially unstudied, deletion and modification of memory (unlearning) is neglected, and almost no agent treats learning as a deliberately chosen action rather than a fixed schedule. All of this is a reading of the 2022-2023 agent literature, so its claims about what is understudied are claims about that snapshot, not measurements.

For a personal knowledge backend built over conversational history, the useful contribution is the memory typology and its access rules. The paper's worked example is close to that exact problem: a retail assistant with read-only semantic memory (the inventory), read-write episodic memory (past interactions), and episodes summarized by the LLM before storage rather than logged raw. Reflection, in the Generative Agents sense, is the mechanism that turns accumulated episodes into reusable semantic facts, which is the step a conversation archive needs to scale beyond retrieval. The paper also gives the field a shared vocabulary; most persistent-memory systems since can be read as instantiations of its episodic-semantic-procedural split. Its caveat is worth keeping too: writing to procedural memory (an agent's own code or prompts) is flagged as the riskiest form of learning, and longer context windows may shift how much long-term memory matters at all.

Read from: full text

Toward a Theory of Hierarchical Memory for Language Agents

Talebirad, Parsaee, Szepesvari, Nadiri, and Zaiane; ICLR 2026; arXiv:2603.21564

This is a theory paper, not an empirical one. The authors observe that a wide range of long-context and agentic memory systems, from document-hierarchy retrievers like RAPTOR and GraphRAG to conversational memories like xMemory and H-MEM to agent execution-trace managers like MemoBrain and StackPlanner, all implement the same underlying pipeline, and they formalize it as three operators. Extraction (alpha) maps raw data to a graph of atomic information units; coarsening (C), a pair of a grouping function pi and a representative function rho, partitions units and assigns each group a compressed representative, iterated to build a multi-level hierarchy; traversal (tau) takes a query and a token budget and returns a subset of atoms. They prove simple information-theoretic facts about this structure: each deterministic, lossy coarsening level strictly decreases entropy, and by the data processing inequality mutual information with any query is monotonically nonincreasing up the hierarchy, so the atom layer sets the information ceiling for everything above it.

The central contribution is the self-sufficiency spectrum and the coarsening-traversal coupling. Self-sufficiency, SS = I(G; rho(G)) / H(G), measures how much of a group's information its representative preserves, from LLM summaries near 1 to bare category labels near 0. The coupling claim is that this value dictates viable retrieval: self-sufficient representatives support collapsed search over all levels at once (RAPTOR's pattern), while referential ones require top-down refinement (H-MEM, xMemory); mismatching the two wastes budget. The authors state plainly that this is consistent with existing evidence but has not been tested in a controlled comparison. Appendices add a computable LLM-based proxy SS-theta, a Fano bound capping branching factor at 2^((B+1)/(1-eps)) given B bits of routing information, an optimal branching factor of e under uniform cost, a depth bound L >= ceil(log_r(N/C)) tying corpus size N and context window C to required levels, and a coherence condition on pi (within-group affinity must exceed between-group affinity) that justifies top-down pruning. The whole framework assumes a static hierarchy; insertion, restructuring, and retrieval-driven decay break the Markov argument, which the authors name as the most pressing open problem.

For a personal knowledge backend built over conversational history, the paper supplies a design vocabulary rather than results: it says to choose summary style and search strategy jointly, that finer-grained extraction raises the retrieval ceiling, and that the depth bound predicts when one summary layer stops sufficing as a corpus grows toward organizational scale. Table 1's mapping of eleven systems onto (alpha, C, tau) is also a compact survey of the persistent-memory field as of early 2026.

Read from: full text

Let Me Speak Freely? A Study on the Impact of Format Restrictions on Performance of Large Language Models

Zhi Rui Tam, Cheng-Kuang Wu, Yi-Lin Tsai, Chieh-Yen Lin, Hung-yi Lee, Yun-Nung Chen. EMNLP 2024 Industry Track (arXiv:2408.02442, cs.CL 2024).

This paper asks whether forcing a language model to answer in a structured format degrades the quality of what it says, not just how it says it. The authors compare three progressively looser regimes on the same tasks: constrained decoding (JSON-mode), format-restricting instructions for JSON, XML, and YAML schemas, and a two-step NL-to-Format conversion where the model answers freely and then reformats. They test three reasoning datasets (GSM8K, Last Letter, Shuffled Objects) and four classification datasets across GPT-3.5-Turbo, Claude-3-Haiku, Gemini-1.5-Flash, LLaMA-3-8B, and Gemma-2-9B, averaging over nine prompt variants and using an LLM as a near-perfect answer parser validated against human annotation.

The central finding is that stricter constraints hurt reasoning more. On GSM8K, adding a schema to the JSON instruction drops Claude-3-Haiku from 86.99 to 23.44 and GPT-3.5-Turbo from 74.70 to 49.25 while inflating variance. The damage is not a parsing artifact: LLaMA-3-8B shows a 38.15 point Last Letter gap in JSON at a parsing error rate of only 0.148 percent. Mechanisms matter too; on Last Letter, JSON-mode placed the answer key before the reasoning key in 100 percent of GPT-3.5-Turbo responses, silently converting chain-of-thought into direct answering. Classification is the counterpoint: JSON-mode often matches or beats free text there, apparently by restricting the answer space. NL-to-Format roughly preserves natural-language accuracy, and a cheap corrective reformatting call repairs most parsing failures. All of this comes from small, low-cost models on simple two-field schemas; the authors state they could not test GPT-4o or 70B-class models, and gpt-4o-mini with OpenAI's stricter JSON-Schema mode narrows but does not close the gap (94.57 free text versus 91.71 on GSM8K).

For the memory harness, this governs the schema-validated LLM call layer that every extraction stage shares: distilling the corpus into tree nodes, building entity records, and emitting dated fact rows all demand structured output from swappable model tiers, and this paper says the schema itself can be the failure cause. A stage that collapses under a cheap constrained model may be reacting to the constraint, not the model, so the stage-wise sensitivity results need a constrained-versus-prompted ablation before blaming the tier, and the paper's regimes supply the arms. Its practical guidance maps cleanly: reasoning-heavy distillation favors loose formats or NL-to-Format with a repair pass, while classification-shaped steps such as entity linking can safely keep strict schemas. The caveat travels with the result: evidence covers small models and two-field schemas only, close to but not identical with the harness's richer record shapes.

Read from: full extracted text of the 19-page PDF (main body pages 1-8 plus the limitations section and appendices B through E).

Wikidata: A Free Collaborative Knowledgebase

Vrandecic and Krotzsch, Communications of the ACM 57(10), 2014, DOI 10.1145/2629489

This is a systems and design article by Wikidata's project director and its data model lead, describing the collaboratively edited knowledgebase Wikimedia launched in October 2012 to hold Wikipedia's factual data in one place. The motivating problem is duplication: the same fact (the population of Rome, say) lived in hundreds of independently edited language articles and disagreed across them, while the underlying data, spread over 30 million articles in 287 languages, was nearly impossible to extract. The article walks through the design decisions (open editing, community-controlled schema, plurality of conflicting claims, secondary rather than primary data, multilingual by construction), the data model, and early growth statistics, and situates the project against Semantic MediaWiki, Freebase, OpenCyc, DBpedia, and YAGO.

The core representational contribution is a layered statement model rather than plain triples. Items carry property-value pairs, with a small set of datatypes (quantity, item, string, time, coordinates, URL); properties are themselves first-class pages with labels and declared datatypes, and the community, not a fixed ontology, decides which properties exist. Statements take subordinate property-value pairs called qualifiers, so "population of Rome = 2,651,040" can carry "as of 2010" and "method: estimation" attached to the assertion rather than the item. Every claim can carry references, themselves lists of property-value pairs, and contributors can mark statements preferred or deprecated to manage contradictory sources; the model also distinguishes "value unknown" and "no value" from mere incompleteness. By August 2014 this held 15.8 million items, 43.2 million statements (23.2 million with references), and 1,176 properties, with property use heavily skewed toward "instance of." The numbers document adoption, not data quality; the authors note the data is far from complete and complex query support did not yet exist.

For a personal knowledge backend built over conversational history, the transferable pieces are the qualifier-and-reference structure (every stored fact carries its temporal context and its source, which is exactly the provenance a memory system needs when conversations contradict each other), the preferred/deprecated mechanism for coexisting conflicting claims instead of forced resolution, and stable never-reused IDs decoupled from labels. The bootstrapping story also matters: Wikidata gained its initial item set by importing an existing artifact (language links), and about 90 percent of its half million daily edits came from bots, with human curation layered on top, a plausible division of labor for automated extraction plus owner review. The scope limit is that Wikidata's schema governance depends on a large volunteer community; a single-user or single-organization system inherits the representation but must supply curation by other means.

Read from: full text

Large Language Models are not Fair Evaluators

Peiyi Wang et al., ACL 2024 (long paper), 2024, arXiv:2305.17926 / aclanthology.org/2024.acl-long.511

This paper demonstrates that the common practice of asking an LLM to compare two candidate responses is fragile in a specific, exploitable way: the ranking depends on which answer appears first in the prompt. The authors take the standard Vicuna benchmark template, in which GPT-4 or ChatGPT scores two assistant responses on a 1 to 10 scale, and simply swap the order of the responses. The verdict frequently flips even though the template explicitly instructs the judge to ignore presentation order. They quantify this with a conflict rate, the fraction of examples on which the two orderings yield contradictory verdicts, then propose three calibrations and validate them against 80 questions hand-annotated win/tie/lose by three of the authors.

The numbers are stark. With ChatGPT as judge comparing Vicuna-13B against ChatGPT, Vicuna's win rate is 2.5 percent in the first slot and 82.5 percent in the second, a conflict rate of 82.5 percent; GPT-4 shows the opposite lean, favoring the first position, at a 46.3 percent conflict rate. Bias concentrates where responses are close in quality: score gaps of 1 or less flip often, gaps of 3 or more rarely do. The fixes are Multiple Evidence Calibration (generate the explanation before the score, sample k times; k of 3 is the sweet spot), Balanced Position Calibration (run both orderings and average the 2k scores), and Human-in-the-Loop Calibration (an entropy score over the win/tie/lose outcomes flags unstable items for human review). MEC plus BPC lifts agreement with the human majority from 52.7 to 62.5 percent for GPT-4 and from 44.4 to 58.8 percent for ChatGPT; escalating the top 20 percent highest-entropy items to humans reaches roughly human-average accuracy at 39 percent less annotation cost. The scope is narrow, though: one 80-question benchmark, two judge models via the OpenAI API, pairwise templates only, and no account of why the bias exists.

For the memory harness this governs the LLM-judge scoring stage, the step that grades answers produced by the three injection arms after the corpus has been distilled into the tree of entities and dated facts. Any pairwise comparison across arms must run both orderings through the schema-validated judge call and average, whatever model tier is doing the judging, and single-order verdicts on close pairs should be treated as noise. The entropy-based escalation maps directly onto the small hand-labeled calibration set: route only the low-agreement items to Justin instead of labeling uniformly.

Read from: full text of the paper, all 11 pages including the conclusion and limitations, extracted with pdfplumber.

NovelQA: Benchmarking Question Answering on Documents Exceeding 200K Tokens

Cunxiang Wang et al., ICLR 2025 (arXiv 2024), arXiv:2403.12766

NovelQA is a long-context question-answering benchmark built from 89 English novels (65 public domain, 24 copyrighted) averaging over 200,000 tokens, well past the 60K-token ceiling of prior long-range datasets. Expert annotators, mostly English literature students who had already read the books they chose, wrote 2,305 question tuples, each carrying a question, a golden answer, four multichoice options, and located textual evidence from the novel. Half the questions came from ten-plus templates targeting known LLM weaknesses; the rest were free-form. Questions divide into multi-hop (35.0 percent), single-hop (42.8 percent), and detail (22.2 percent) complexity bands and seven aspect types (times, meaning, span, setting, relation, character, plot). Quality control kept only 79.4 percent of submissions and an inter-annotator test on multichoice answers hit 94.6 percent Cohen's kappa.

The evaluations establish that even 128K-200K context models fall well short of readers. Claude-3.5-Sonnet, the best model tested, reached 62.30 percent generative accuracy against a 90 percent human baseline (97 percent multichoice); Llama-3.1-70B managed 51.50 percent. Accuracy collapses when evidence sits past the 100K-token mark: the weighted generative average drops from 46.84 to 25.36 percent, a decline at the end of very long texts rather than the expected lost-in-the-middle dip. Generative scoring uses GPT-4 as judge, validated at 89.25 percent kappa against two human evaluators on 800 outputs, a defensible arrangement here because the questions are objective and evidence-grounded. Two caveats travel with these numbers: close-book runs show models already know the famous novels (GPT-4 scores 60.94 percent multichoice with no text at all), so open-book scores partly measure memorization, and the benchmark is English-only, novels-only, with truncation from the front when inputs overflow.

For the harness, NovelQA governs the question-design and evidence-annotation stage of the three injection arms. Its per-question evidence location is the template to copy when the corpus-to-tree ingest builds entity and dated-fact records: every probe question in the counterfactual and corrected-facts bands should carry a golden answer plus a pointer to the exact source span, so that a miss can be traced to retrieval versus synthesis. Its complexity-by-aspect taxonomy gives the schema for the schema-validated question-generation calls across model tiers, and its judge validation protocol (objective questions, kappa against human raters on a sample) is the procedure the LLM-judge scoring arm should reproduce before its scores are trusted. The close-book control is equally worth copying: without it, contamination masquerades as memory.

Read from: extracted text of pages 1-15 of the 23-page PDF, covering the full main paper (abstract through conclusion and references) plus the limitations, ethics, and opening appendix sections.

Searching for Best Practices in Retrieval-Augmented Generation

Wang et al., EMNLP 2024, arXiv:2407.01219

The paper treats a RAG pipeline as a sequence of swappable modules (query classification, chunking, embedding, vector database, retrieval, reranking, repacking, summarization, generator fine-tuning) and searches for the best method at each position. Since testing every combination is infeasible, the authors run a greedy three-step procedure: compare candidate methods per module in isolation, then optimize one module at a time within a full pipeline while holding the others fixed, then assemble recommended configurations. The end-to-end evaluation uses a Llama2-7B-Chat generator over a Milvus index of 10 million English Wikipedia passages plus 4 million medical texts, scored across commonsense reasoning, fact checking, open-domain QA, multihop QA, and medical QA, with RAGAs-style RAG metrics, sub-sampled to at most 500 examples per dataset.

The per-module findings come with specific conditions. Sentence-level chunking at 512 tokens scored best for faithfulness (97.59) on a corpus of just sixty pages of a Lyft 10-K, so the chunk-size result is thin evidence. LLM-Embedder matched BAAI/bge-large-en on MS MARCO retrieval at a third the size. On TREC DL19/20, HyDE plus hybrid search (BM25 weighted at alpha 0.3 against dense scores) gave the best retrieval quality, while query rewriting and decomposition did not help. For reranking on MS MARCO, monoT5 balanced quality and cost; TILDEv2 was 225 times faster (0.02 versus 4.5 seconds per query) at some accuracy loss. In the full pipeline, query classification (a BERT classifier deciding whether to retrieve at all, 0.95 accuracy) raised the average score from 0.428 to 0.443 and cut latency from 16.41 to 11.58 seconds per query. Reverse repacking, putting the most relevant document nearest the query, beat forward and sides. Fine-tuning the generator on a mix of one relevant and one random document made it most resistant to retrieval noise. The two recipes: best performance uses Hybrid with HyDE, monoT5, reverse, Recomp (average 0.483, about 11.7 s/query); the balanced recipe swaps in plain hybrid retrieval and TILDEv2, cutting latency to roughly 1.5 seconds at similar quality.

For a conversational-history backend, the transferable lessons are the deciding-when-to-retrieve gate (the single cheapest win in both accuracy and latency), hybrid sparse-plus-dense retrieval with a light reranker, and ordering context so the best evidence sits nearest the query. The caveat matters for memory work generally: these numbers come from public QA corpora and a 7B generator, and the paper itself concedes chunking was only lightly explored, so the recipes are starting priors rather than settled law for personal or organizational memory stores.

Read from: pages 1-16 of 22

Same Author or Just Same Topic? Towards Content-Independent Style Representations

Wegmann, Schraagen, Nguyen 2022, Same Author or Just Same Topic?, RepL4NLP 2022, arXiv:2204.04907

The paper attacks a confound in learning writing-style embeddings: models trained on authorship verification (AV, deciding whether two texts share an author) can score well by memorizing what an author writes about rather than how. The authors propose two changes to the training task. First, a contrastive AV setup (CAV): instead of a binary pair, the model sees an anchor utterance and must pick which of two candidates shares its author, trained with a triplet loss. Second, content control (CC): the different-author distractor is sampled from the same conversation as the anchor, from the same domain (subreddit), or at random, so that at the strictest level topic can no longer separate authors. They fine-tune siamese BERT, cased BERT, and RoBERTa models (RoBERTa consistently won on dev) on 210k training tasks built from a 2018 Reddit sample of 100 English-language subreddits, 600 conversations each, with a 70/15/15 author-disjoint split.

Conversation-controlled tasks are the hardest form of AV: every model scores lowest on that test set (around .69 AUC for models trained on it) and highest on the random one (up to .79 for the no-control model), consistent with the distractors carrying fewer content cues. The decisive evidence comes from a new STEL-Or-Content task, which pits a same-style paraphrase against a same-content, different-style sentence. Base RoBERTa picks style only 5 percent of the time; the CAV model with conversation control reaches .42 accuracy on the formal/informal dimension, against .24 or less for domain or no control. The authors argue this is not mere punishment of lexical overlap, since the same model holds even AV performance across all three test sets and normal STEL performance. Agglomerative clustering of its embeddings (7 clusters, 14,756 utterances) surfaces concrete stylistic signatures: terminal-punctuation habits, lowercase i, contraction spelling, apostrophe versus right single quotation mark, line breaks. All of this is one model family on one platform in one language; the conversation label also does not transfer directly to non-conversational corpora, a limit the paper states itself.

For a personal knowledge backend built over conversational history, the operative lesson is that “same source” signals are contaminated: any retrieval or user-modeling layer that embeds utterances will silently entangle who is speaking with what they discuss, and disentangling requires deliberately constructed hard negatives, here obtained for free from conversation structure. That trick, mining supervision from the thread a message sits in, scales to organizational chat archives without labeling. The paper also models an evaluation discipline that persistent-memory work mostly lacks: do not trust the training metric; build a probe where the shortcut and the target answer disagree.

Read from: full text

Machine Knowledge: Creation and Curation of Comprehensive Knowledge Bases

Weikum, Dong, Razniewski, Suchanek; Foundations and Trends in Databases, 2021; arXiv:2009.11564

This is a 289-page survey of how large knowledge bases (KBs) are built and maintained automatically from web and text sources. The authors organize the field into a pipeline of four phases: discovery (source selection, entity detection, taxonomy construction), canonicalization (entity linking and matching), augmentation (extracting attributes and relations, then growing an open schema when the fixed one runs out), and cleaning (constraint reasoning, rule mining, embeddings, provenance and temporal scoping). Each phase gets a methods chapter, ranging from hand-written regex patterns and distant supervision through CRFs, graph algorithms, and transformer-based extractors, followed by case studies of YAGO, DBpedia, NELL, Wikidata, and industrial graphs at Google and Amazon.

What the survey establishes is less a single result than a set of engineering conclusions drawn across two decades of systems. The quality bar they set is error rates below 5 percent, ideally under 1 percent, and they show the two goals of correctness and coverage pull in opposite directions: conservative extraction from premium sources (Wikipedia, IMDB, MayoClinic) gets the low-hanging fruit, then that initial KB seeds distant supervision for riskier open extraction. Public KBs hold millions of entities and up to billions of statements; proprietary ones run one to two orders of magnitude larger. Entities and types must come first, because attributes and relations are only as good as the canonicalized backbone under them. And no KB construction is a one-shot task: it is a life-cycle with humans (architects and curators) permanently in the loop, since no single method wins across the precision, recall, and cost trade-offs.

Two threads bear directly on a personal memory backend over conversational history. First, the wrap-up names the personal KB as an open research problem: an “augmented memory” on the user’s own device, built from a longitudinal digital footprint, with unresolved questions about what to capture, how to contextualize statements for interpretability, and how users manage the life-cycle. That is nearly a spec for a conversation-derived memory tree, and it inherits the survey’s machinery (canonicalization so one entity has one node, temporal scoping so facts age, completeness assessment so the system knows what it does not know). Second, the language-model discussion is a useful caution: they show a 2021-era model completes “Paris is located on the banks of the [?]” correctly but fails on Dylan’s protest songs, results they call “sort of correct, but completely useless,” which is their argument that latent model knowledge cannot replace an explicit, curated store. Scoped to pre-GPT-3-era models, but the structural point about verifiable symbolic memory versus parametric recall still frames the field.

Read from: pages 1-20 and 229-236 of 289

Efficient Guided Generation for Large Language Models

Brandon T. Willard, Rémi Louf; arXiv (cs.CL), 2023; implemented as the Outlines library; arXiv:2307.09702

Willard and Louf reformulate constrained text generation as transitions between the states of a finite-state machine. The standard way to force an LLM's output to match a regular expression or context-free grammar is to scan the entire vocabulary at every decoding step, zeroing the logits of tokens that would break the constraint; that costs O(N) per token, with N (vocabulary size) commonly above 50,000. Their alternative precomputes an index. Before generation starts, every vocabulary string is walked through the constraint's FSM from every viable start state (their Algorithm 3), producing a map from FSM states to the exact set of tokens acceptable in each state (Algorithm 4). At decode time the sampler tracks the current FSM state and looks up the valid-token mask in a hash map, so masking costs O(1) on average. The paper then extends the scheme past regular expressions to context-free grammars via pushdown automata and augmented LALR(1) parser operations, with a trie indexing parser stack configurations, which reaches formats like JSON, Python, and SQL.

The empirical claims are modest but pointed. Against the Guidance library on GPT-2 (N = 50,257), Outlines holds near-flat runtime as generated length grows while Guidance climbs past 100 seconds by 60 tokens; the comparison is a single timed sample per setting, and the authors themselves hedge about configuration oversights, so read it as an asymptotic argument rather than a benchmark. The memory trade is quantified once: naive un-reduced indices for an augmented Python grammar run about 50 MB. Worked examples on GPT2-medium show the character of the guarantee, and its limit: a regex-guided model always emits a well-formed year or IP address, but the values (1952 for Chomsky's birth year, 2.2.6.1 for Google DNS) are wrong. Structure is guaranteed; truth is not.

For the memory harness this is the contract mechanism at the extraction stage, the step that turns corpus text into tree nodes, entity records, and dated fact rows through schema-validated LLM calls. Because the FSM index is built from the schema and the vocabulary alone, the method is model agnostic, which is exactly what the swappable model tier design needs: when the local 7B is served through Ollama with Outlines-style enforcement, schema-invalid output becomes a property switched off at decode time rather than an error rate to count, and retry loops disappear from that stage. It says nothing about the downstream arms: injection strategy and LLM-judge scoring still measure semantic quality, since guided decoding guarantees only that every record parses, never that its contents are right.

Read from: the full 18-page paper (all sections, pages 1-18).

Recursively Summarizing Books with Human Feedback

Wu et al. (OpenAI Alignment team), arXiv preprint, 2021, arXiv:2109.10862

The paper trains GPT-3 models (6B and 175B parameters, 2048-token context) to summarize entire fiction novels by fixed recursive task decomposition: a chunking algorithm splits the book, the model summarizes each chunk, then summarizes concatenations of those summaries, recursing to depth 3 for books of hundreds of thousands of words. Each task is conditioned on prior summaries at the same level ("previous context") so summaries flow in order. A single model handles every level, trained first by behavioral cloning on human demonstrations, then by reinforcement learning against a reward model fit to human comparisons, following Stiennon et al. (2020). The framing is scalable oversight: labelers judge each subtask against only its local input, so no one has to read the whole book, which the authors estimate makes each data point over 50x cheaper than end-to-end collection.

On 40 popular books published in 2020 (unseen in pretraining, primarily narrative fiction averaging over 100K words), the best 175B RL model produced summaries rated 6 of 7 about 5 percent of the time and 5 of 7 over 15 percent, but the average stayed well below the human-written summaries; labeler agreement on relative model quality was near 80 percent. RL clearly beat behavioral cloning at 175B, less so at 6B, and comparisons became more label-efficient than demonstrations past 10-20K labels while being 3x faster to collect. Training only on the first subtree generalized as well as training on the full tree; the final full-tree model actually got worse, which the authors tie to compounding lower-level errors and auto-induced distributional shift. The 175B models set state of the art on BookSum (beating non-oracle baselines by 3-4 ROUGE points and every baseline on BERTScore), and their depth 1 summaries let a zero-shot 3B UnifiedQA model score competitively on NarrativeQA.

For a personal knowledge backend built over conversational history, this is the canonical evidence that hierarchical summarization can compress arbitrarily long corpora while keeping facts traceable back to source text, and that summary trees support downstream question answering without retrieval over raw text. Its failure modes are equally instructive: local context produced a Pride and Prejudice summary that turned a request for a dance into a marriage proposal, book-level output read as event lists rather than coherent narrative, and themes spread thinly across many chunks vanished. Any memory tree that distills leaves independently inherits exactly these risks (within this study's scope: fiction, GPT-3-era models, fixed decomposition), so cross-leaf context and error auditing at composition points are design requirements, not refinements.

Read from: pages 1-20 of 37

MemoryAgentBench: Evaluating Memory in LLM Agents via Incremental Multi-Turn Interactions

Yuanzhe Hu, Yu Wang, Julian McAuley (UC San Diego), ICLR 2026, arXiv:2507.05257

The paper builds a benchmark for what it calls memory agents: LLM systems that accumulate information across many turns rather than reading one long document. Drawing on memory science, the authors name four competencies a memory system needs: accurate retrieval, test-time learning, long-range understanding, and selective forgetting. Their core methodological move is to feed context incrementally, chunk by chunk, wrapped in user messages with explicit memorize-this instructions, so that the agent's storage mechanism actually fires; questions come only after ingestion is complete. They restructure existing long-context datasets (document QA, LongMemEval, intent classification, novel summarization) and add two of their own, EventQA for temporal recall over novels and FactConsolidation for overwriting facts with later contradictory ones. The result is 2,071 questions over contexts of 103K to 1.44M tokens, run against long-context agents, three flavors of RAG (BM25, embedding models, structure-augmented systems like GraphRAG and HippoRAG-v2), and agentic memory products including MemGPT, MIRIX, Mem0, Cognee, and Zep, most with GPT-4o-mini as backbone.

No method masters all four competencies. RAG agents beat their raw backbone on accurate retrieval (HippoRAG-v2 averages 65.1 against GPT-4o-mini's 49.2), but long-context models win test-time learning and long-range understanding, where retrieving top-k passages cannot deliver a global view; GraphRAG scores 0.4 on novel summarization. Selective forgetting breaks everyone: no system exceeds 28 percent on multi-hop fact updates, and even o4-mini, which reaches 80 on a 6K-token version, collapses to 14 at 32K, so the failure is length-driven, not task-driven. The commercial memory products fare worst overall (Mem0 21.1, Cognee 20.6, Zep 24.0, against 60.6 for plain long-context GPT-5-mini). Ablations show smaller chunks help retrieval but hurt holistic tasks, and a stronger backbone lifts MIRIX by 9.7 points while barely moving simple RAG. Scope caveats: one backbone family dominates the main table, and budget limited which agents were tested.

For anyone building a personal knowledge backend over conversational history, the incremental-ingestion protocol is exactly the deployment shape, and the findings are cautionary in a specific way: retrieval-first architectures, which is what a summarized archive with search amounts to, handle lookup well but fail at superseding stale facts and at questions requiring the whole corpus. That argues for explicit update-and-consolidation machinery (newest-wins provenance, periodic re-summarization) rather than append-and-retrieve alone. For the field, the paper supplies a shared vocabulary of four competencies and hard evidence that current commercial memory layers lag a plain long context, which reframes where the open problems actually are.

Read from: pages 1-10 of 33

Improving Unsupervised Dialogue Topic Segmentation with Utterance-Pair Coherence Scoring

Linzi Xing, Giuseppe Carenini; SIGDIAL 2021; arXiv:2106.06719

Xing and Carenini attack dialogue topic segmentation (DTS), the task of cutting a conversation into topically coherent runs of utterances, in the unsupervised setting that the field is stuck with because labeled dialogue boundaries are scarce; the labeled sets that exist are small or synthetic and get spent on evaluation. Their move is to keep the classic TextTiling inference (coherence scores over adjacent utterance pairs, converted to depth scores, boundary wherever depth exceeds a mean-minus-half-sigma threshold) but replace its surface-feature coherence measure with a fine-tuned Next Sentence Prediction BERT. Since no gold coherence labels exist, they manufacture training data from open-domain corpora (DailyDialog in English, NaturalConv in Chinese): adjacent utterance pairs that follow basic dialogue-act flows such as Questions-Inform become positives, and negatives come from swapping in a non-adjacent turn from the same dialogue or a turn from a different-topic dialogue, trained with a marginal ranking loss. This yields 91,581 English and 599,148 Chinese paired samples with no human annotation.

On DialSeg 711 the full model reaches Pk 26.80, WindowDiff 28.24, and F1 0.776 against 40.44, 44.63, and 0.608 for plain TextTiling, and it beats every baseline on Doc2Dial (Pk 45.23) and the Chinese ZYS set (Pk 40.99), though the Chinese gains are smaller and unexplained. The ablation is the sharpest finding: dropping the dialogue-flow constraint on positive sampling costs far more than dropping the topic constraint on negatives (Pk 32.60 versus 26.95 on DialSeg 711), so knowing that a question and its answer belong together matters more than knowing two dialogues discuss different topics. The paper also shows why document-trained segmenters underperform here: TeT variants powered by GloVe or vanilla BERT, which win on monologue text, fail to consistently beat bare TextTiling on conversational data, because signals learned from monologue prose mislead on fragmented, informal turns. Scope limits ride along with these numbers: all three test sets are goal-oriented or synthetic dialogues, inference is tied to TextTiling, and the method needs a large open-domain corpus in the target language.

For the memory harness, this governs the corpus-to-tree ingestion stage, the cut from raw conversation logs into segments before any entity extraction, dated-fact rowing, or tree placement happens. It is the published warning that a document-tuned chunker on chat transcripts is the wrong default, and its Pk and WindowDiff protocol supplies the ground-truth metric frame for scoring whatever segmenter, deterministic TextTiling arm or schema-validated LLM call through the swappable model tiers, feeds the three injection arms downstream; segmentation quality then becomes measurable input to the LLM-judge scores rather than an eyeballed assumption.

Read from: full paper, all 11 pages including conclusions and references, via pypdf text extraction.

Retrieval-Augmented Generation with Knowledge Graphs for Customer Service Question Answering

Xu, Cruz, Guevara, Wang, Deshpande, Wang, and Li (LinkedIn), SIGIR 2024 industry track, arXiv:2404.17723

This is a five-page industry paper describing a question-answering system deployed inside LinkedIn's customer service organization. The problem is retrieval over historical issue-tracking tickets (Jira), where conventional RAG treats tickets as plain text, chunks them to fit embedding-model context windows, and thereby loses two things: the internal structure of each ticket and the explicit links between tickets. The authors replace the flat text corpus with a knowledge graph built in two phases. Intra-ticket parsing turns each ticket into a tree whose nodes are sections (summary, description, steps to reproduce, priority, root cause, comments), using rule-based extraction for predefined fields and an LLM guided by a YAML template for the rest. Inter-ticket connection then links trees by explicit references recorded in Jira (clone, related-to) and by implicit edges added when the cosine similarity of ticket-title embeddings crosses a threshold. Node text is embedded with pretrained models (E5, BERT) into a vector database (Qdrant); the graph lives in Neo4j.

At query time an LLM parses the user's question into named entities keyed to template sections plus an intent (for example, steps to reproduce). Entity values are matched against section-specific node embeddings, node scores are summed to the ticket level to rank the top K tickets, and the LLM then rewrites the query with the winning ticket ID and translates it into a Cypher query that walks the tree to the intent node. A fallback reverts to plain text retrieval if query execution fails. On a curated golden dataset, with GPT-4 and E5 held fixed in both arms, the method lifts MRR from 0.522 to 0.927 (a 77.6 percent gain), Recall@3 from 0.640 to 1.000, and BLEU from 0.057 to 0.377. In a randomized split of the support team over roughly six months, median per-issue resolution time fell 28.6 percent (P50 from 7 to 5 hours, mean from 40 to 15). The golden dataset is small and internal, the graph template is hand-built, and the benchmark numbers reflect one domain, so the figures establish a deployment result rather than a general benchmark.

For a personal knowledge backend over conversational history, the transferable idea is the dual-level representation: parse each unit (here a ticket, there a session or leaf) into a section tree, then connect units by explicit references and thresholded embedding similarity, so retrieval can return a coherent subgraph instead of arbitrary chunks. The section-aware scoring, which sums similarity only over nodes of the queried type, is a concrete mechanism for intent-scoped retrieval, and the paper's own future-work list (automatic template extraction, dynamic graph updates from queries) marks exactly the parts that were manual here and would need to be automatic at organizational scale.

Read from: full text

Extract, Define, Canonicalize: An LLM-based Framework for Knowledge Graph Construction

Bowen Zhang, Harold Soh; EMNLP 2024 (main conference); arXiv:2404.03868

EDC is a three-phase framework for building knowledge graphs from text with prompted LLMs and no fine-tuning of the base models. Prior LLM-based KG construction stuffed the whole schema into the prompt, which breaks once the schema outgrows the context window; EDC inverts the order. Phase one runs open information extraction, pulling free-form [subject, relation, object] triplets with no schema in sight. Phase two asks the LLM to write a natural language definition for each relation the extraction induced. Phase three canonicalizes: definitions are embedded with a sentence transformer, vector search proposes the nearest schema candidates, and the LLM verifies whether each substitution is legitimate in context, which curbs the over-generalization that pure embedding clustering suffers. The framework runs in two modes, aligning to a given target schema (rejecting triplets with no equivalent) or growing a canonical schema from empty. An optional refinement pass, EDC+R, feeds prior triplets plus relations fetched by a trained Schema Retriever (a fine-tuned E5-mistral-7b embedder) back into extraction as a hint.

On WebNLG, REBEL, and Wiki-NRE (schemas of 159, 200, and 45 relations, sizes that break prompt-embedded baselines), EDC matches or beats the trained specialists REGEN and GenIE, with GPT-4, GPT-3.5-turbo, and a local Mistral-7b all viable for extraction. Refinement helps everywhere: with GPT-3.5 on WebNLG, Partial F1 climbs from 0.746 to 0.794, and ablating the Schema Retriever gives most of that gain back. In self-canonicalization, human-judged precision drops only from 0.982 to 0.956 on WebNLG while relation count falls from 529 to 200, far ahead of CESI at 0.724. The caveats are real: only relations get canonicalized (entity de-duplication is future work), only the extraction phase was tested across model tiers while everything else ran GPT-3.5, and the pipeline is call-heavy at roughly $0.009 per example even on cheap models.

For the harness, EDC is the closest published shape to the stage where raw dated fact rows and entity mentions must merge into a canonical entity knowledge base. Its define-then-verify canonicalization is directly portable Python: schema-validated extraction calls through swappable model tiers map onto EDC's finding that a local 7B holds up at extraction while verification may need a stronger tier, and its target-alignment rejection of non-matching triplets gives the measured-rejection band published company. Its human-scored precision protocol (agreement 0.94) is a sanity anchor for the LLM-judge scoring that grades what the three injection arms retrieve, though EDC says nothing about segmentation, tree ingestion, or injection ordering; it governs canonicalization alone.

Read from: full text of all 17 pages (body pages 1 to 9 covering abstract, method, experiments, conclusion, and limitations, plus references), extracted via pypdf.

Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena

Lianmin Zheng et al.; NeurIPS 2023 Datasets and Benchmarks Track; 2023; arXiv:2306.05685

This is the systematic study that made “LLM-as-a-judge” a defensible evaluation method rather than a convenience. The authors build two human-preference benchmarks: MT-bench, 80 hand-written multi-turn questions across eight categories (writing, roleplay, extraction, reasoning, math, coding, and two knowledge areas), and Chatbot Arena, a crowdsourced platform where users vote in anonymous head-to-head battles. Against roughly 3K controlled expert votes and 3K crowd votes, they test whether a strong LLM judge reproduces human preference. They define three judging modes (pairwise comparison, single-answer grading, and reference-guided grading) and audit each for failure.

The headline result is that GPT-4 agrees with human experts at 85 percent on non-tie votes, above the 81 percent humans reach with each other, with the caveat that agreement is easiest when models differ sharply in quality; on close pairs it falls toward 70 percent. The bias audit supplies the numbers a practitioner needs. Position bias is severe: Claude-v1 was order-consistent in only 23.8 percent of cases and only GPT-4 exceeded 60 percent, so the paper’s conservative fix is to judge both orderings and count disagreement as a tie. A “repetitive list” verbosity attack fooled Claude-v1 and GPT-3.5 on 91.3 percent of padded answers versus 8.7 percent for GPT-4. Self-enhancement bias is observed (GPT-4 favors itself by about 10 percent win rate) but the authors state their data cannot confirm it as causal. Judges also fail on math they can solve in isolation; a reference-guided prompt, where the judge answers first and grades against its own answer, cut the failure rate from 14 of 20 to 3 of 20. Single-answer grading tracked pairwise results closely, which licenses the cheaper mode. The study measures helpfulness only, folding accuracy, relevance, and creativity into one score.

For the memory harness, this paper governs the final scoring stage: after the corpus is segmented into a tree of entities and dated facts, and after answers are produced under the three injection arms through schema-validated calls at each model tier, an LLM judge scores those answers against gold. The design consequences are direct. The judge grades against a fixed reference answer (the setting where reference-guided grading nearly eliminated math failures), positions get swapped or a stable single-answer rubric is used, verbose answers are treated as suspect, and the judge model stays pinned rather than varying with the arm. The 80 to 85 percent agreement figure is the calibration target the hand-labeled set should match, while the near-random consistency of weaker judges warns against scoring with the same small local models being tested.

Read from: full main body, pages 1 to 10 of the arXiv v4 PDF (abstract through conclusion; appendices and references not read), via pypdf text extraction.

A Comprehensive Survey on Automatic Knowledge Graph Construction

Zhong, Wu, Li, Peng, and Wu; ACM Computing Surveys 56(4), 2024; arXiv:2302.05019

This is a survey of more than 300 methods for building knowledge graphs automatically from raw data. The authors organize construction into three stages: knowledge acquisition (entity discovery, coreference resolution, relation extraction), knowledge refinement (graph completion and fusion with other graphs), and knowledge evolution (conditional and temporal knowledge, dynamics over time). They define a knowledge graph as G = {E, R, T, F_k}, where T is a set of triples (head, tail, relation) and F_k is background knowledge, a rule set or schema that constrains which facts count as knowledge rather than mere data; a graph without this component, they argue, is just a data graph. The survey frames its coverage with the HACE view of big data (heterogeneous, autonomous, complex, evolving) and compares itself against nine prior surveys, claiming broader coverage of conditions, coreference, and semi-structured sources.

The construction-from-text sections lay out a stable pipeline vocabulary. Entity discovery splits into named entity recognition, fine-grained entity typing, and entity linking (disambiguation against existing nodes, creating a node when none exists). Coreference resolution runs as mention detection followed by antecedent scoring, with the end-to-end span-ranking model of Lee et al. as the standard deep architecture. Relation extraction divides by configuration: open extraction without a schema, schema-bound classification, distant supervision with its noise problem and the at-least-one multi-instance assumption, plus document-level, few-shot, and joint variants. For each task the survey tracks the same progression, from rules and wrappers through statistical models (CRFs, SVMs) to CNN, LSTM, attention, GCN, and pre-trained-language-model designs. It also catalogs practical resources: about two dozen KG projects with sizes (Wikidata at 96M+ items, YAGO at 2B+ facts, ConceptNet at 34M+ items) and tools from spaCy and OpenNRE to the end-to-end gBuilder.

For a personal knowledge backend built over conversational history, the useful contributions are the pipeline decomposition itself (a memory system doing entity linking and coreference over chat logs is running exactly these tasks), the insistence that a schema or background knowledge separates a knowledge graph from a data graph, and the storage discussion (relational, key/value, and native graph stores such as Neo4j and RDF systems). The open problems the authors flag map directly onto that setting: long intricate contexts requiring coreference, logic, and commonsense reasoning; knowledge dynamics as facts change; and federated learning for privacy when scaling to an organization, where encrypted entity alignment remains unsolved. The survey predates LLM-based extraction, so its method coverage stops at BERT-era models.

Read from: pages 1-20 (introduction, definitions, resources, preprocessing, entity discovery, coreference, relation extraction through distant supervision) and pages 33-37 (knowledge dynamics, storage, discussion, conclusion); skipped pages 21-32 (few-shot, document-level, and joint extraction; knowledge refinement; most of knowledge evolution) and the references.

UniversalNER: Targeted Distillation from Large Language Models for Open Named Entity Recognition

Wenxuan Zhou, Sheng Zhang, Yu Gu, Muhao Chen, Hoifung Poon; ICLR 2024; arXiv:2308.03279

The paper asks whether a small model can match a frontier model on one broad task class rather than on everything, and answers with what it calls targeted distillation through mission-focused instruction tuning. Instead of diversifying instructions the way Alpaca and Vicuna do, the authors diversify inputs: they sample 50,000 passages of at most 256 tokens from the Pile's 22 subcorpora, have ChatGPT (gpt-3.5-turbo-0301, temperature 0) tag every entity and its type with no type list supplied, and after filtering obtain 45,889 passage-annotation pairs covering 240,725 entities and 13,020 distinct entity types. LLaMA 7B and 13B are then tuned on a conversation-style template that asks one entity type per query and expects a JSON list back, with negative entity types sampled in proportion to corpus frequency so the model learns to return empty lists.

To evaluate, they assemble the largest open NER benchmark to date: 43 datasets across 9 domains (biomedicine, clinical, programming, social media, law, finance, and others), scored by strict entity-level micro F1. Zero-shot, UniNER-7B reaches 41.7 average F1 and UniNER-13B 43.4, against 34.9 for ChatGPT itself and roughly 14 for Vicuna-7B; generalist distillates like Alpaca sit near zero, over 30 points behind. The ablations carry as much information as the headline: frequency-based negative sampling beats no sampling by 21.9 F1, asking all types in one query costs 3.3 F1 versus one type per query, and definition-style type labels cost 11.8 on standard benchmarks while improving tolerance to type paraphrasing. With supervised finetuning added, UniNER-7B hits 84.78 average F1 on 20 datasets, above InstructUIE-11B at 81.16. The scope is real but narrow in the same breath: English only, NER only, a 2023-era teacher, and even the winning zero-shot average of 41.7 says open-type extraction remains far from solved.

For the memory harness, this paper governs the entity extraction stage of the corpus-to-tree pipeline, where schema-validated LLM calls pull entities and dated facts into the KB before the three injection arms and LLM-judge scoring ever run. It calibrates what the swappable cheap tier can be expected to do there: a 7B-class model can plausibly own extraction, but only a specialized one; a generalist small model is the named failure mode, trailing by about 30 F1. Two of its mechanisms transfer directly without any distillation of our own: one-entity-type-per-call prompting over all-in-one requests, and deliberate negative queries so the extractor learns that an empty list is a valid answer.

Read from: extracted text of pages 1 through 13 of the 19-page PDF, covering the full main paper (introduction through conclusion on page 9) plus part of the references; the remaining pages are references and appendix tables.